

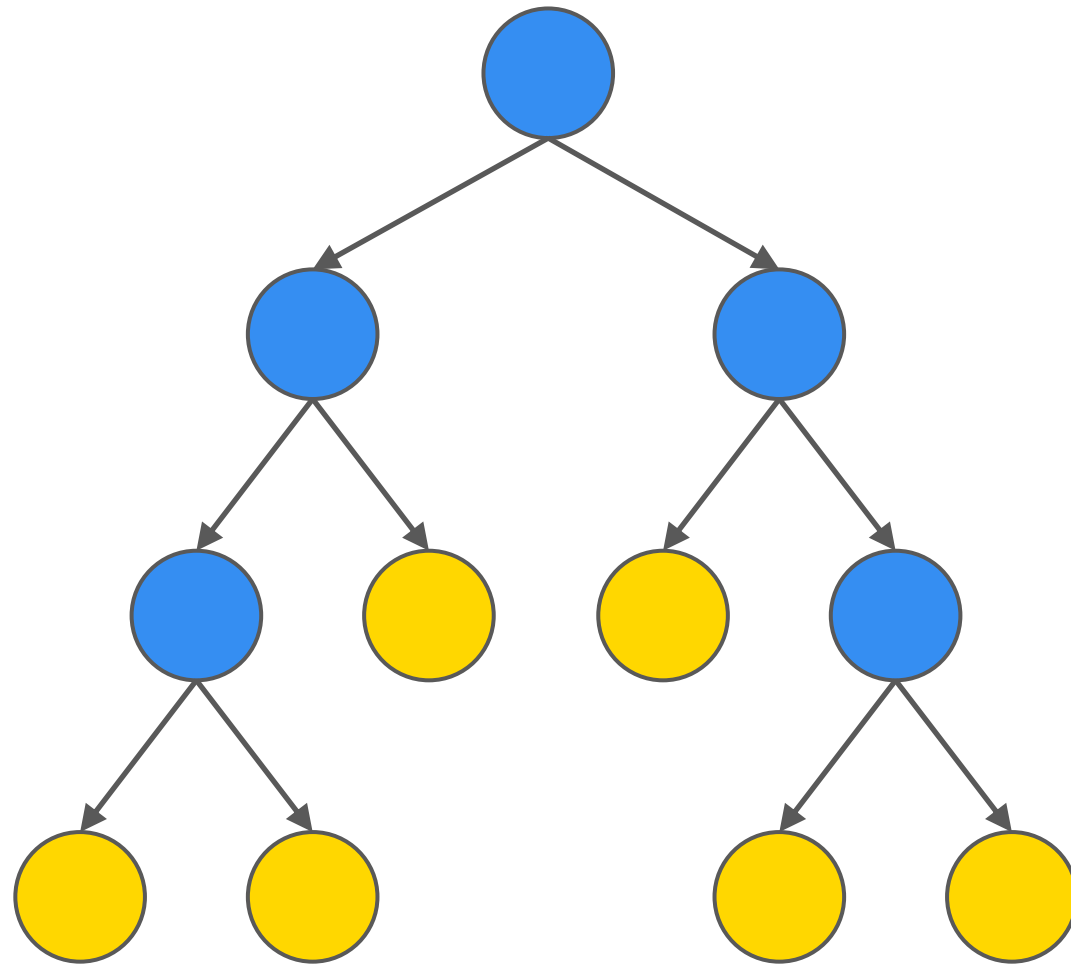
Inteligência Artificial



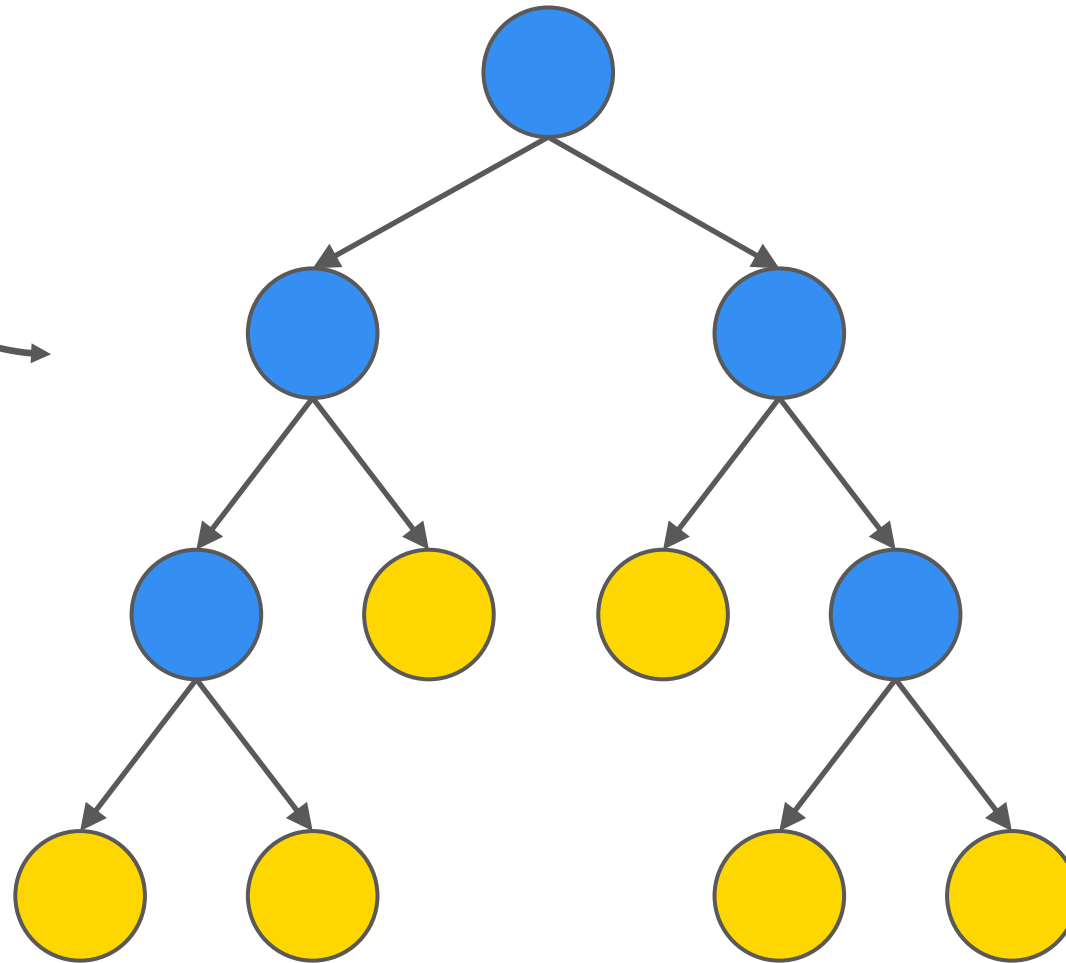
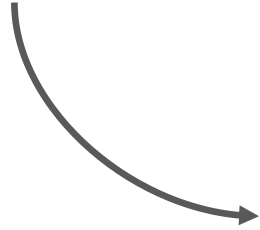
Decision Trees

Prof. Dr. Samuel Martins (Samuka)
samuel.martins@ifsp.edu.br

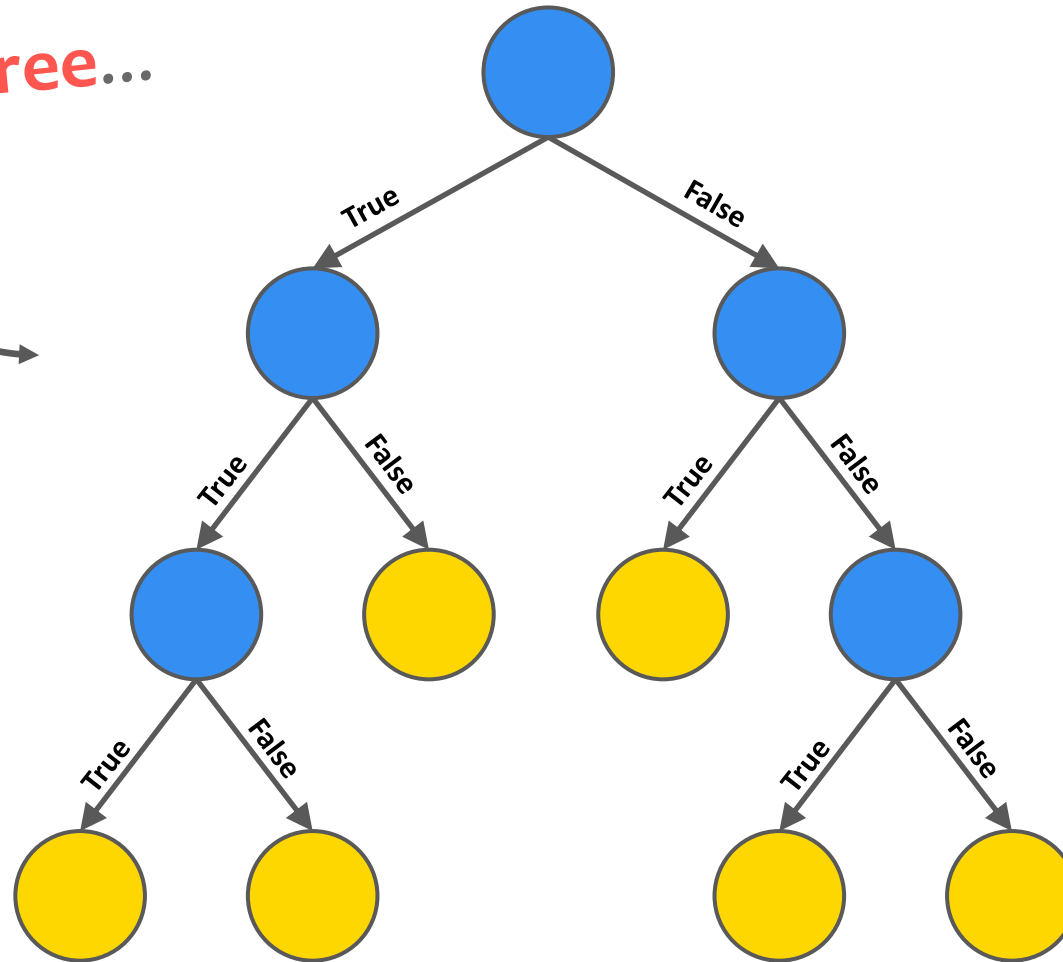




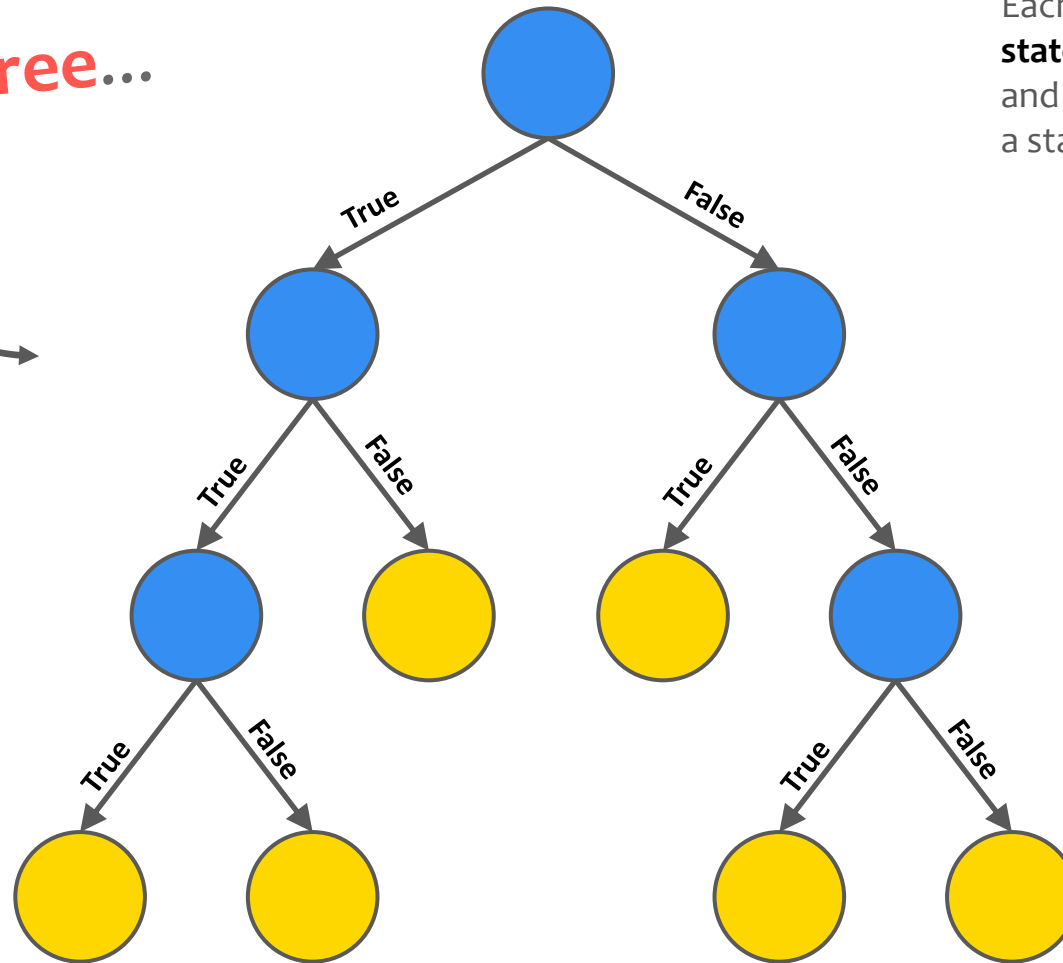
This is a **Tree**...



This is a **Decision Tree**...

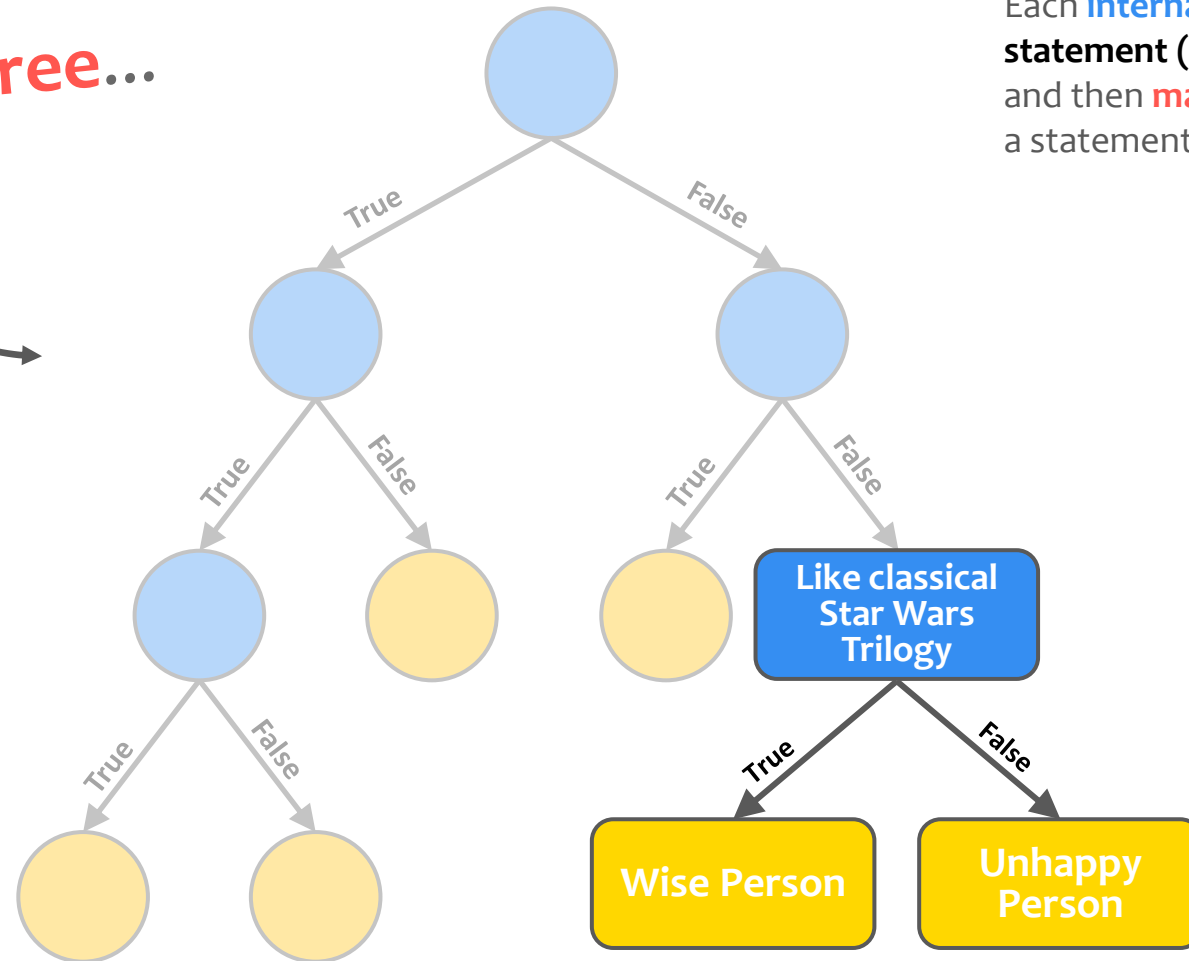
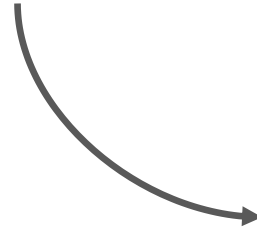


This is a **Decision Tree**...



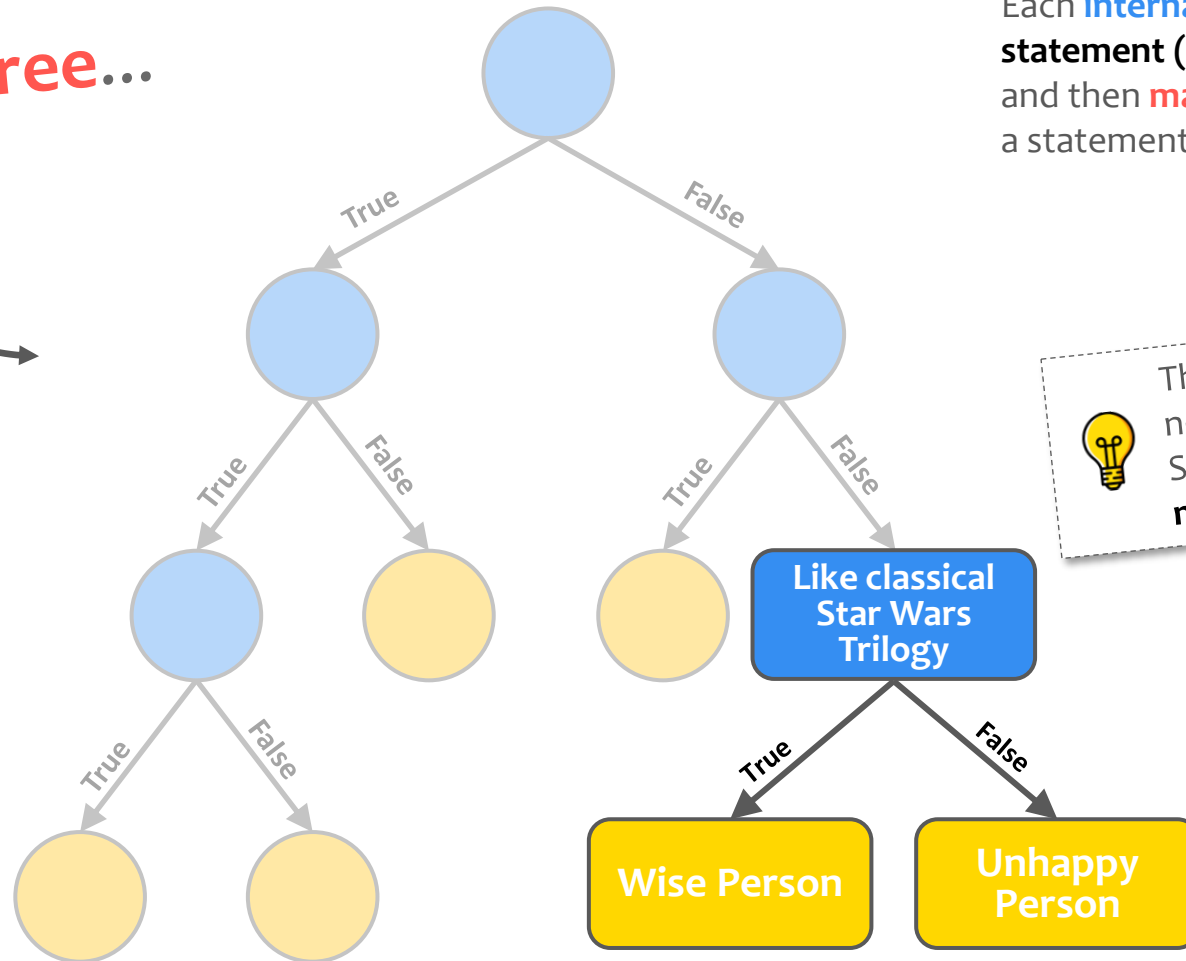
Each **internal (blue) node** makes a **statement (rule)** about a **feature** and then **makes a decision** on such a statement (two or more options).

This is a **Decision Tree**...



Each **internal (blue) node** makes a **statement (rule)** about a **feature** and then **makes a decision** on such a statement (two or more options).

This is a **Decision Tree**...



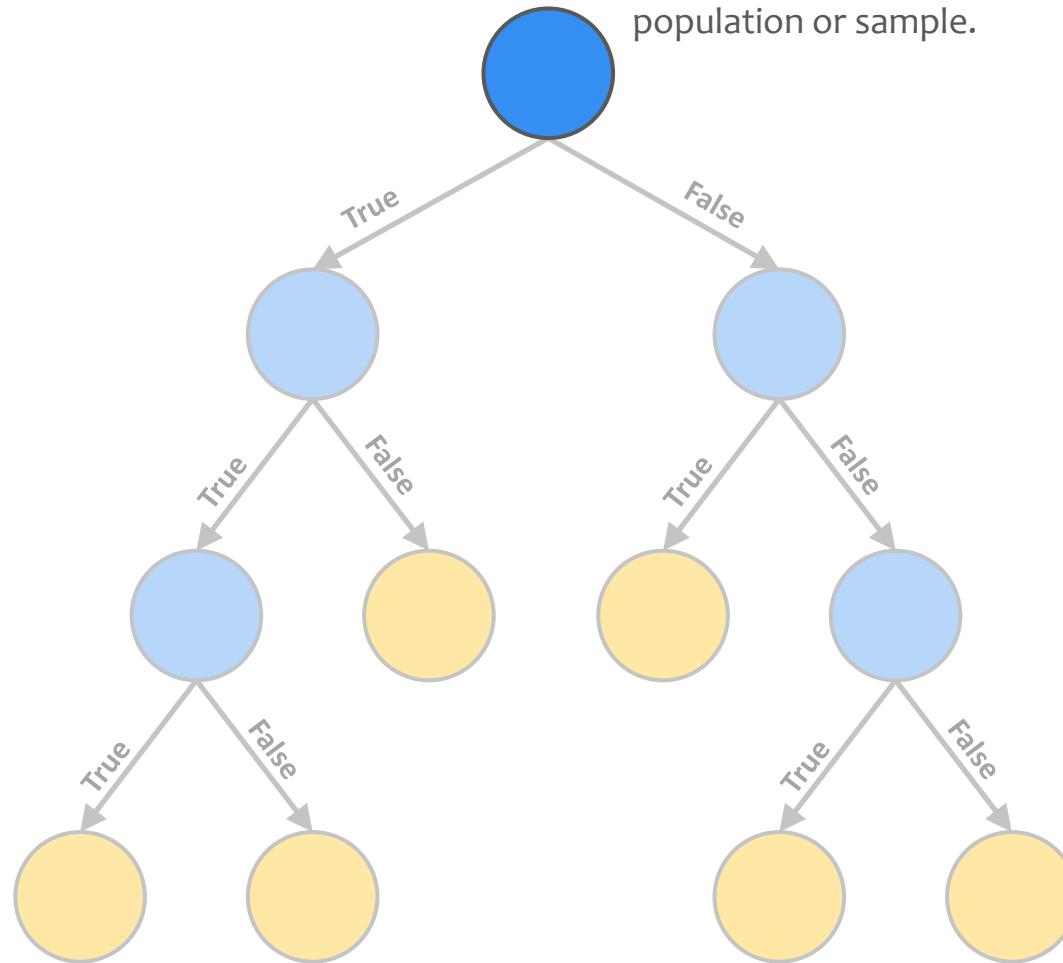
Each **internal (blue) node** makes a **statement (rule)** about a **feature** and then **makes a decision** on such a statement (two or more options).



The **decision** may not be necessarily **binary**.
Some algorithms provide more than 2 options.

1. Root Node

It represents the entire population or sample.

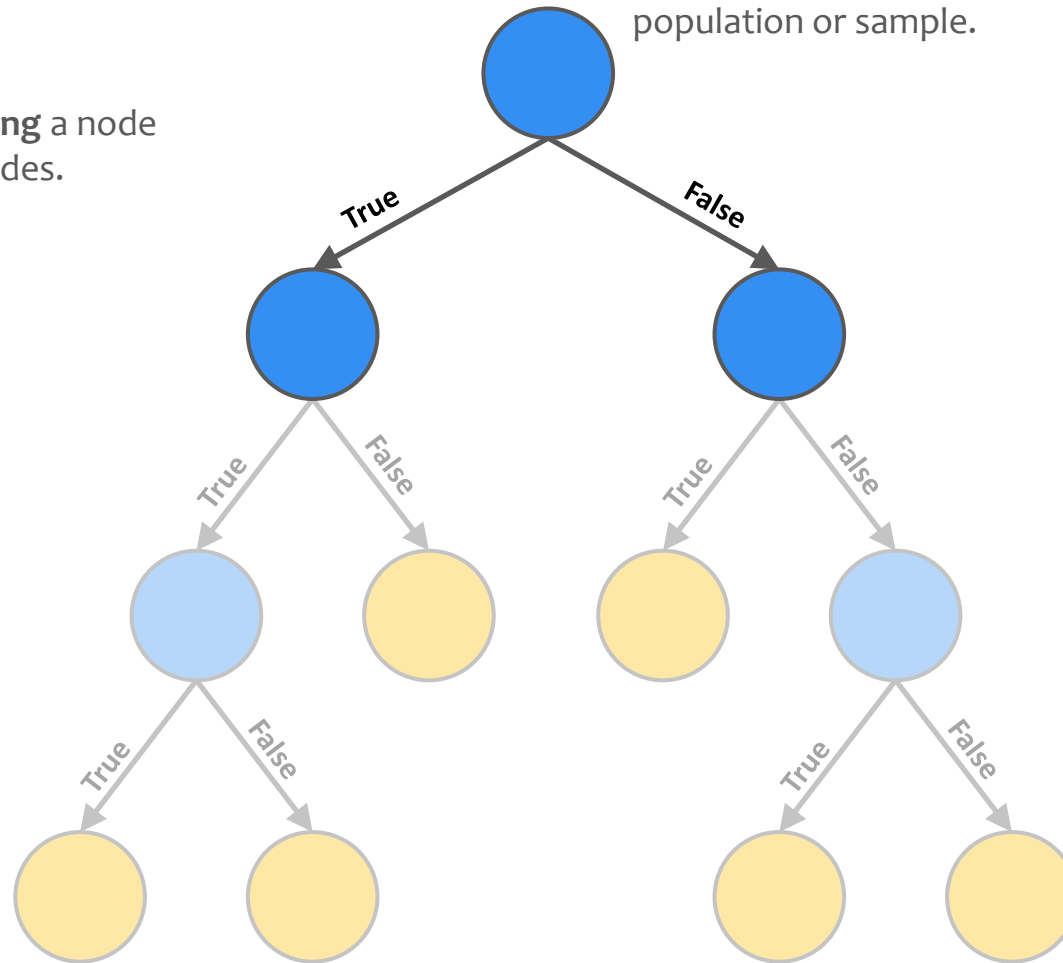


1. Root Node

It represents the entire population or sample.

2. Splitting

It is the process of **dividing** a node into two or more sub-nodes.



1. Root Node

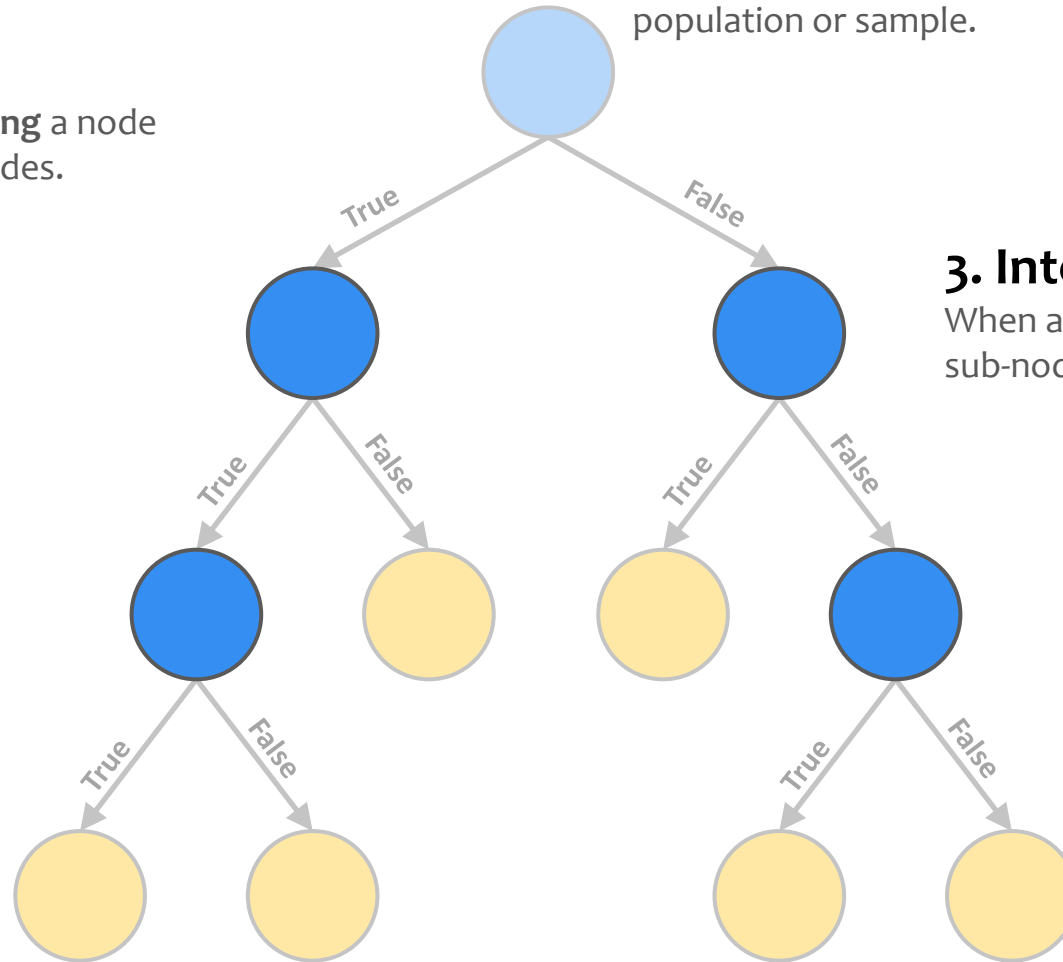
It represents the entire population or sample.

2. Splitting

It is the process of **dividing** a node into two or more sub-nodes.

3. Internal/Decision Nodes

When a sub-node splits into further sub-nodes, it is called a decision node.



1. Root Node

It represents the entire population or sample.

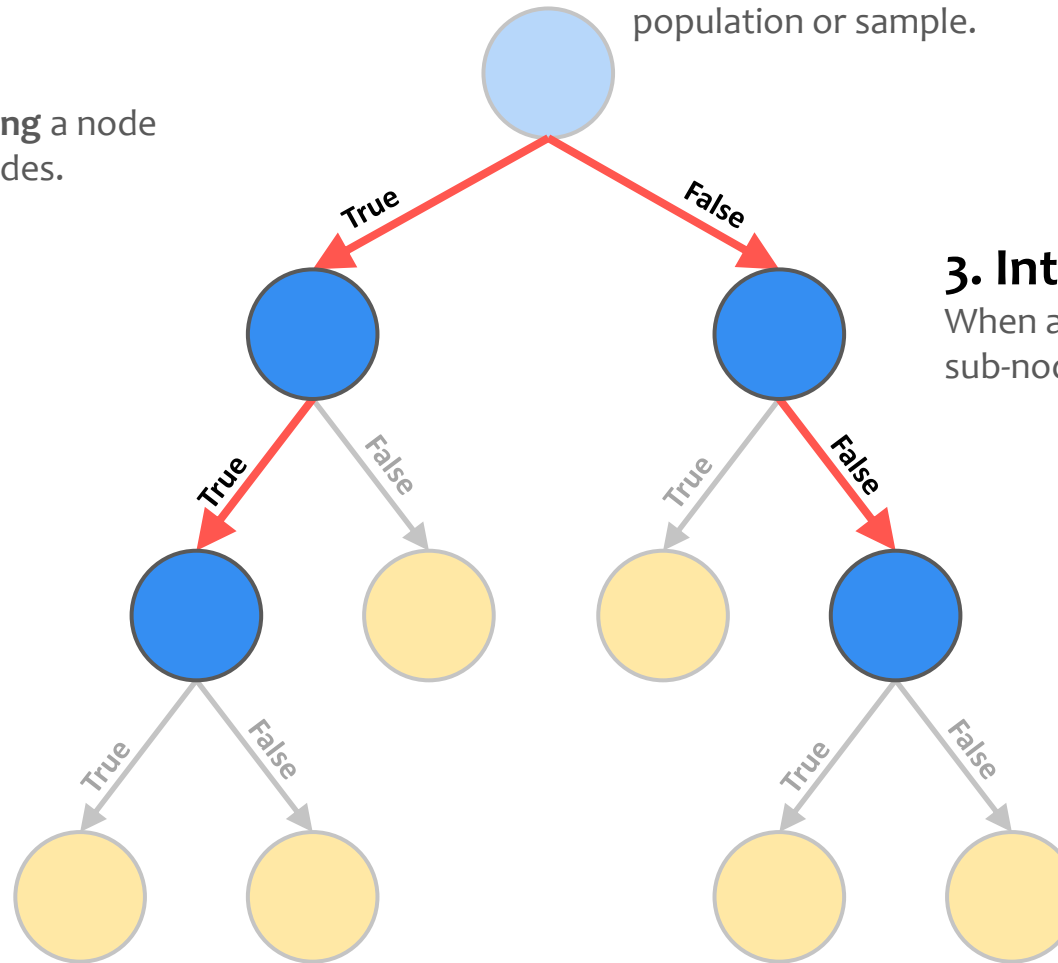
2. Splitting

It is the process of **dividing** a node into two or more sub-nodes.

3. Internal/Decision Nodes

When a sub-node splits into further sub-nodes, it is called a decision node.

They have **arrows** pointing to them...



1. Root Node

It represents the entire population or sample.

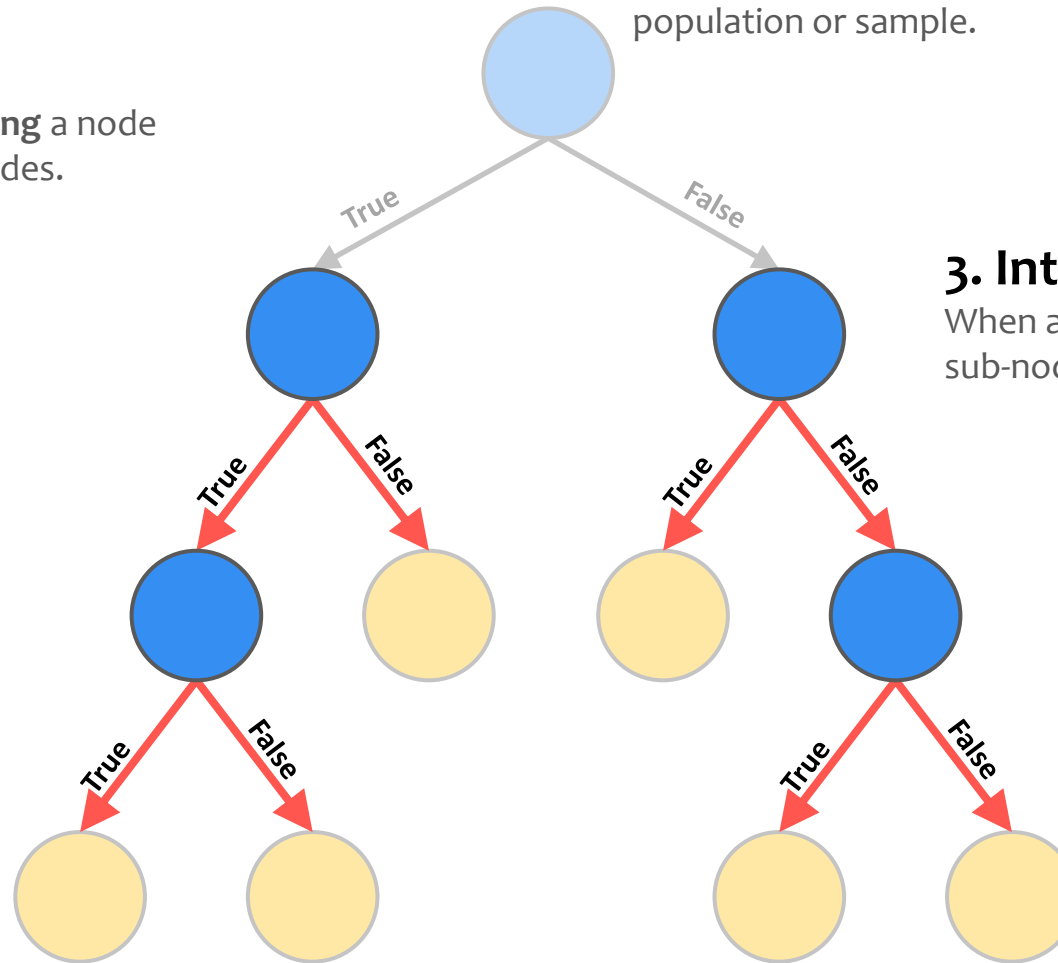
2. Splitting

It is the process of **dividing** a node into two or more sub-nodes.

3. Internal/Decision Nodes

When a sub-node splits into further sub-nodes, it is called a decision node.

...and they have **arrows** pointing away from them.



1. Root Node

It represents the entire population or sample.

2. Splitting

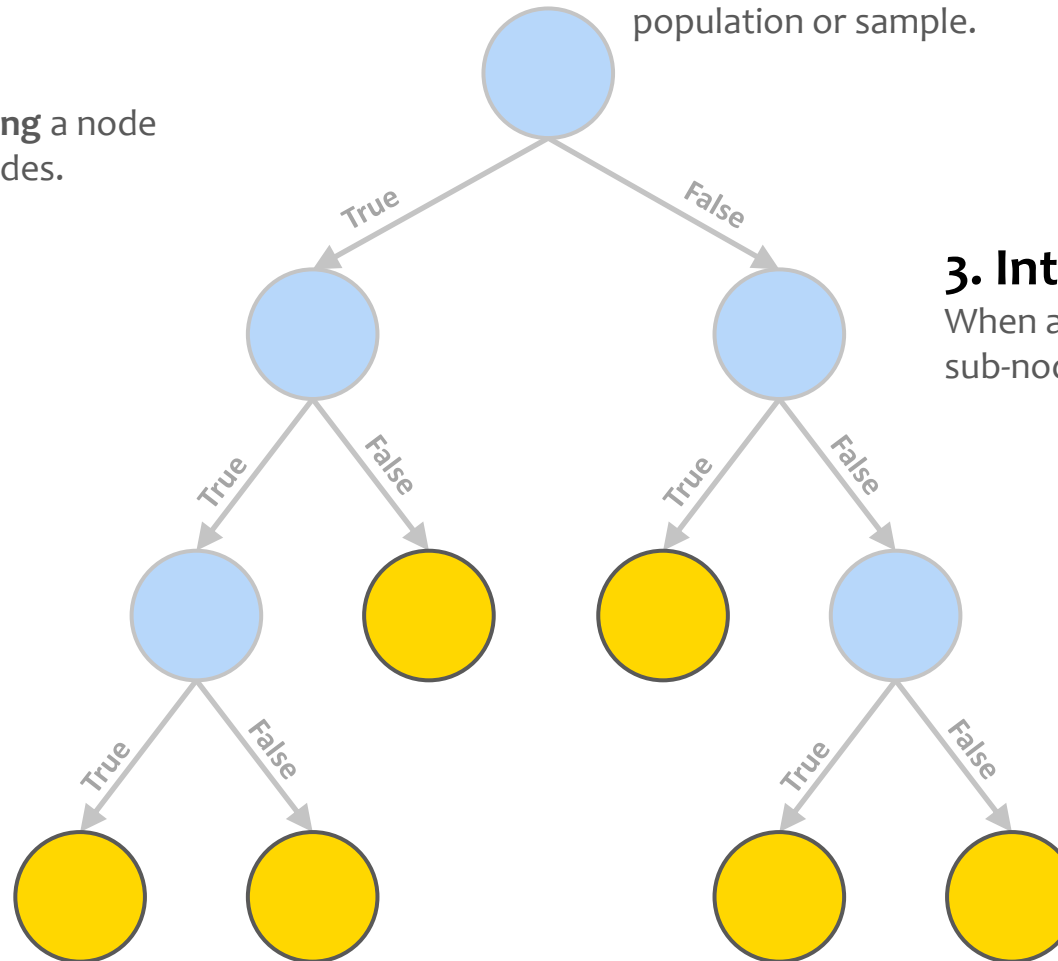
It is the process of **dividing** a node into two or more sub-nodes.

3. Internal/Decision Nodes

When a sub-node splits into further sub-nodes, it is called a decision node.

4. Leaf/Terminal Nodes

Nodes that do not split.



1. Root Node

It represents the entire population or sample.

2. Splitting

It is the process of **dividing** a node into two or more sub-nodes.

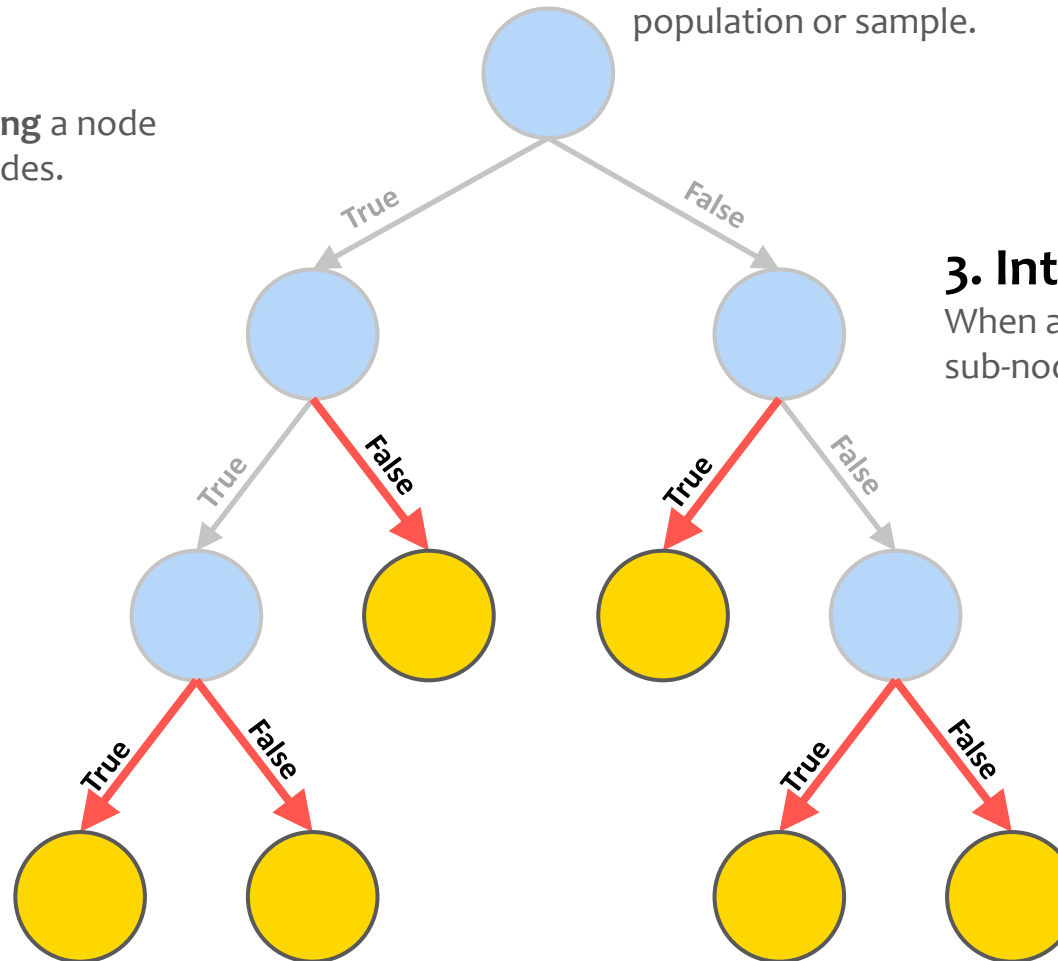
3. Internal/Decision Nodes

When a sub-node splits into further sub-nodes, it is called a decision node.

4. Leaf/Terminal Nodes

Nodes that do not split.

They have **arrows** pointing to them but no arrows pointing away from them.



1. Root Node

It represents the entire population or sample.

2. Splitting

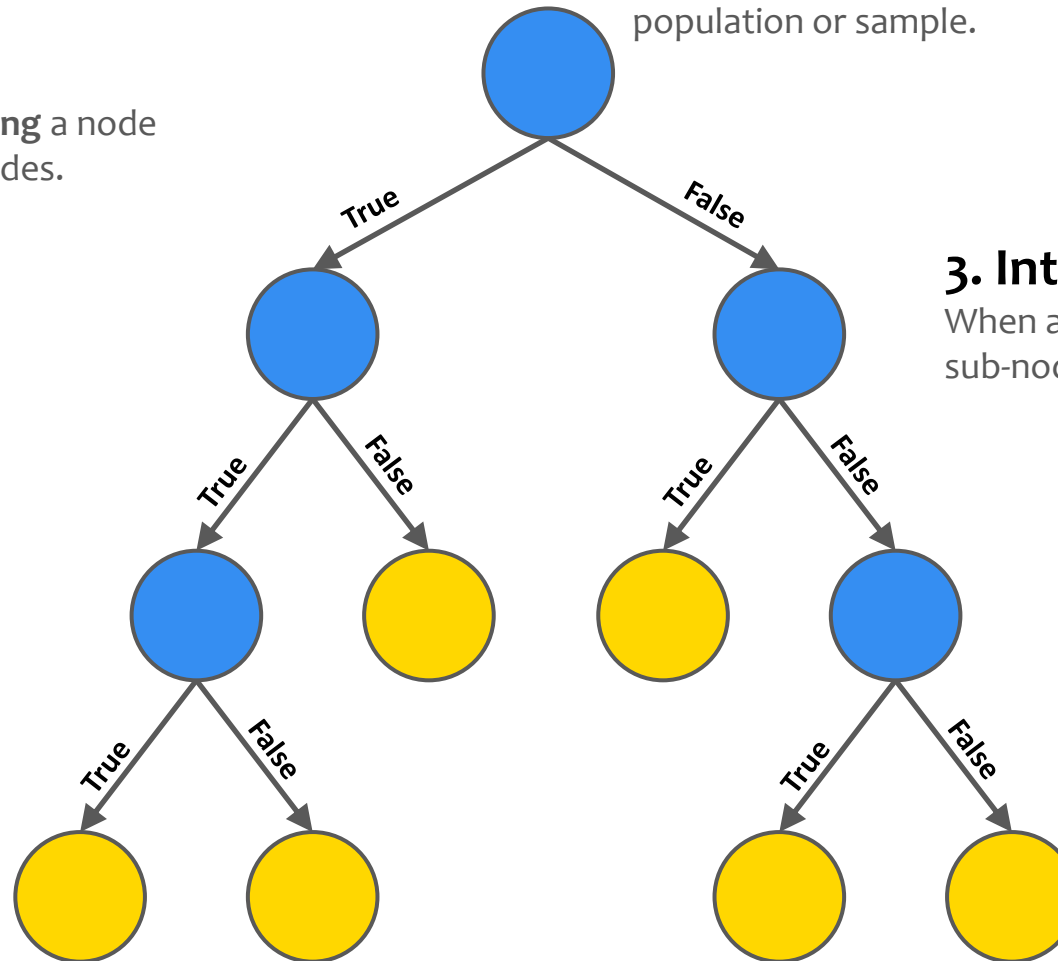
It is the process of **dividing** a node into two or more sub-nodes.

3. Internal/Decision Nodes

When a sub-node splits into further sub-nodes, it is called a decision node.

4. Leaf/Terminal Nodes

Nodes that do not split.



1. Root Node

It represents the entire population or sample.

2. Splitting

It is the process of **dividing** a node into two or more sub-nodes.

3. Internal/Decision Nodes

When a sub-node splits into further sub-nodes, it is called a decision node.

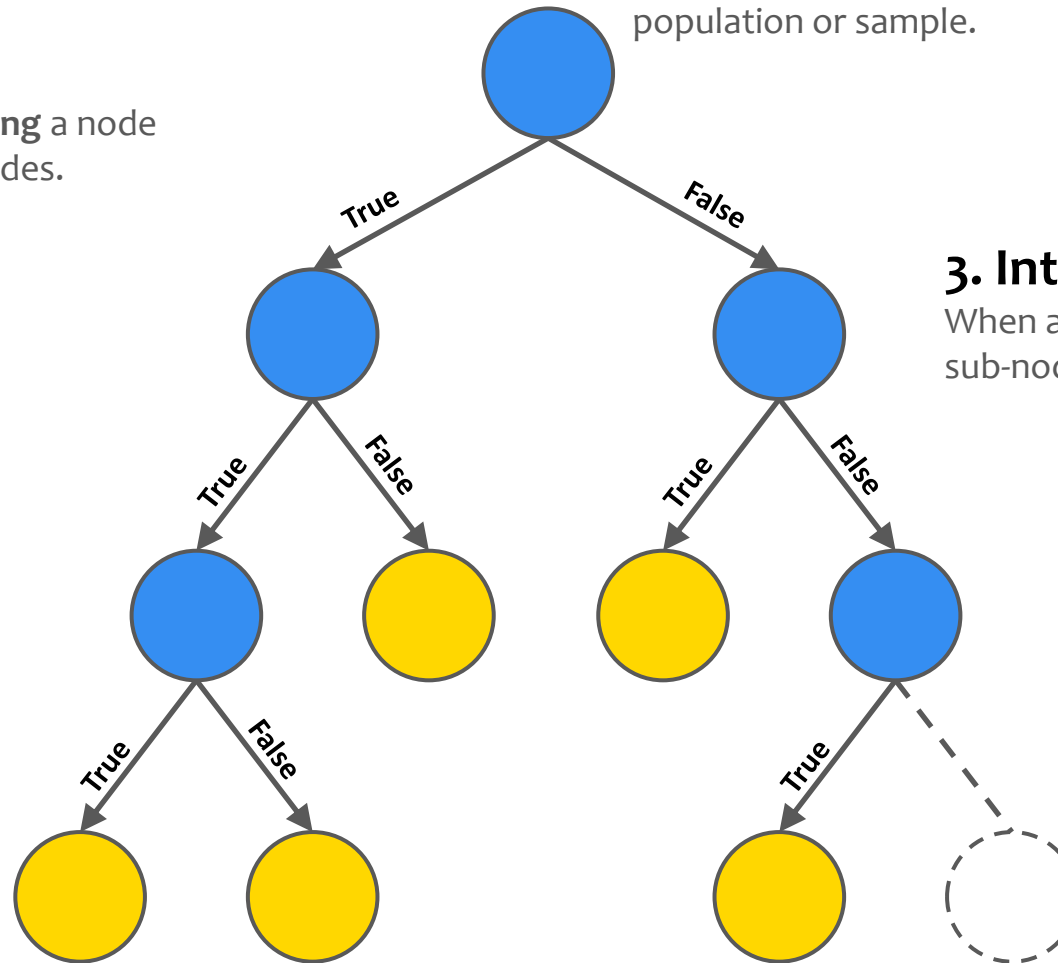
4. Leaf/Terminal Nodes

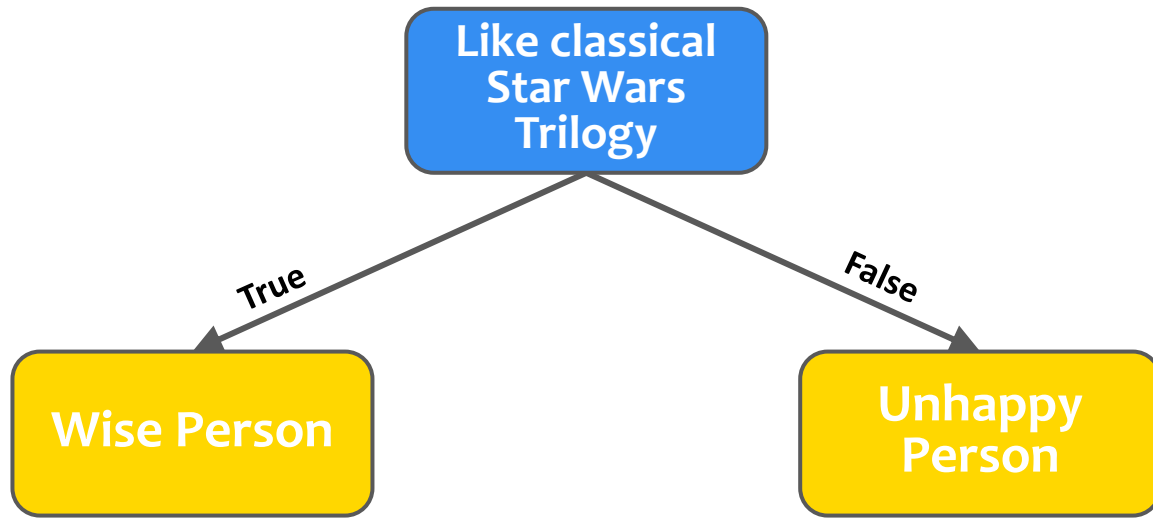
Nodes that do not split.

5. Pruning

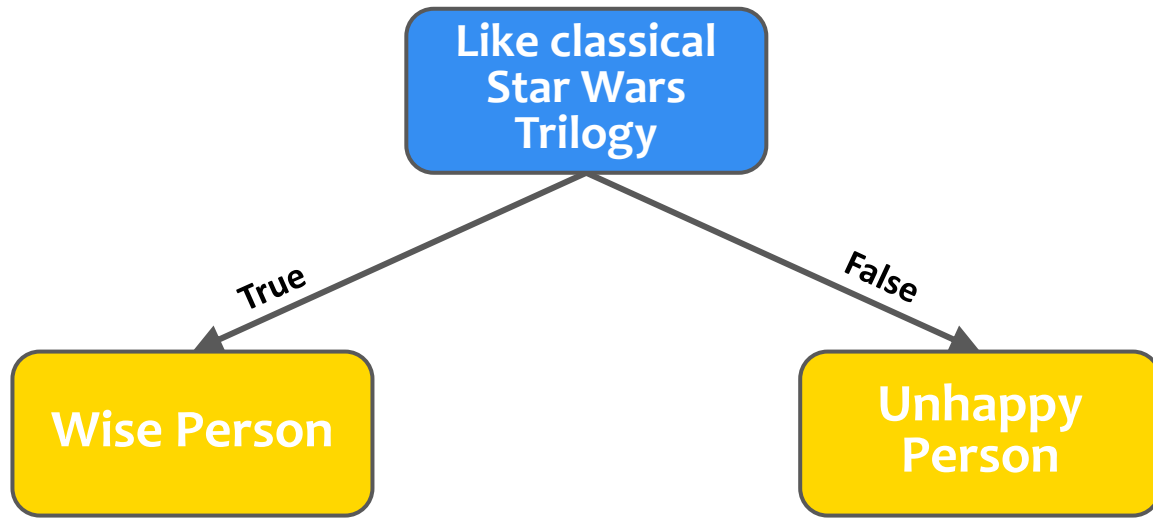
Process of removing sub-nodes of a decision node.

Some algorithms rely on *pruning* to delete unnecessary nodes.

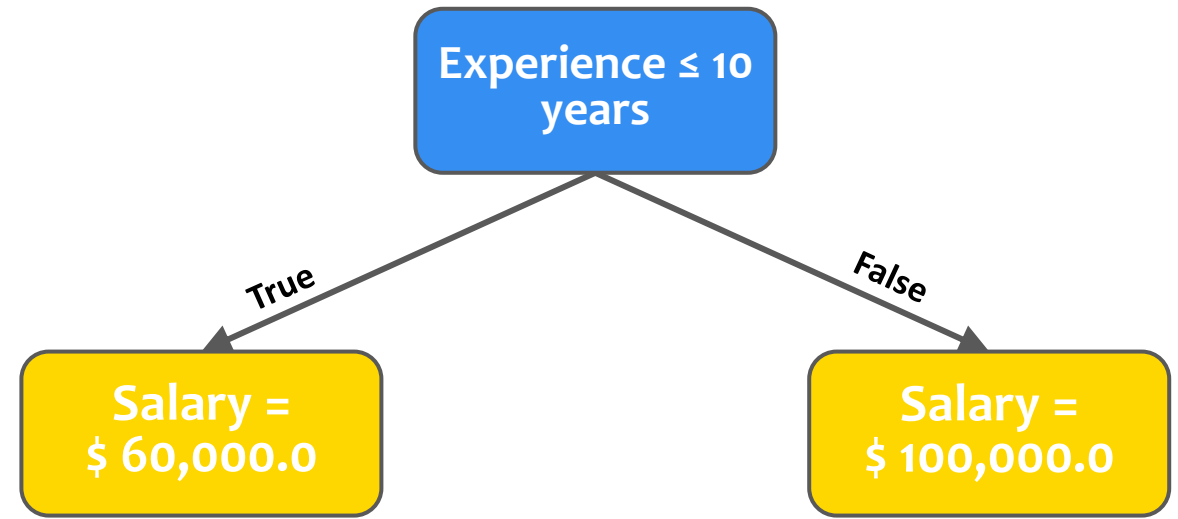




When a **Decision Tree** estimates **categories**, it is called a **Classification Tree**.

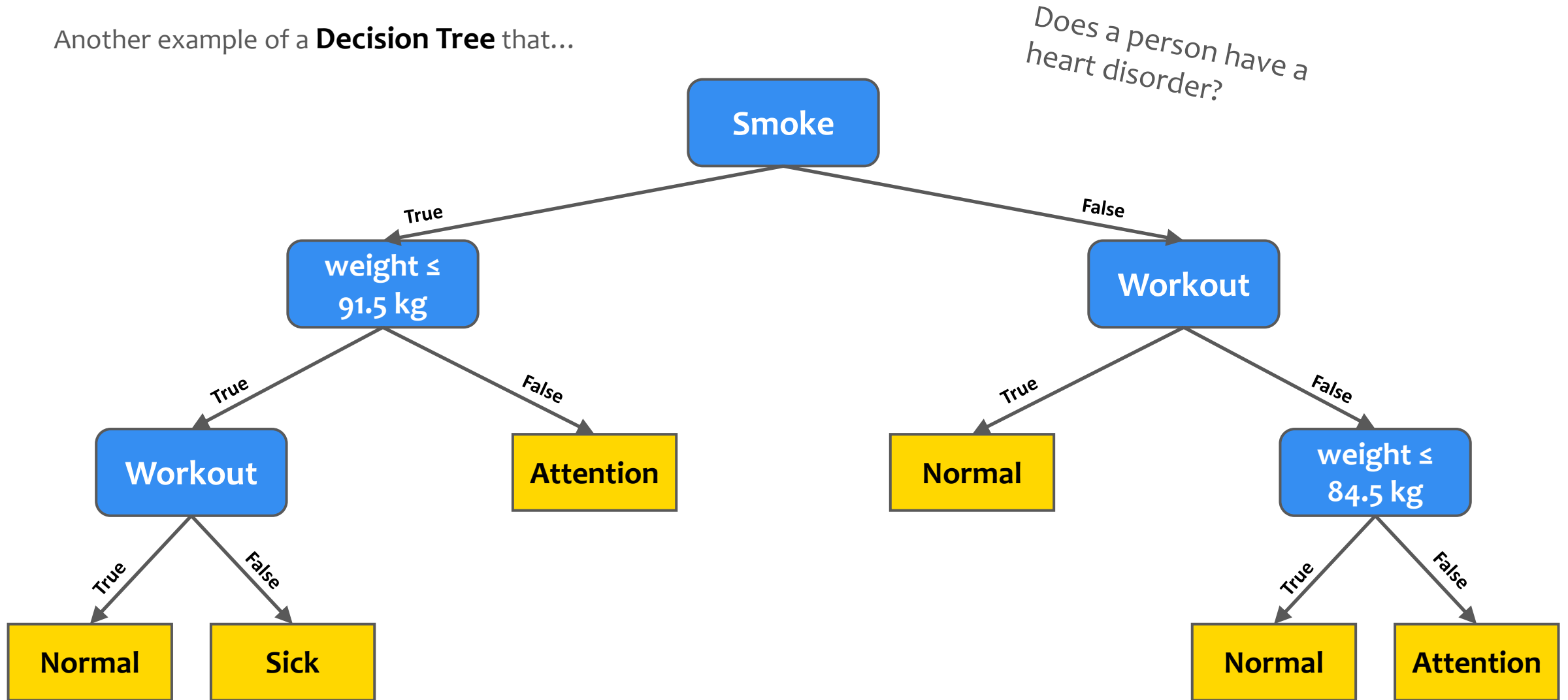


When a **Decision Tree** estimates **categories**, it is called a **Classification Tree**.

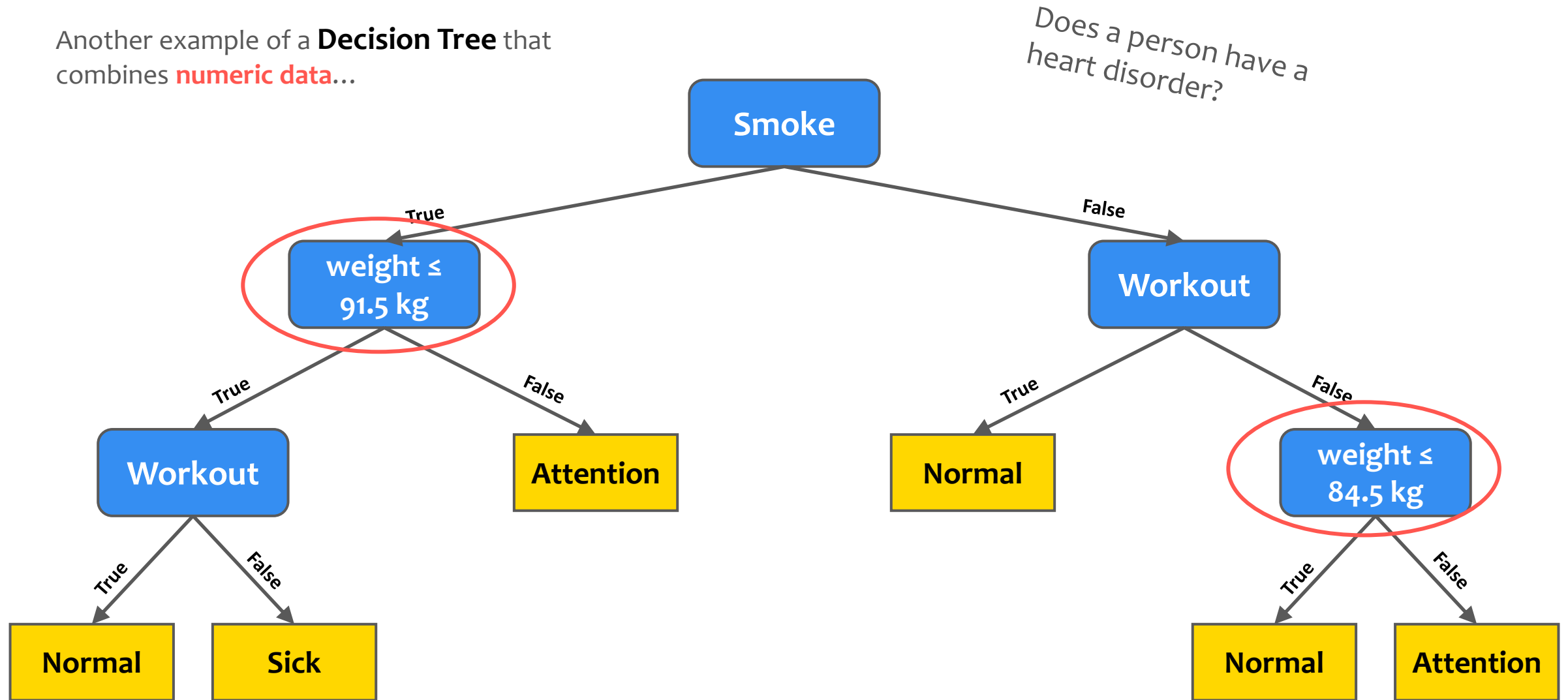


When a **Decision Tree** predicts **numeric values**, it is called a **Regression Tree**.

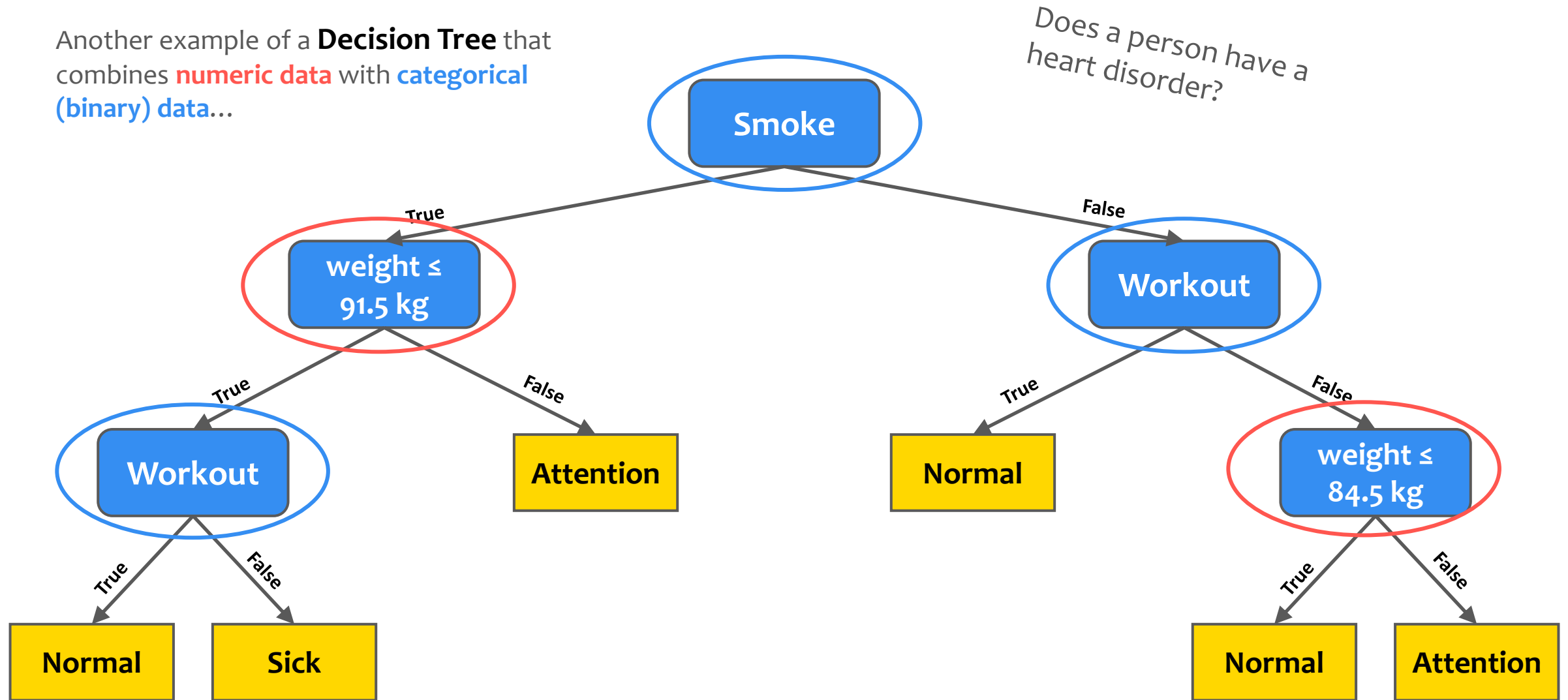
Another example of a **Decision Tree** that...



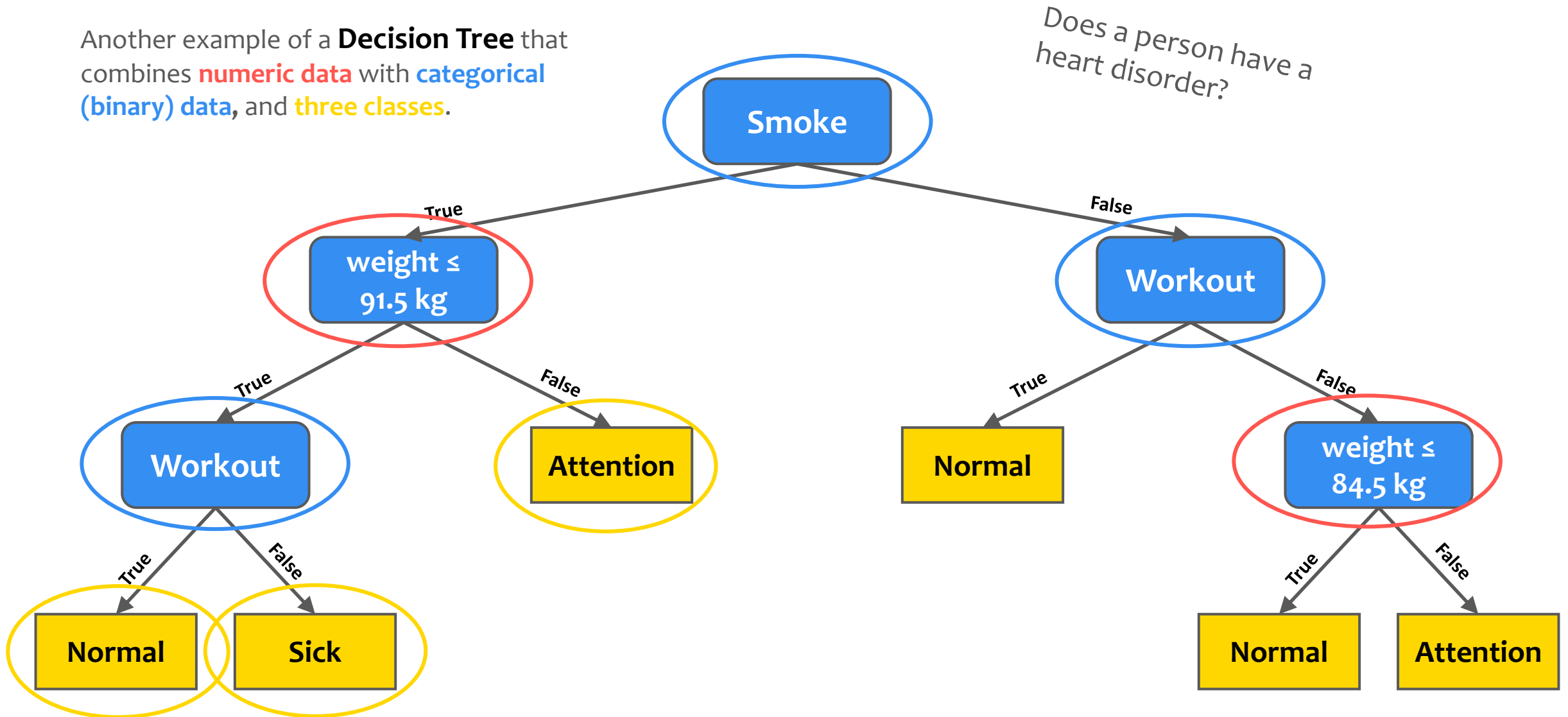
Another example of a **Decision Tree** that combines **numeric data**...



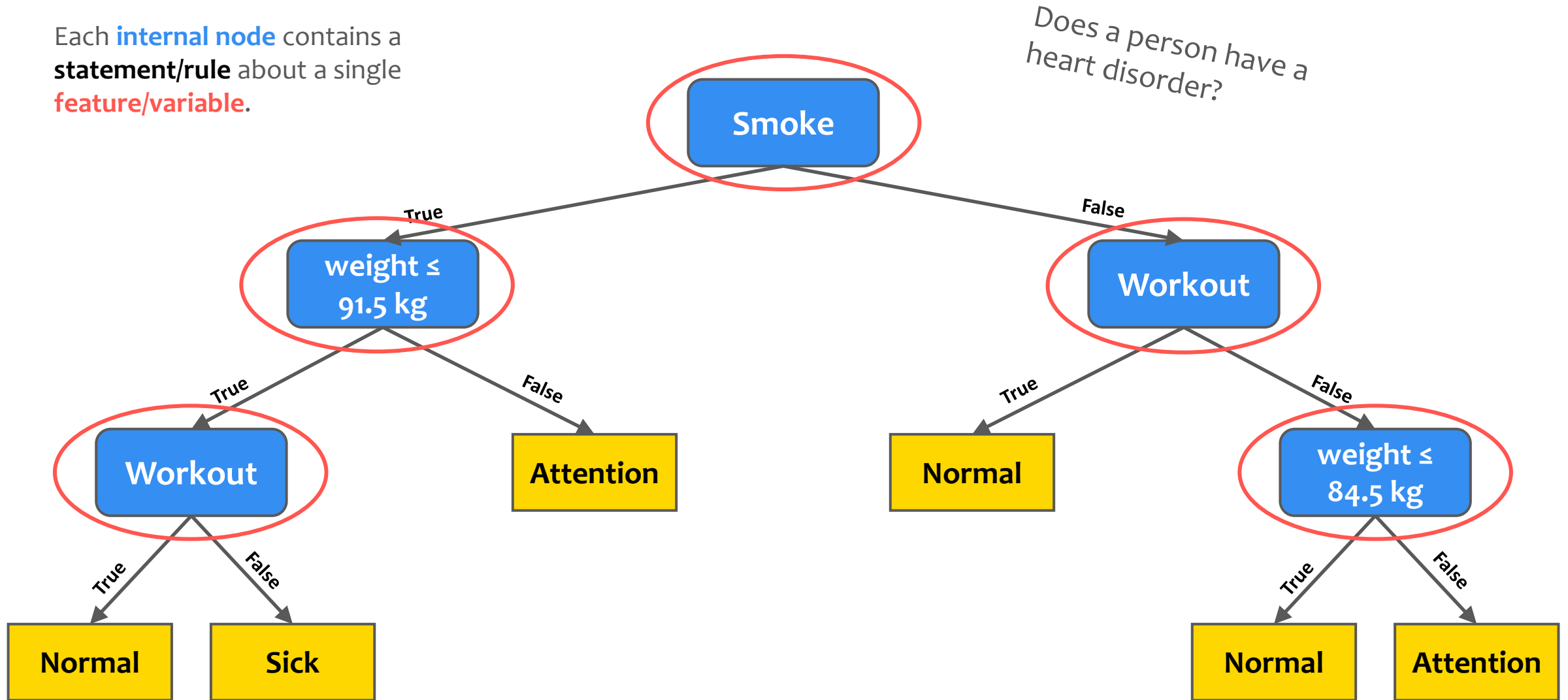
Another example of a **Decision Tree** that combines **numeric data** with **categorical (binary) data**...



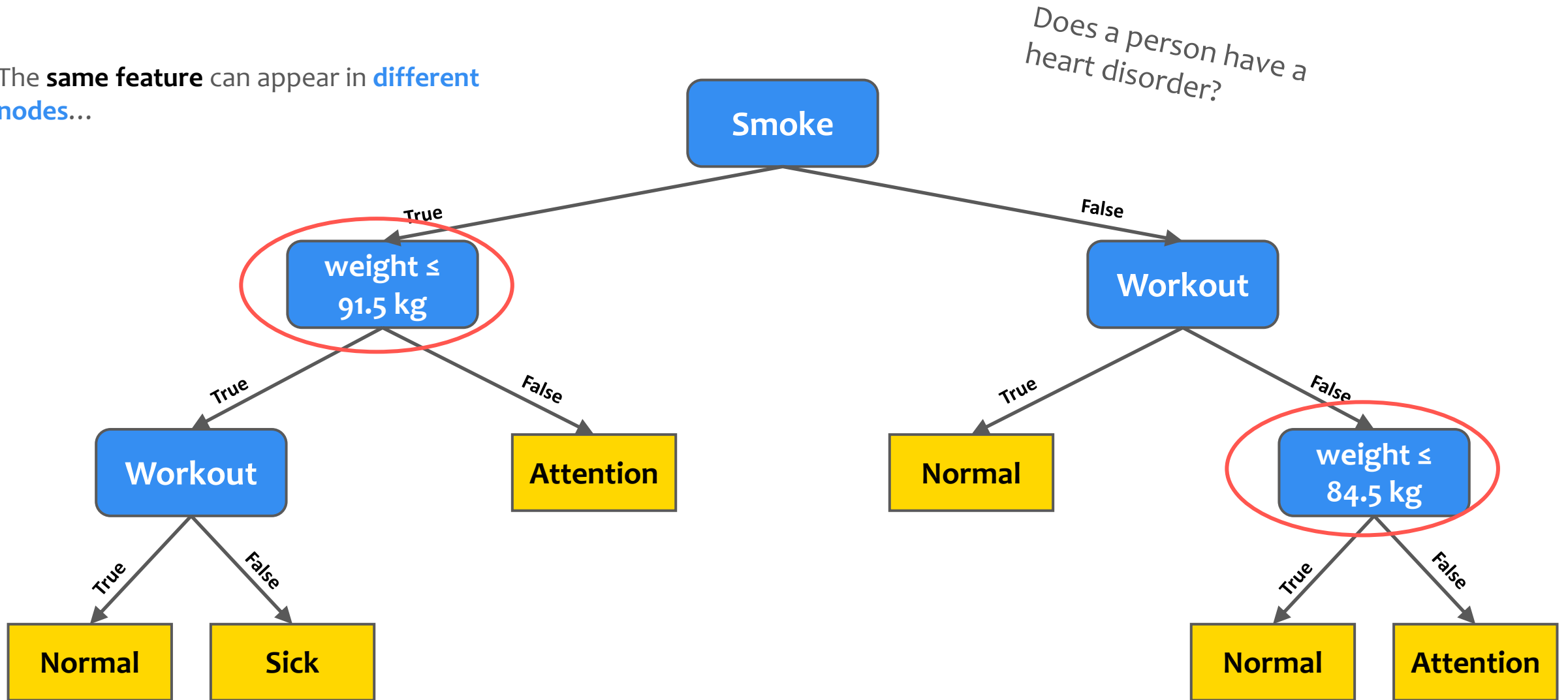
Another example of a **Decision Tree** that combines **numeric data** with **categorical (binary) data**, and **three classes**.



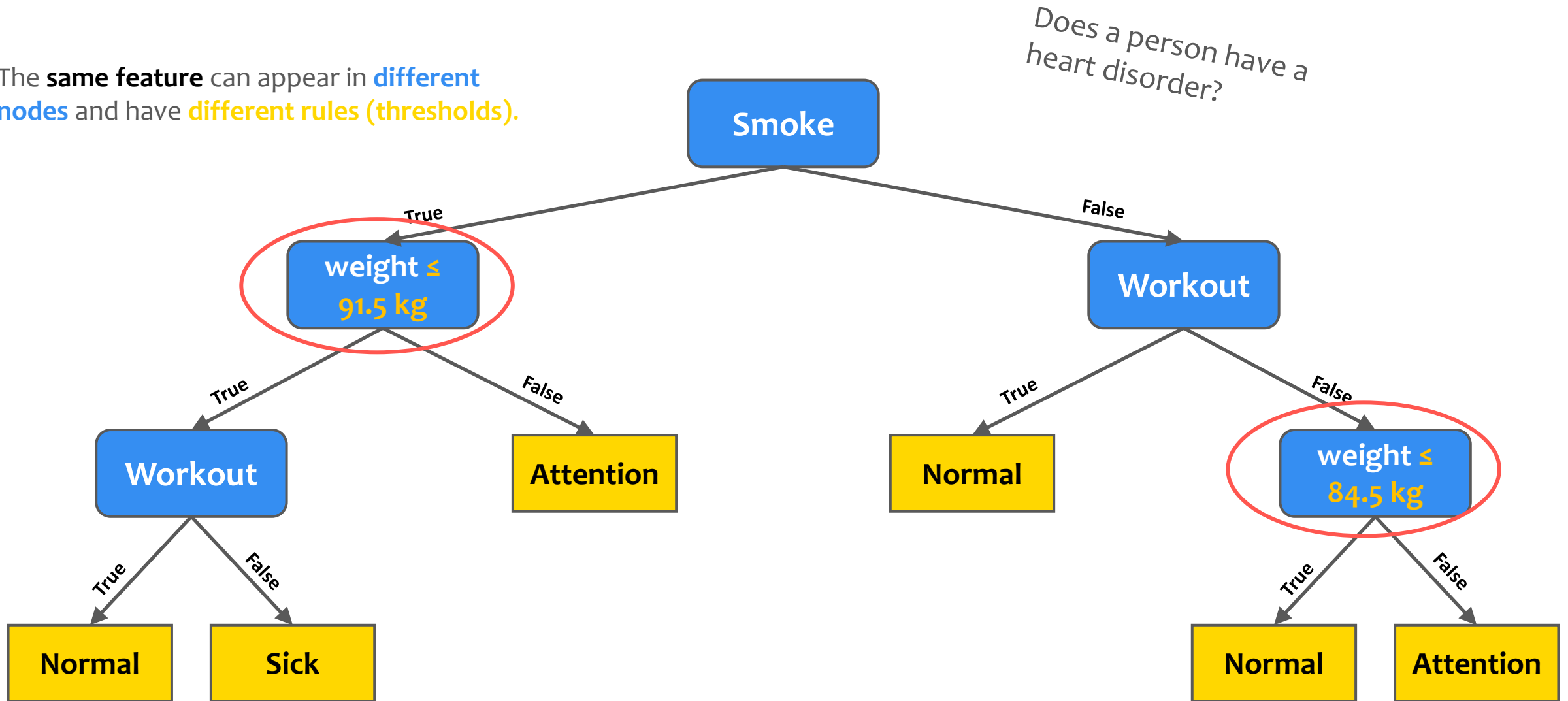
Each **internal node** contains a **statement/rule** about a single **feature/variable**.



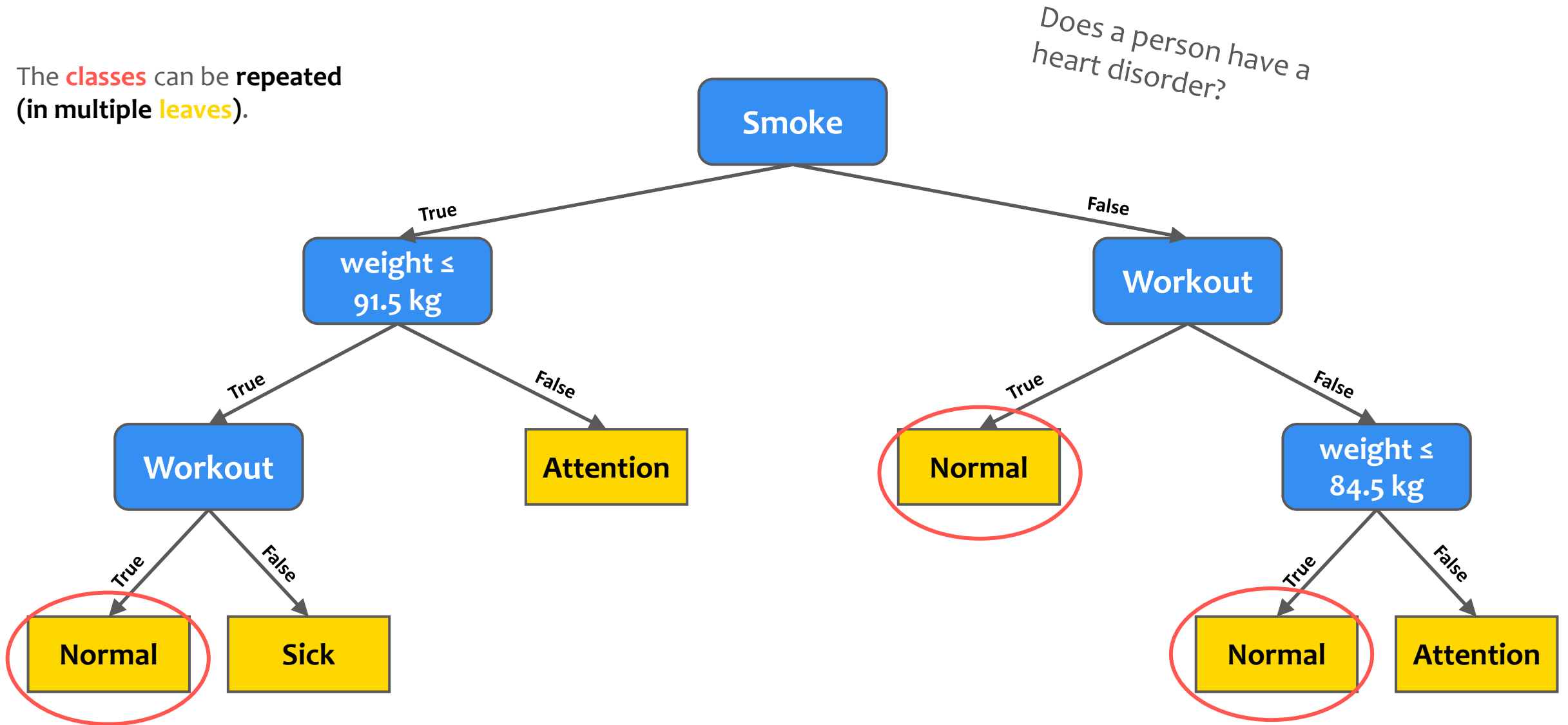
The **same feature** can appear in **different nodes**...



The **same feature** can appear in **different nodes** and have **different rules (thresholds)**.

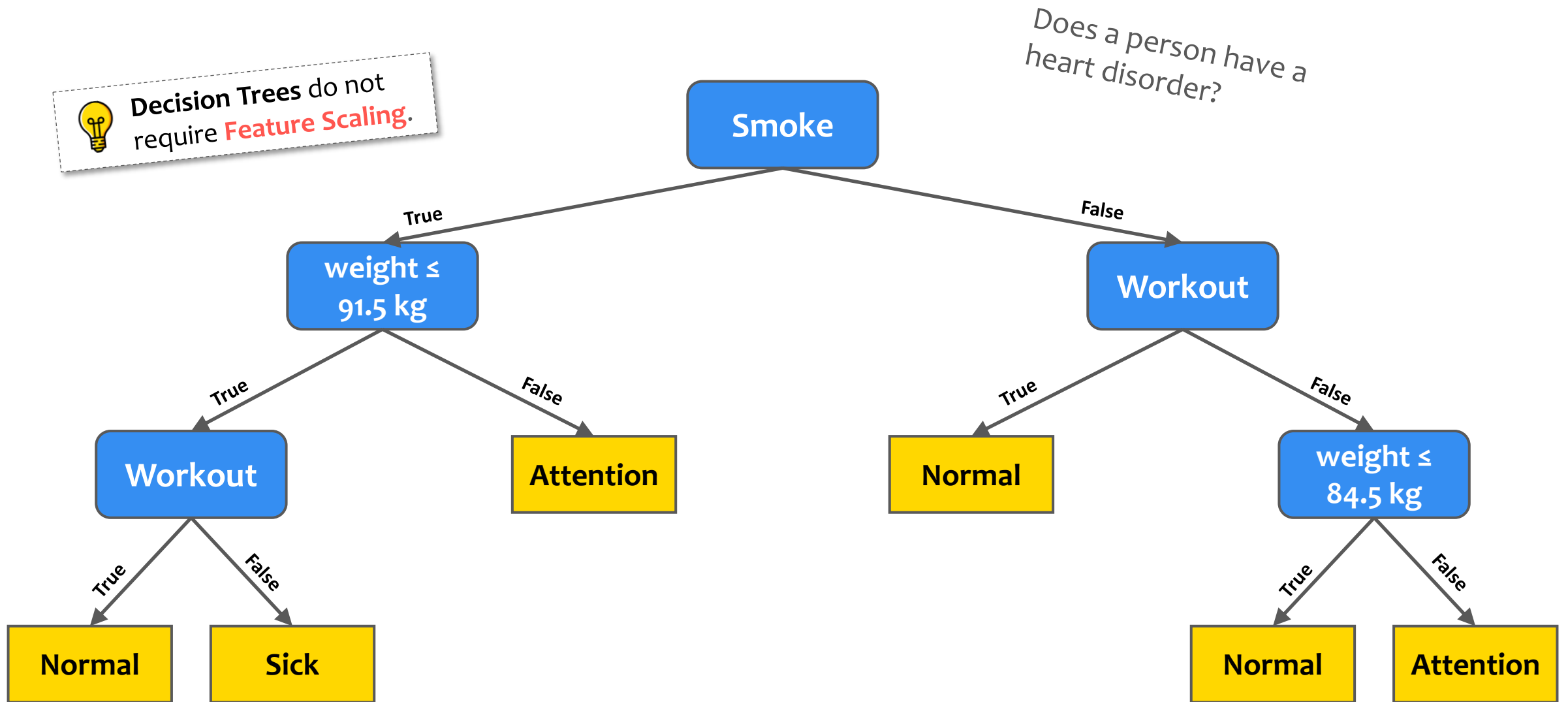


The **classes** can be **repeated**
(in multiple **leaves**).



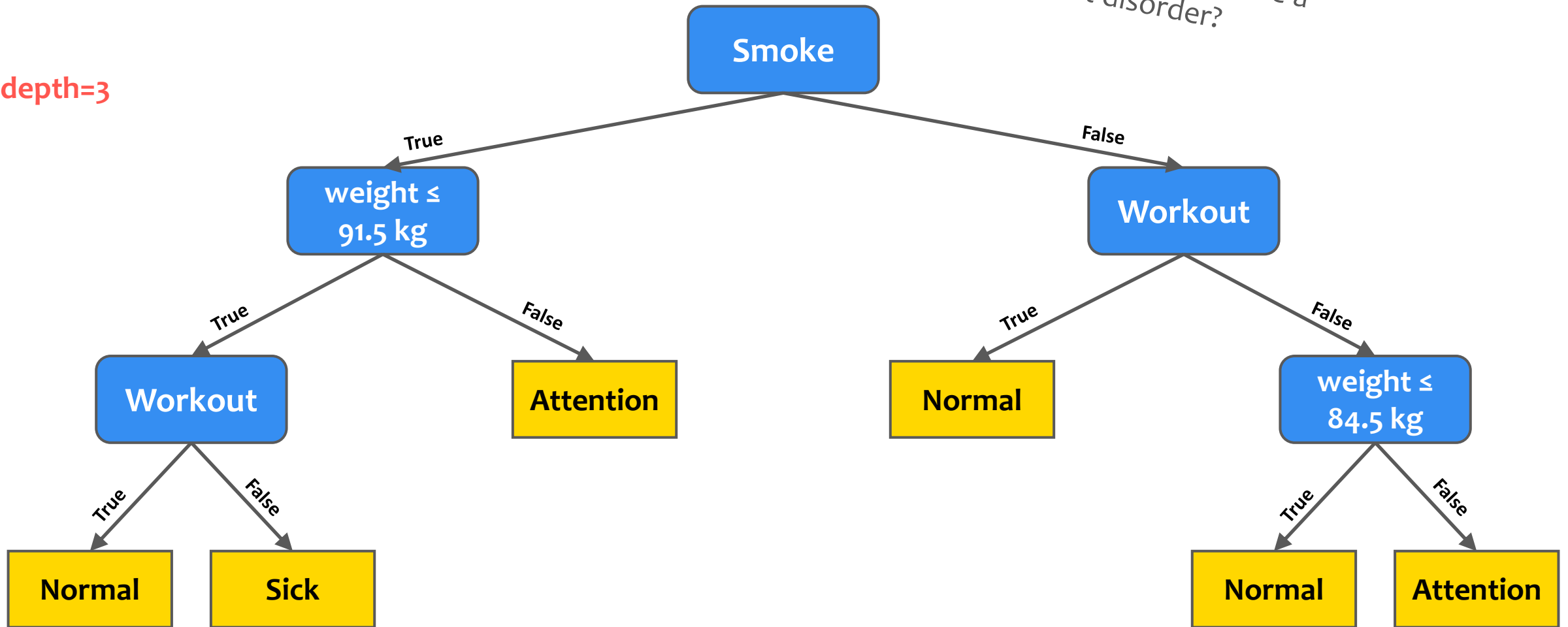


Decision Trees do not require **Feature Scaling**.



Does a person have a heart disorder?

depth=3



Predicting a sample is simple:

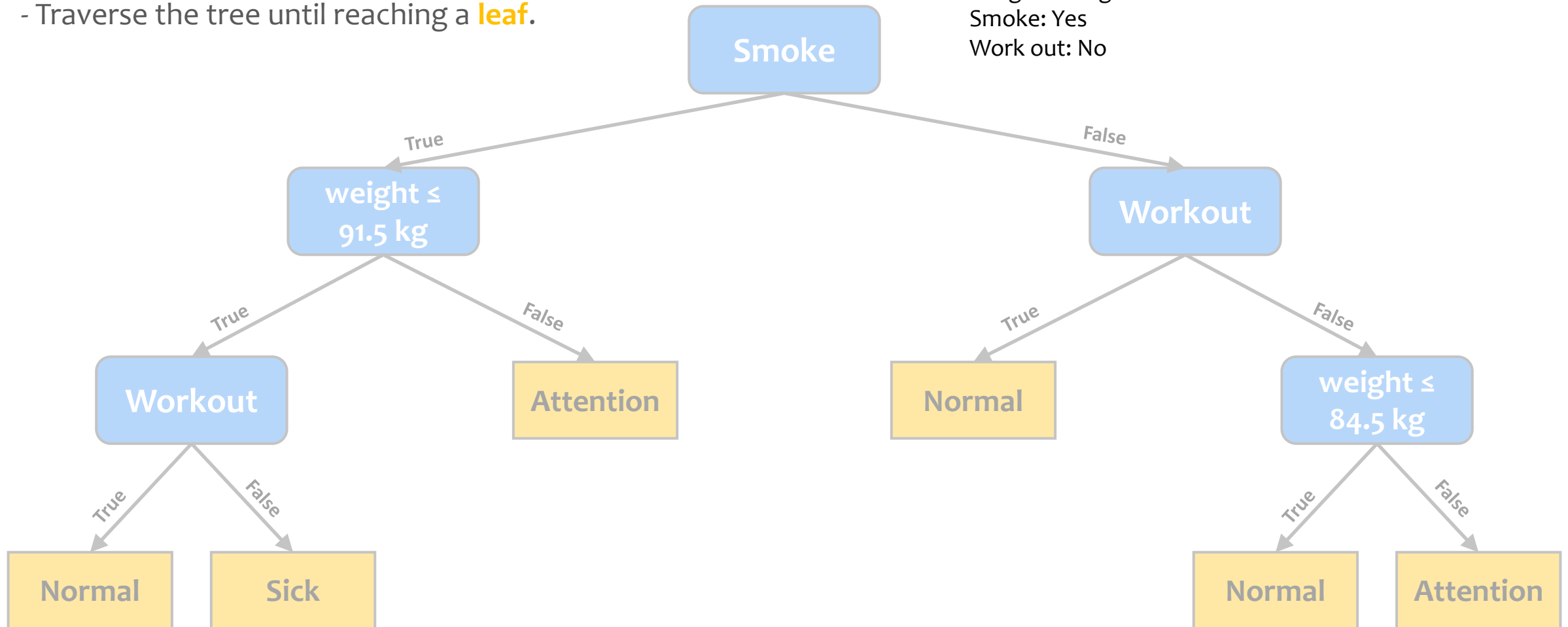
- Start at the **root node**.
- Traverse the tree until reaching a **leaf**.

Does this person have a heart disorder?

Weight: 88 kg

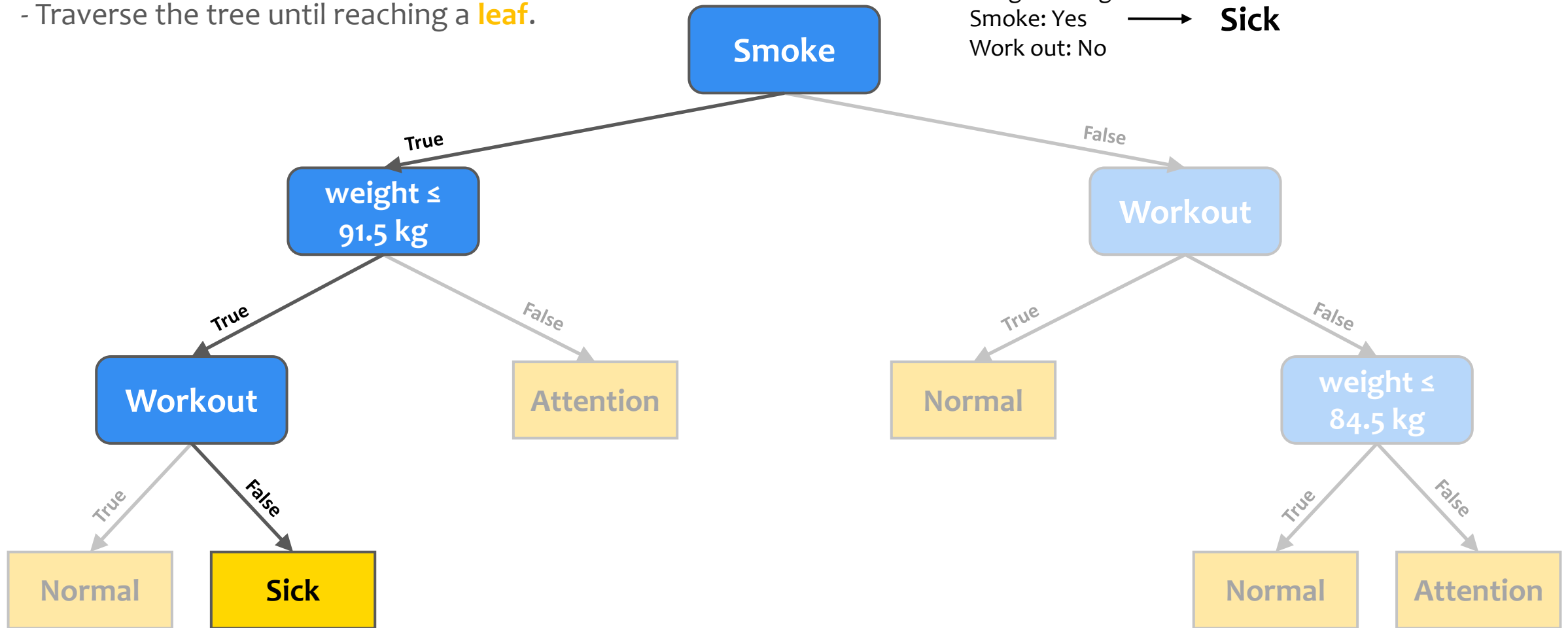
Smoke: Yes

Work out: No



Predicting a sample is simple:

- Start at the **root node**.
- Traverse the tree until reaching a **leaf**.





We will consider the **CART Algorithm** –
Classification **A**nd **R**egression **T**rees –
which is used by Scikit-Learn.



These trees only **splits** the **nodes** into
two sub-nodes.

Training a Decision Tree

smoke	workout	weight	heart condition
No	Yes	85.0	Sick
No	Yes	78.0	Normal
Yes	No	75.0	Sick
No	Yes	84.0	Normal
No	Yes	72.0	Normal
No	No	78.0	Sick
Yes	No	120.0	Attention
...	...	...	...
Yes	Yes	84.0	Sick
No	No	98.0	Attention

Consider this simple (and fake) **dataset** about **heart condition** of a set of 100 people.

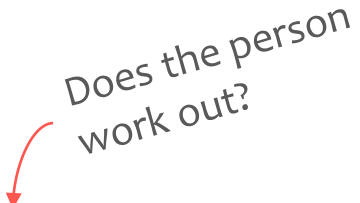
$m = 100$ samples

Does the person smoke?

smoke	workout	weight	heart condition
No	Yes	85.0	Sick
No	Yes	78.0	Normal
Yes	No	75.0	Sick
No	Yes	84.0	Normal
No	Yes	72.0	Normal
No	No	78.0	Sick
Yes	No	120.0	Attention
...	...	...	...
Yes	Yes	84.0	Sick
No	No	98.0	Attention

Consider this simple (and fake) **dataset** about **heart condition** of a set of 100 people.

$m = 100$ samples



smoke	workout	weight	heart condition
No	Yes	85.0	Sick
No	Yes	78.0	Normal
Yes	No	75.0	Sick
No	Yes	84.0	Normal
No	Yes	72.0	Normal
No	No	78.0	Sick
Yes	No	120.0	Attention
...	...	...	...
Yes	Yes	84.0	Sick
No	No	98.0	Attention

Consider this simple (and fake) **dataset** about **heart condition** of a set of 100 people.

$m = 100$ samples

What is the person's weight (kg)?

smoke	workout	weight	heart condition
No	Yes	85.0	Sick
No	Yes	78.0	Normal
Yes	No	75.0	Sick
No	Yes	84.0	Normal
No	Yes	72.0	Normal
No	No	78.0	Sick
Yes	No	120.0	Attention
...	...	...	...
Yes	Yes	84.0	Sick
No	No	98.0	Attention

Consider this simple (and fake) **dataset** about **heart condition** of a set of 100 people.

$m = 100$ samples

What is the person's
heart condition (**class**)?

smoke	workout	weight	heart condition
No	Yes	85.0	Sick
No	Yes	78.0	Normal
Yes	No	75.0	Sick
No	Yes	84.0	Normal
No	Yes	72.0	Normal
No	No	78.0	Sick
Yes	No	120.0	Attention
...	...	...	...
Yes	Yes	84.0	Sick
No	No	98.0	Attention

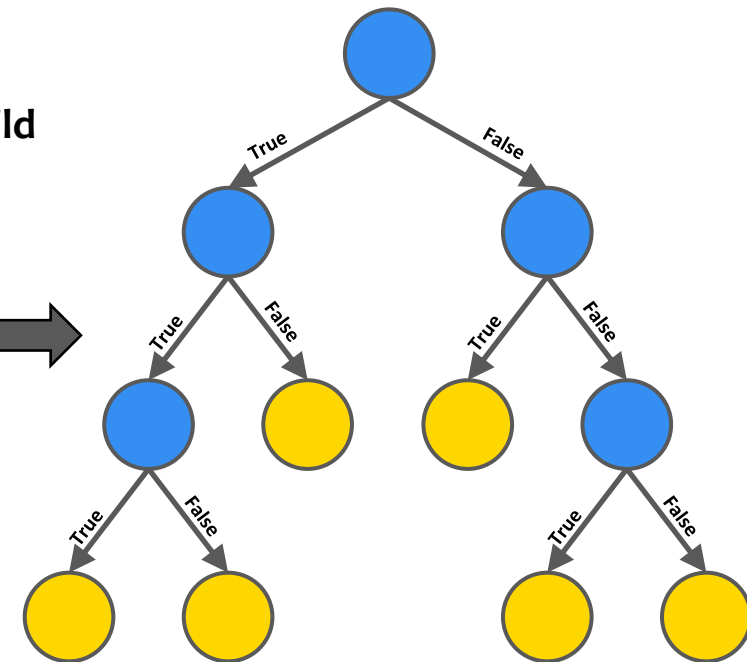
Consider this simple (and fake) **dataset** about
heart condition of a set of 100 people.

$m = 100$ samples

smoke	workout	weight	heart condition
No	Yes	85.0	Sick
No	Yes	78.0	Normal
Yes	No	75.0	Sick
No	Yes	84.0	Normal
No	Yes	72.0	Normal
No	No	78.0	Sick
Yes	No	120.0	Attention
...	...	...	...
Yes	Yes	84.0	Sick
No	No	98.0	Attention

$m = 100$ samples

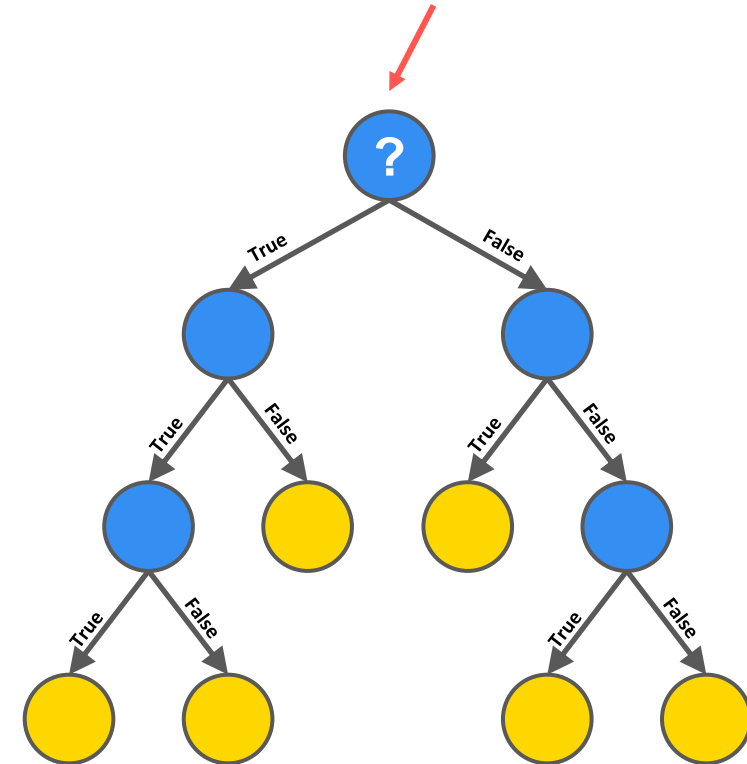
Let's see how to **train/build** a **Decision Tree** from a **dataset**.



smoke	workout	weight	heart condition
No	Yes	85.0	Sick
No	Yes	78.0	Normal
Yes	No	75.0	Sick
No	Yes	84.0	Normal
No	Yes	72.0	Normal
No	No	78.0	Sick
Yes	No	120.0	Attention
...	...	...	...
Yes	Yes	84.0	Sick
No	No	98.0	Attention

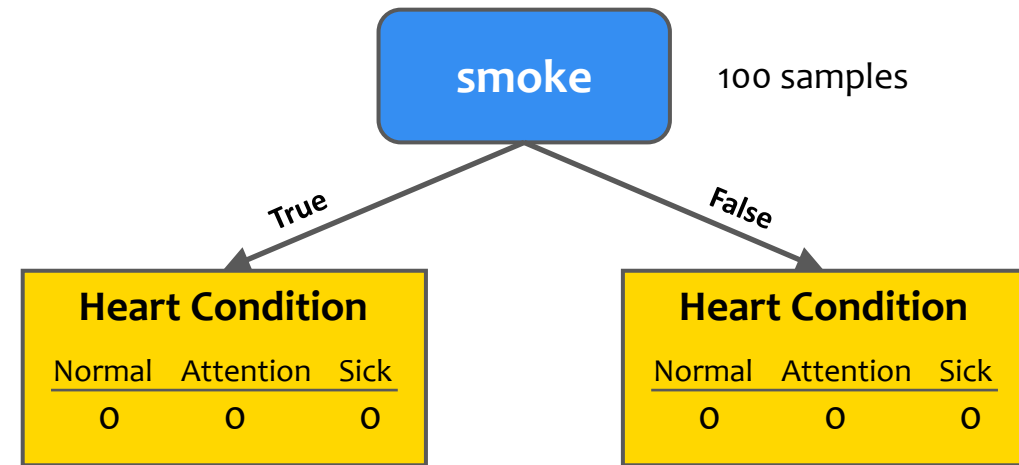
$m = 100$ samples

Which **feature** should be at the **root**?



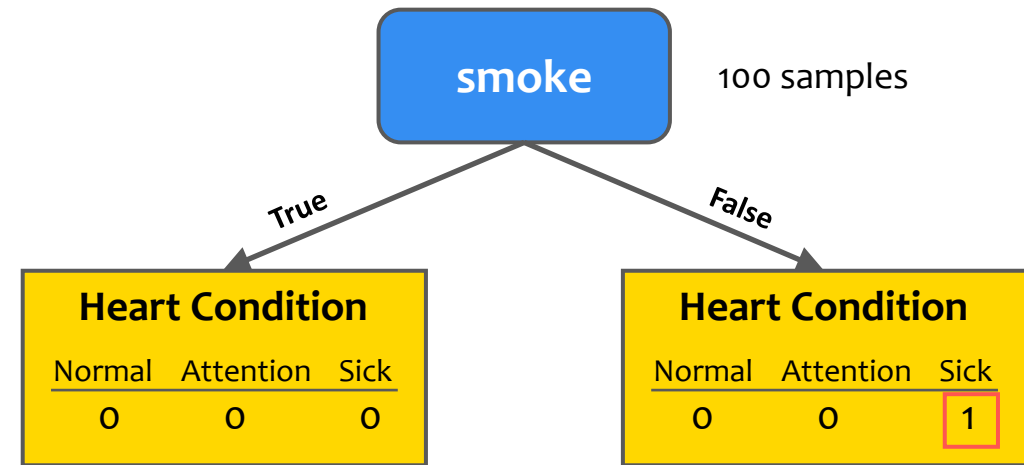
smoke	workout	weight	heart condition
No	Yes	85.0	Sick
No	Yes	78.0	Normal
Yes	No	75.0	Sick
No	Yes	84.0	Normal
No	Yes	72.0	Normal
No	No	78.0	Sick
Yes	No	120.0	Attention
...	...	...	...
Yes	Yes	84.0	Sick
No	No	98.0	Attention

$m = 100$ samples



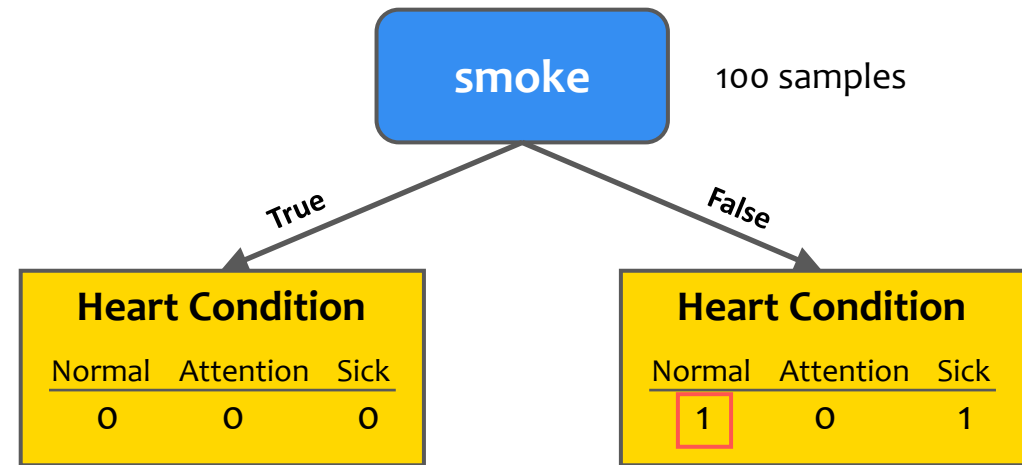
smoke	workout	weight	heart condition
No	Yes	85.0	Sick
No	Yes	78.0	Normal
Yes	No	75.0	Sick
No	Yes	84.0	Normal
No	Yes	72.0	Normal
No	No	78.0	Sick
Yes	No	120.0	Attention
...	...	...	...
Yes	Yes	84.0	Sick
No	No	98.0	Attention

$m = 100$ samples



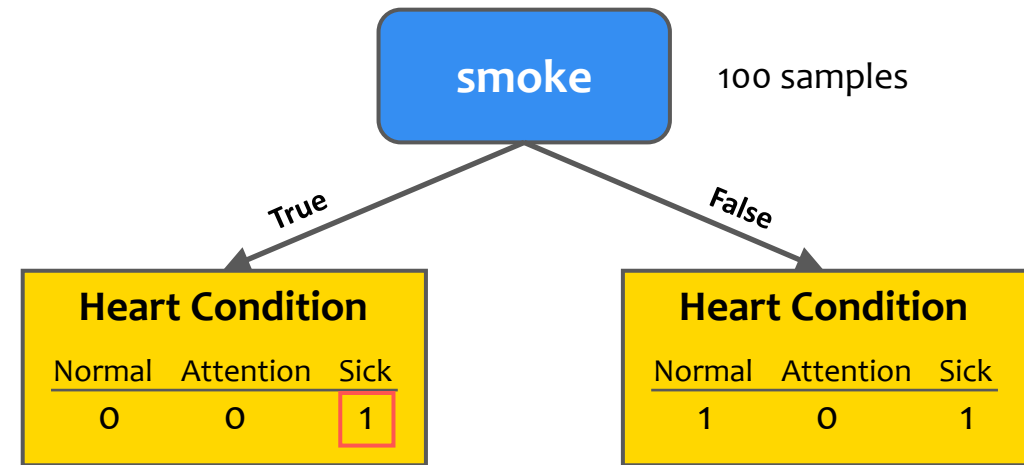
smoke	workout	weight	heart condition
No	Yes	85.0	Sick
No	Yes	78.0	Normal
Yes	No	75.0	Sick
No	Yes	84.0	Normal
No	Yes	72.0	Normal
No	No	78.0	Sick
Yes	No	120.0	Attention
...	...	...	...
Yes	Yes	84.0	Sick
No	No	98.0	Attention

$m = 100$ samples



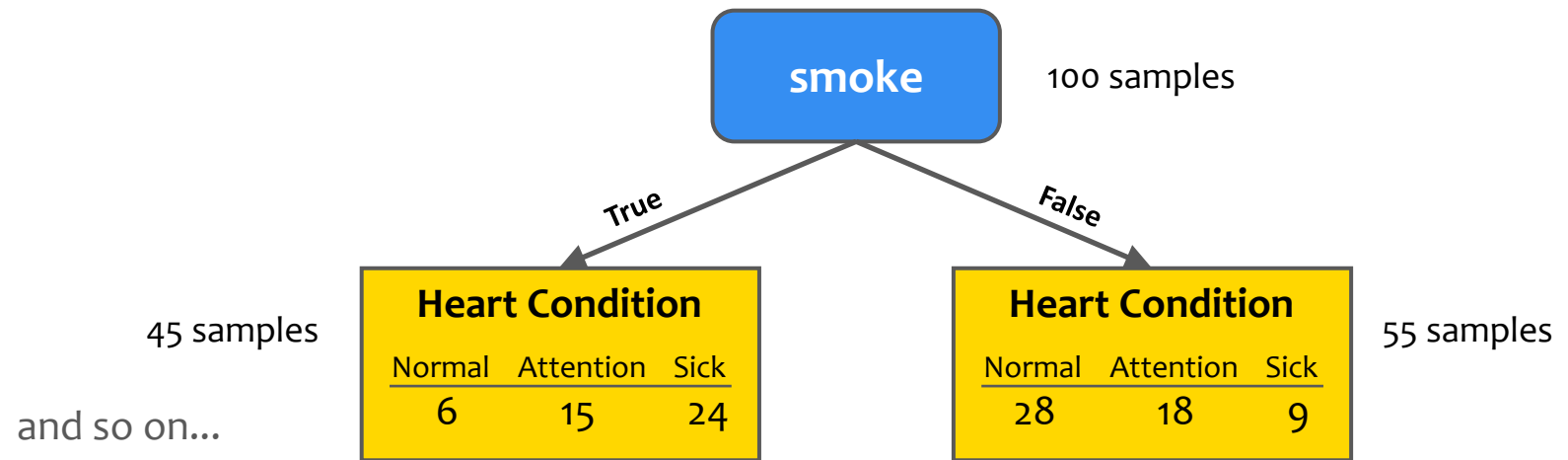
smoke	workout	weight	heart condition
No	Yes	85.0	Sick
No	Yes	78.0	Normal
Yes	No	75.0	Sick
No	Yes	84.0	Normal
No	Yes	72.0	Normal
No	No	78.0	Sick
Yes	No	120.0	Attention
...	...	...	...
Yes	Yes	84.0	Sick
No	No	98.0	Attention

$m = 100$ samples



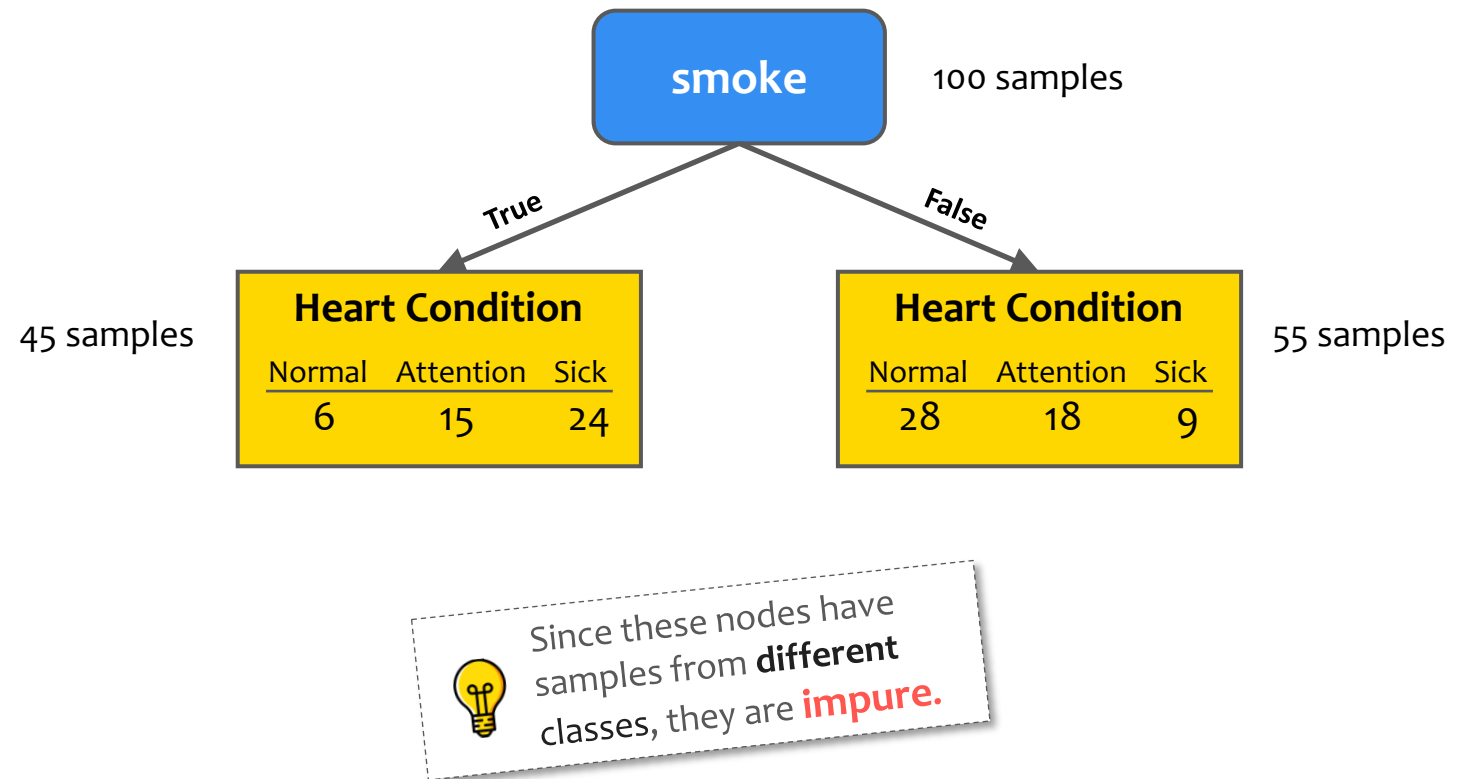
smoke	workout	weight	heart condition
No	Yes	85.0	Sick
No	Yes	78.0	Normal
Yes	No	75.0	Sick
No	Yes	84.0	Normal
No	Yes	72.0	Normal
No	No	78.0	Sick
Yes	No	120.0	Attention
...	...	...	...
Yes	Yes	84.0	Sick
No	No	98.0	Attention

$m = 100$ samples



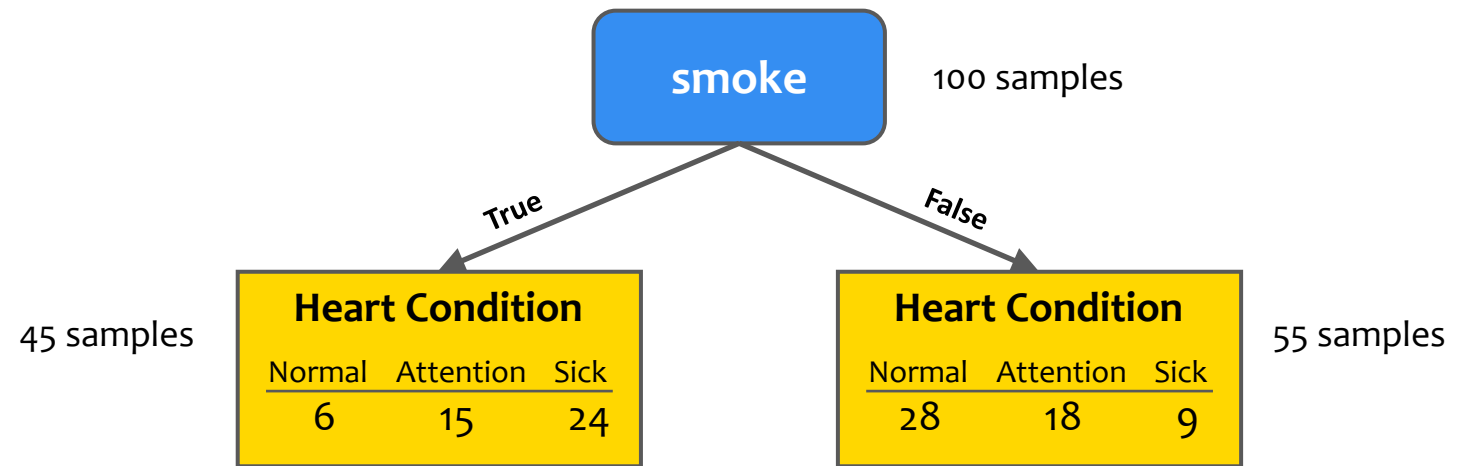
smoke	workout	weight	heart condition
No	Yes	85.0	Sick
No	Yes	78.0	Normal
Yes	No	75.0	Sick
No	Yes	84.0	Normal
No	Yes	72.0	Normal
No	No	78.0	Sick
Yes	No	120.0	Attention
...	...	...	...
Yes	Yes	84.0	Sick
No	No	98.0	Attention

$m = 100$ samples



smoke	workout	weight	heart condition
No	Yes	85.0	Sick
No	Yes	78.0	Normal
Yes	No	75.0	Sick
No	Yes	84.0	Normal
No	Yes	72.0	Normal
No	No	78.0	Sick
Yes	No	120.0	Attention
...	...	...	...
Yes	Yes	84.0	Sick
No	No	98.0	Attention

$m = 100$ samples



Since these nodes have samples from **different** classes, they are **impure**.

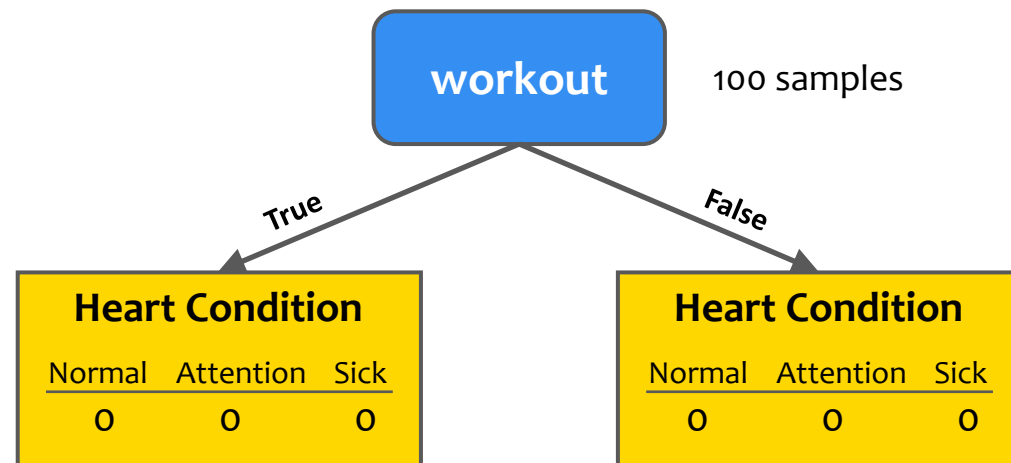
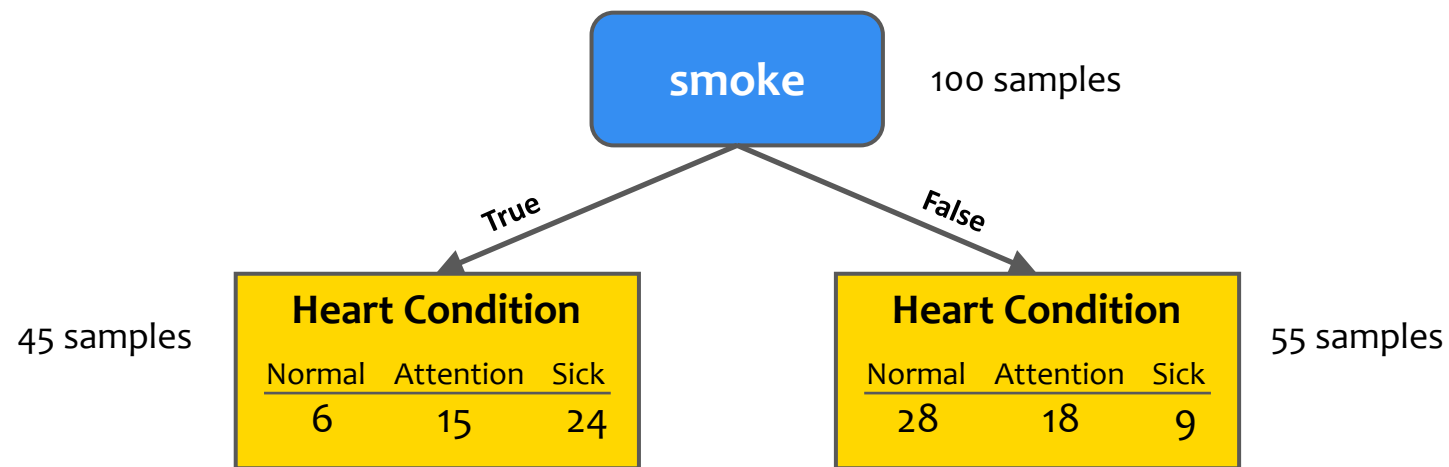


A node is **pure** when **all** its samples belong to the **same class**.

Heart Condition		
Normal	Attention	Sick
45	0	0

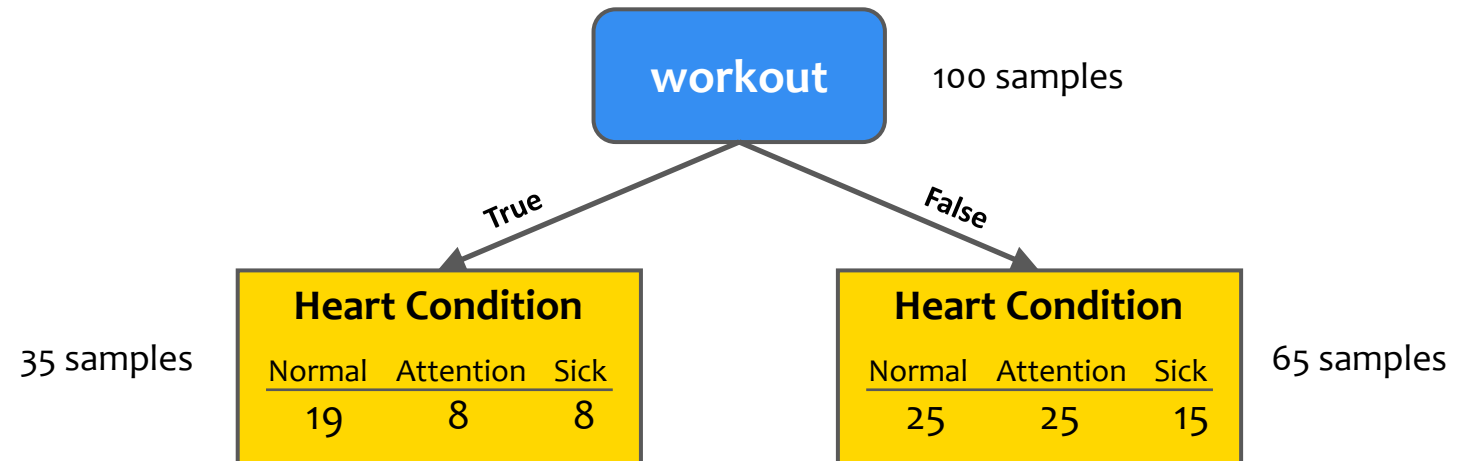
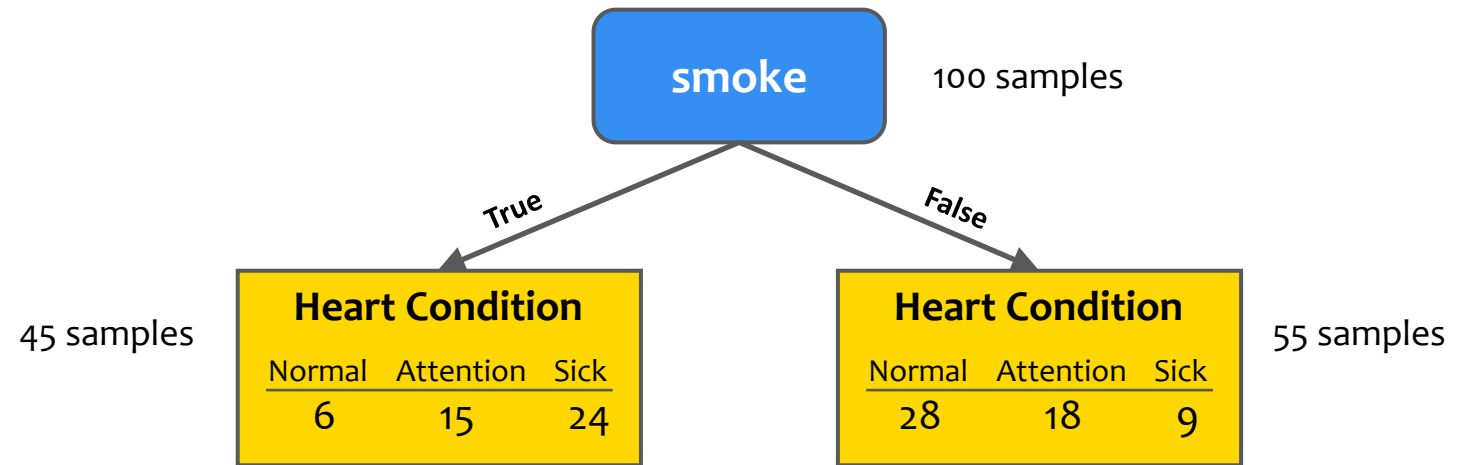
smoke	workout	weight	heart condition
No	Yes	85.0	Sick
No	Yes	78.0	Normal
Yes	No	75.0	Sick
No	Yes	84.0	Normal
No	Yes	72.0	Normal
No	No	78.0	Sick
Yes	No	120.0	Attention
...	...	...	...
Yes	Yes	84.0	Sick
No	No	98.0	Attention

$m = 100$ samples

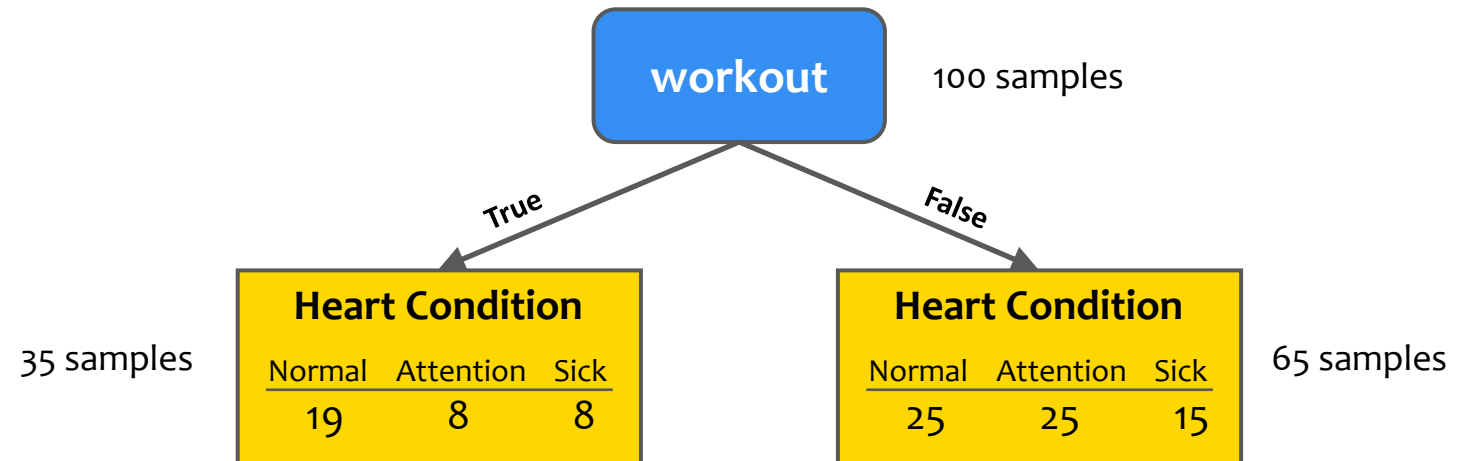
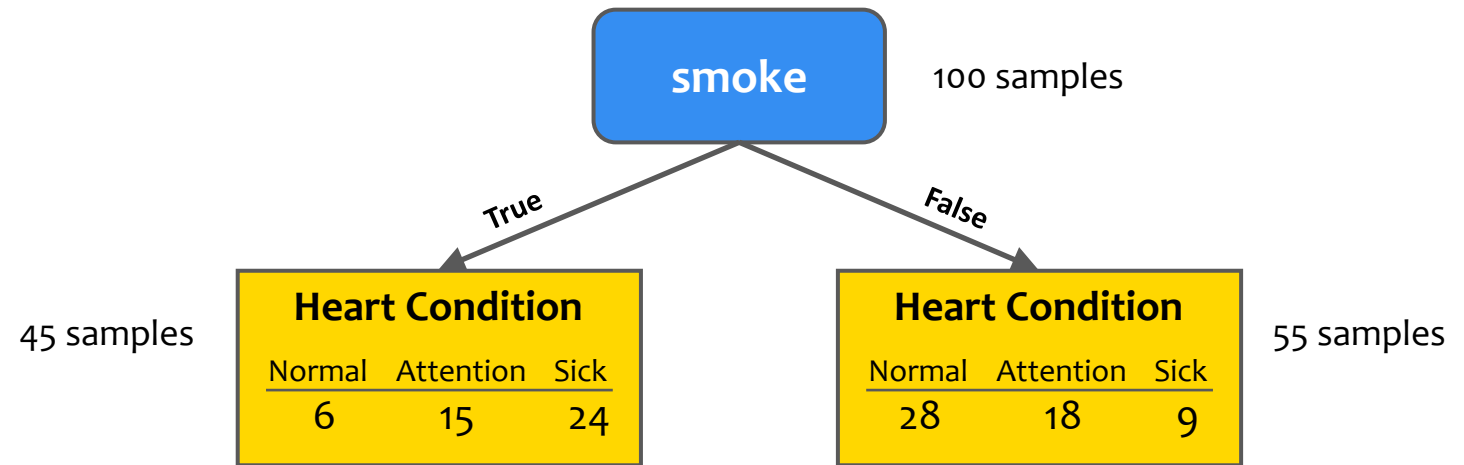


smoke	workout	weight	heart condition
No	Yes	85.0	Sick
No	Yes	78.0	Normal
Yes	No	75.0	Sick
No	Yes	84.0	Normal
No	Yes	72.0	Normal
No	No	78.0	Sick
Yes	No	120.0	Attention
...	...	...	...
Yes	Yes	84.0	Sick
No	No	98.0	Attention

$m = 100$ samples

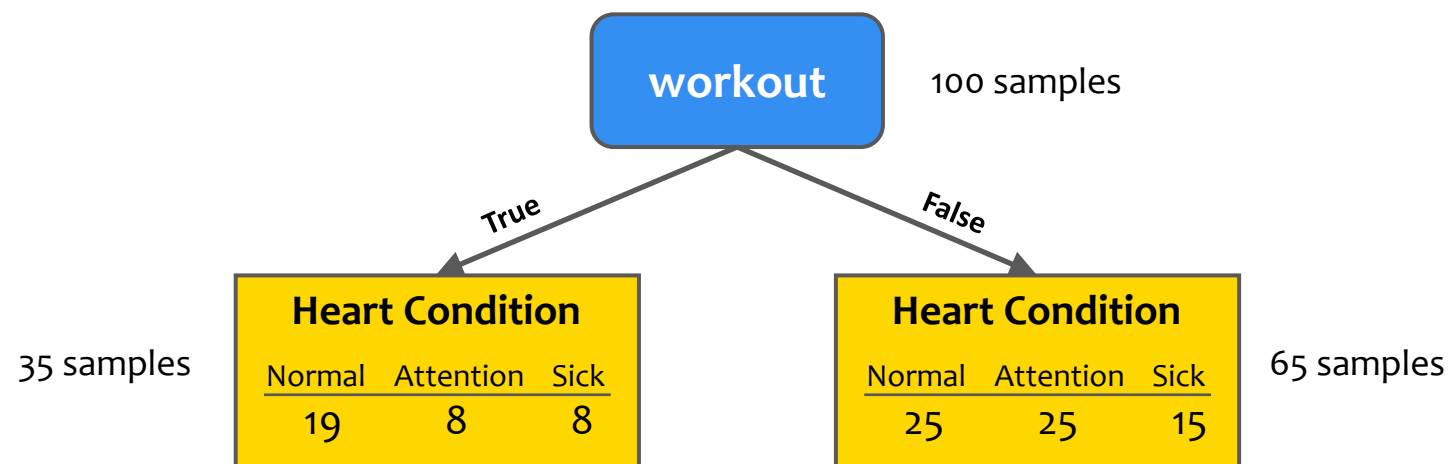
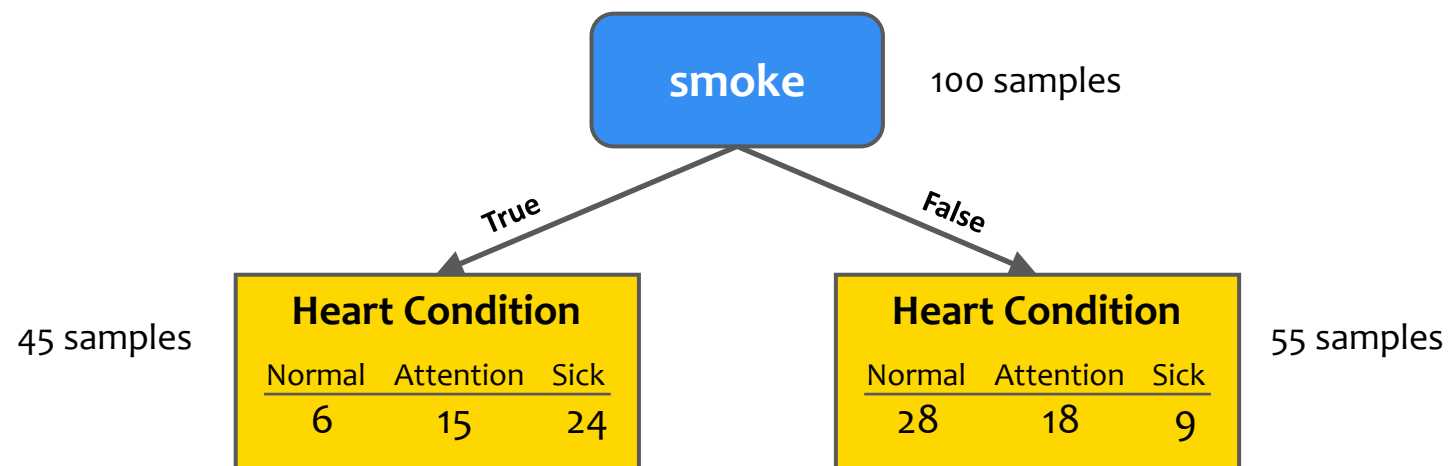


Analyzing just these **two features**,
which should we choose?



Analyzing just these **two features**,
which should we choose?

↓
The **least impure** /
the **purest**...

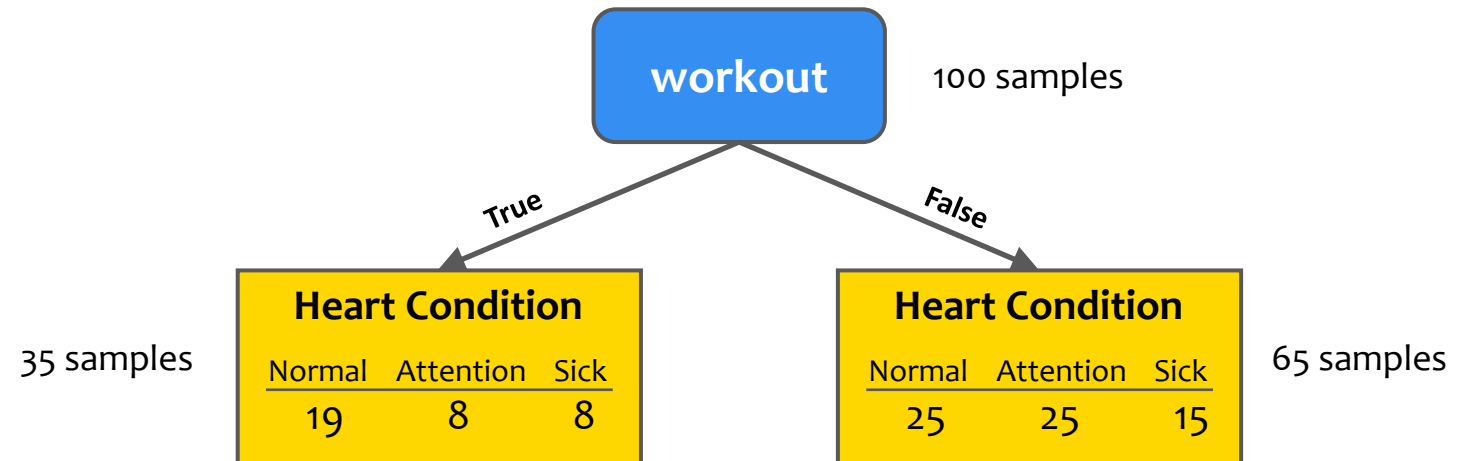
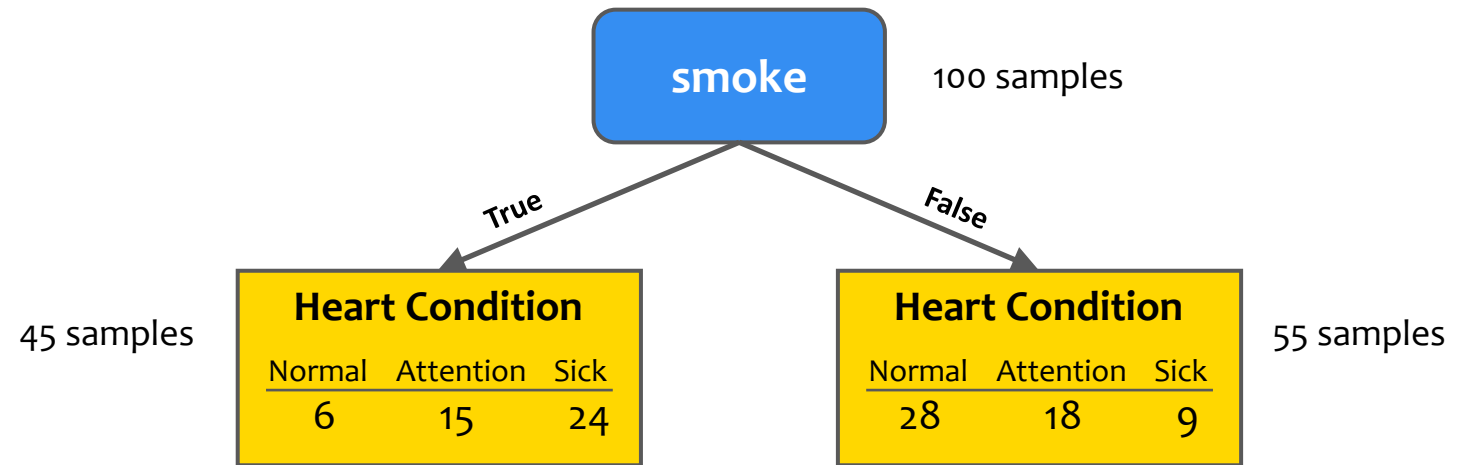


Analyzing just these **two features**,
which should we choose?

The **least impure** /
the **purest**...

How to compute **impurity**?

- Gini impurity
- Entropy
- Information Gain



Analyzing just these **two features**,
which should we choose?

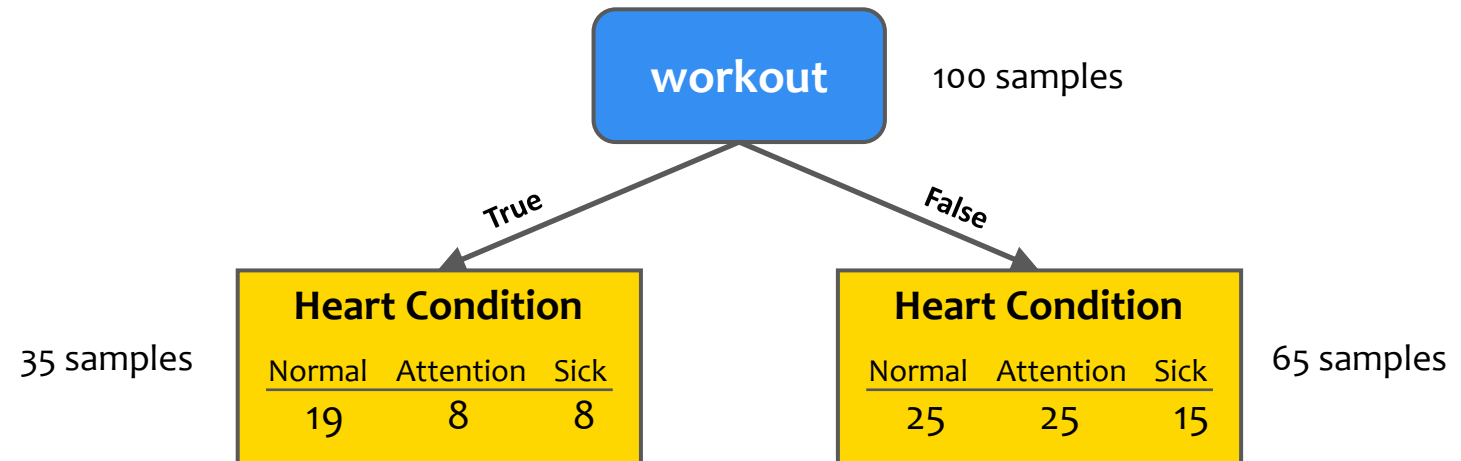
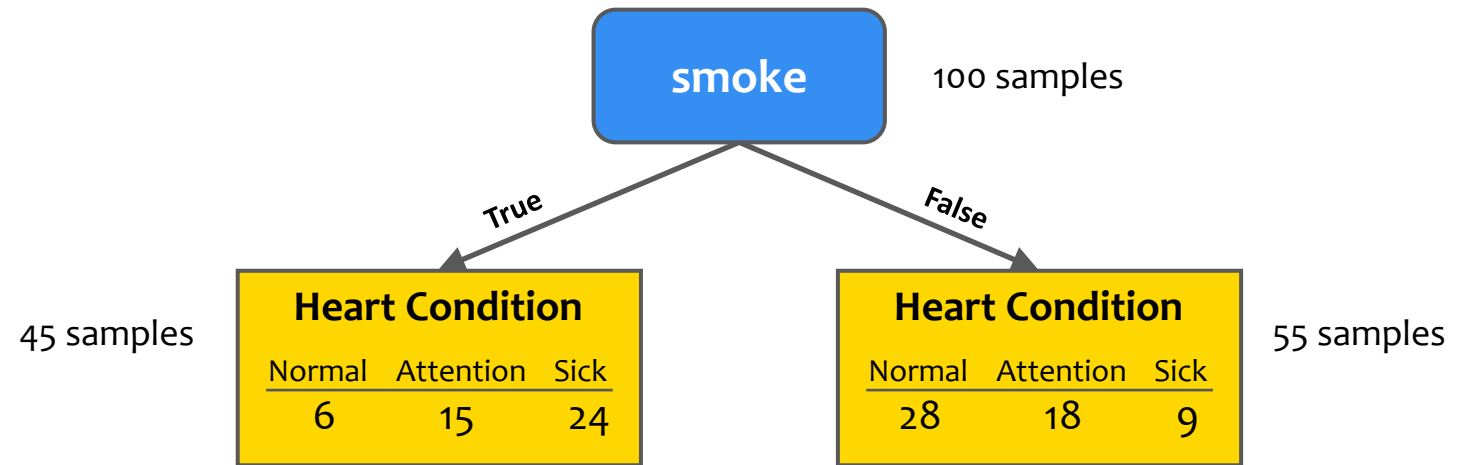
The **least impure** /
the **purest**...

How to compute **impurity**?

- **Gini impurity**

- Entropy

- Information Gain

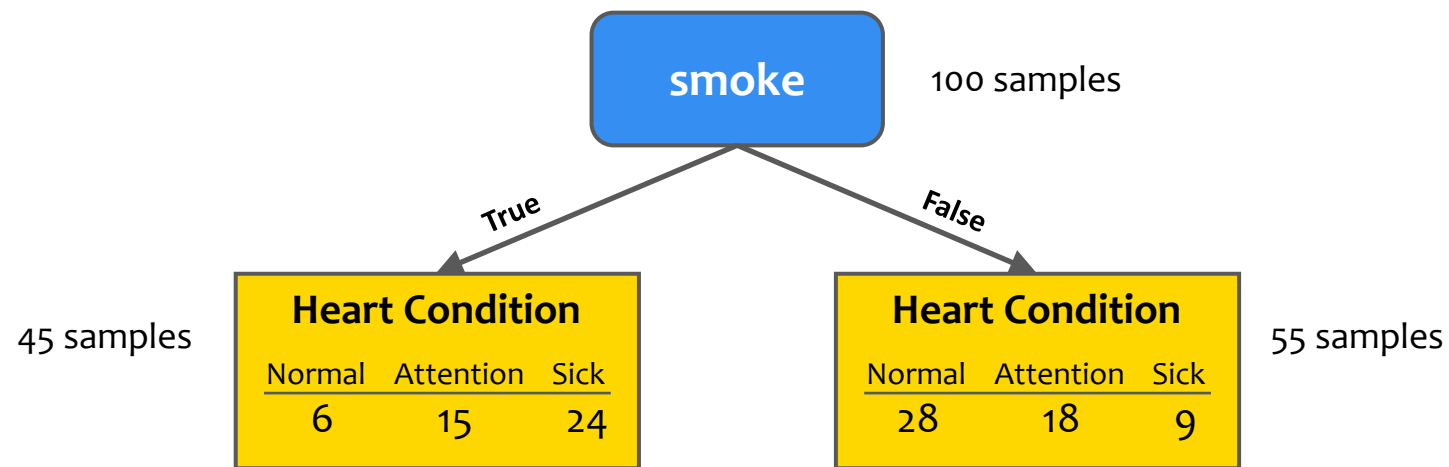


Gini impurity for a node i

$$G_i = 1 - \sum_{c=1}^{n_classes} p_{i,c}^2$$

$p_{i,c}$ is the **probability** of **class c** of i^{th} node.

Ratio of **class c instances** among **all the instances** in the i^{th} node.

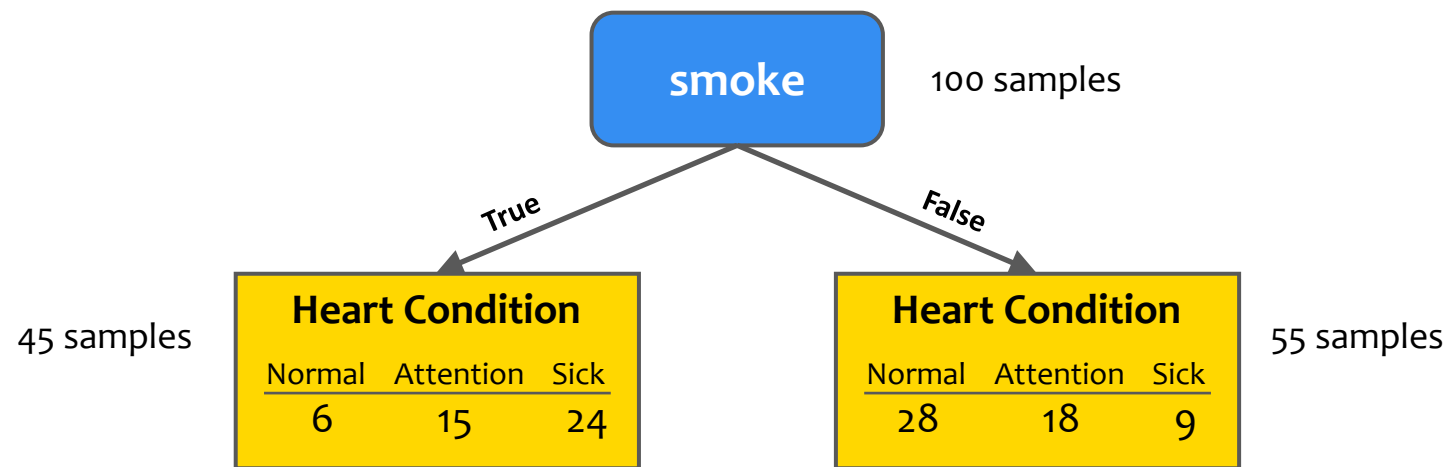


Gini impurity for a node i

$$G_i = 1 - \sum_{c=1}^{n_classes} p_{i,c}^2$$

$p_{i,c}$ is the **probability** of **class c** of i^{th} node.

Ratio of **class c instances** among **all the instances** in the i^{th} node.



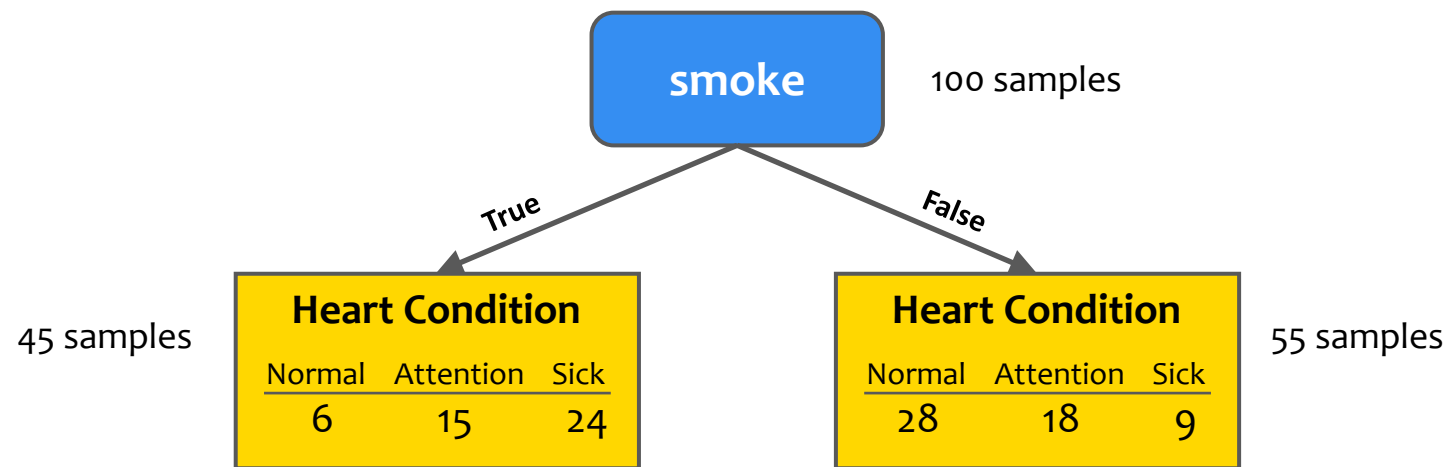
$$G_{smoke_{true}} = 1 - ((\text{the prob. of "Normal"})^2 + (\text{the prob. of "Attention"})^2 + (\text{the prob. of "Sick"})^2)$$

Gini impurity for a node i

$$G_i = 1 - \sum_{c=1}^{n_classes} p_{i,c}^2$$

$p_{i,c}$ is the **probability** of **class c** of i^{th} node.

Ratio of **class c instances** among **all the instances** in the i^{th} node.



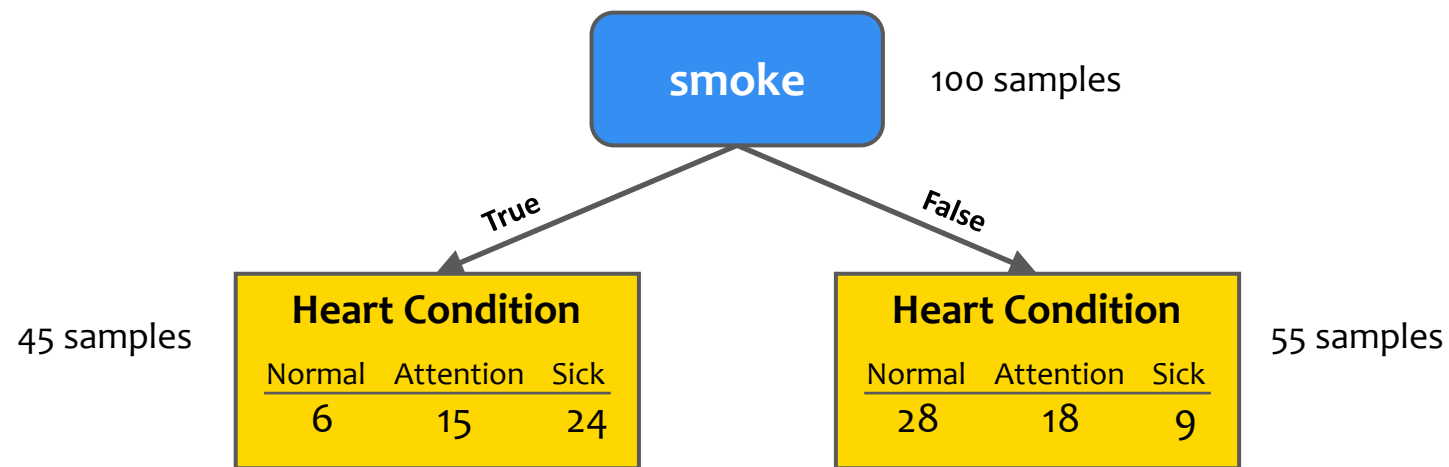
$$G_{smoke_{true}} = 1 - ((\text{the prob. of "Normal"})^2 + (\text{the prob. of "Attention"})^2 + (\text{the prob. of "Sick"})^2)$$

Gini impurity for a node i

$$G_i = 1 - \sum_{c=1}^{n_classes} p_{i,c}^2$$

$p_{i,c}$ is the **probability** of **class c** of i^{th} node.

Ratio of **class c instances** among **all the instances** in the i^{th} node.



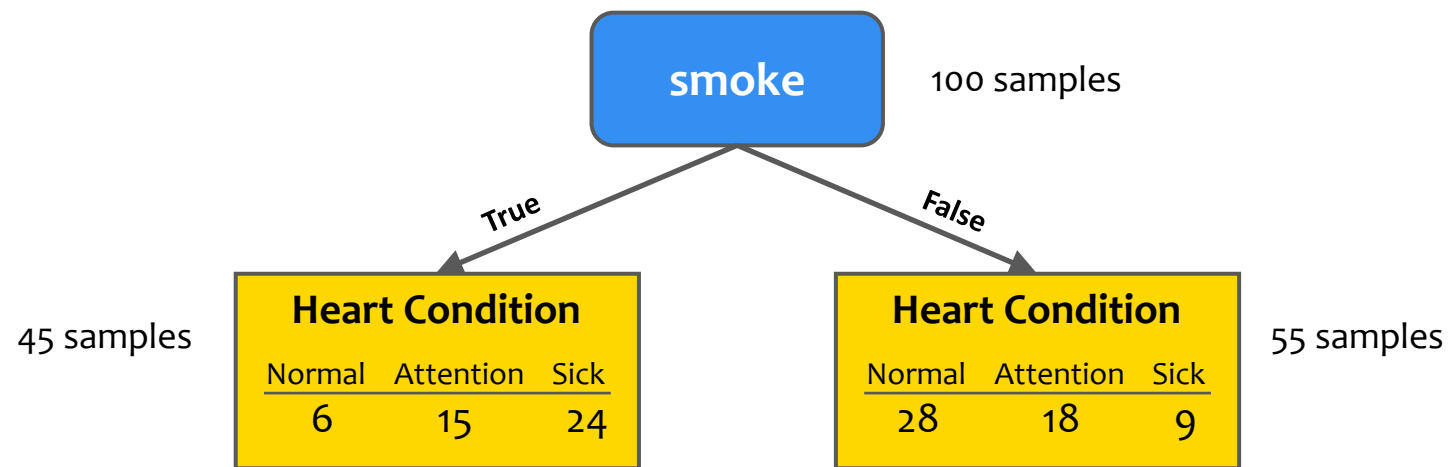
$$G_{smoke_{true}} = 1 - \left(\left(\frac{6}{6+15+24} \right)^2 + \right. \\ \left. \begin{array}{l} \text{(the prob. of "Attention")}^2 + \\ \text{(the prob. of "Sick")}^2 \end{array} \right)$$

Gini impurity for a node i

$$G_i = 1 - \sum_{c=1}^{n_classes} p_{i,c}^2$$

$p_{i,c}$ is the **probability** of **class c** of i^{th} node.

Ratio of **class c instances** among **all the instances** in the i^{th} node.



$$G_{smoke_{true}} = 1 - ((0.1333)^2 +$$

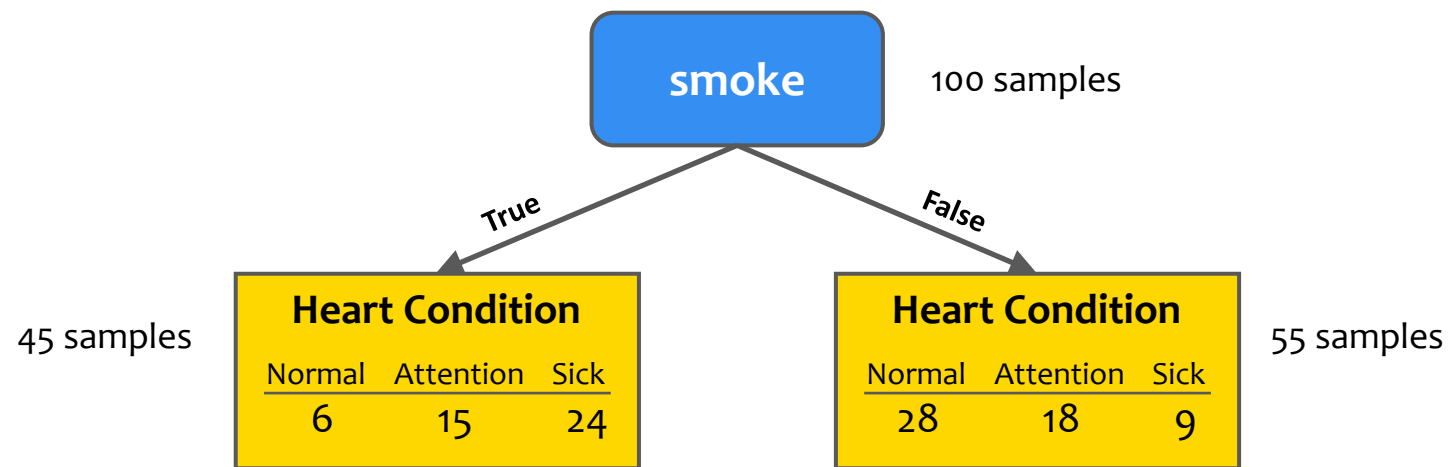
(the prob. of "Attention")² +
(the prob. of "Sick")²)

Gini impurity for a node i

$$G_i = 1 - \sum_{c=1}^{n_classes} p_{i,c}^2$$

$p_{i,c}$ is the **probability** of **class c** of i^{th} node.

Ratio of **class c instances** among **all the instances** in the i^{th} node.



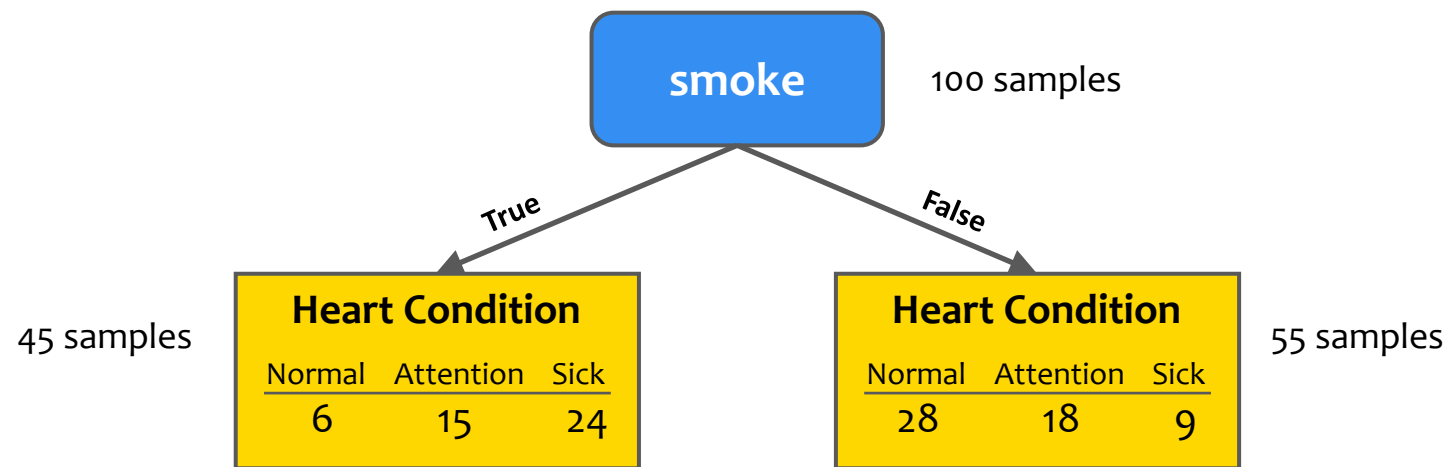
$$G_{smoke_{true}} = 1 - (0.0177 + (\text{the prob. of "Attention"})^2 + (\text{the prob. of "Sick"})^2)$$

Gini impurity for a node i

$$G_i = 1 - \sum_{c=1}^{n_classes} p_{i,c}^2$$

$p_{i,c}$ is the **probability** of **class c** of i^{th} node.

Ratio of **class c instances** among **all the instances** in the i^{th} node.



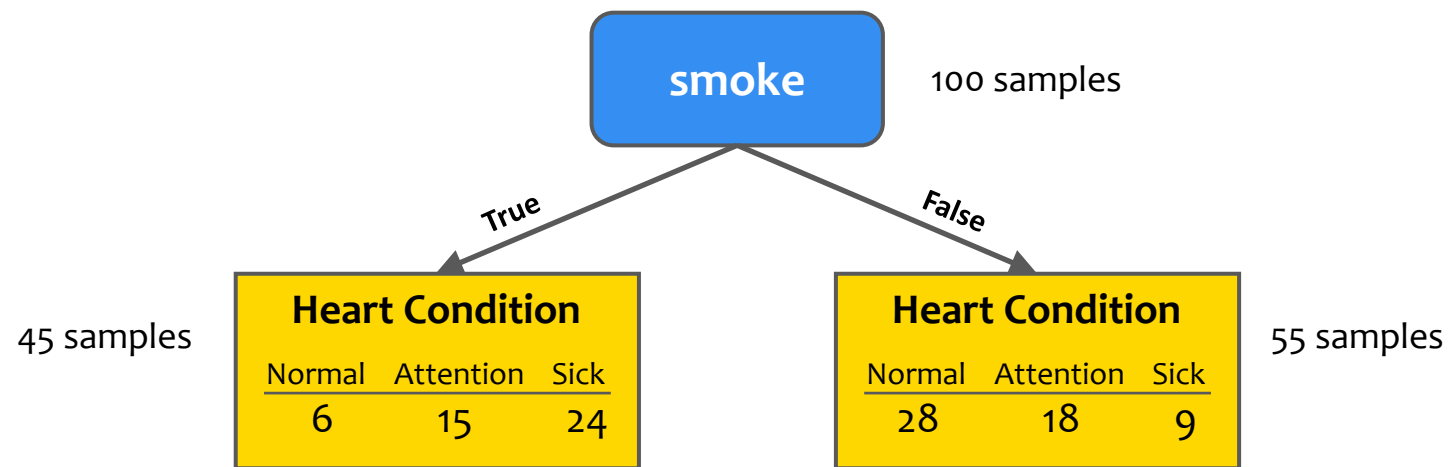
$$G_{smoke_{true}} = 1 - (0.0177 + \boxed{(\text{the prob. of "Attention"})^2} + (\text{the prob. of "Sick"})^2)$$

Gini impurity for a node i

$$G_i = 1 - \sum_{c=1}^{n_classes} p_{i,c}^2$$

$p_{i,c}$ is the **probability** of **class c** of i^{th} node.

Ratio of **class c instances** among **all the instances** in the i^{th} node.



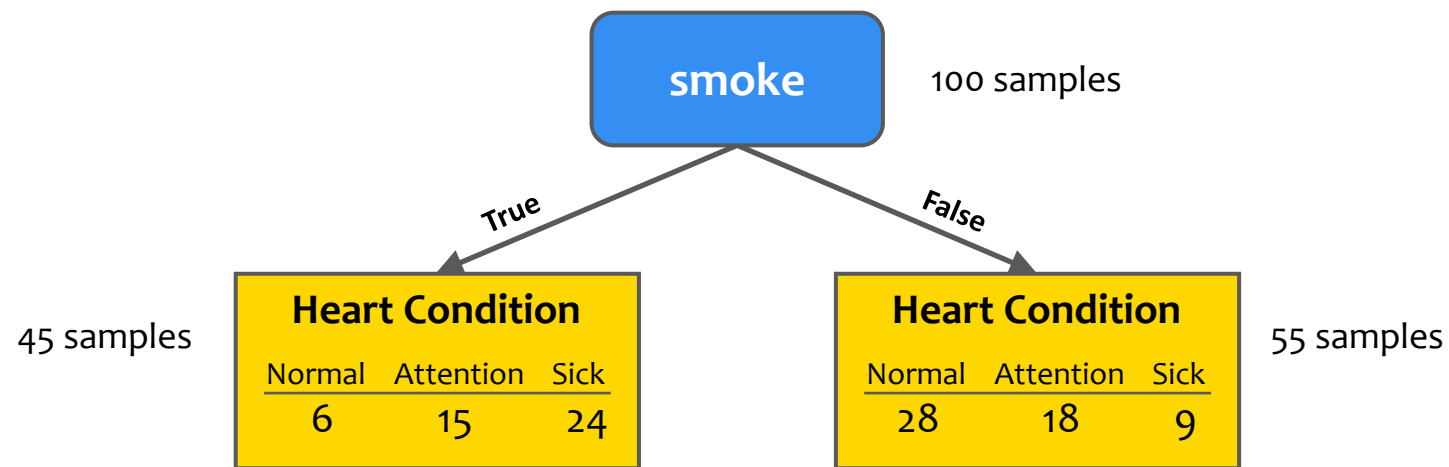
$$G_{smoke_{true}} = 1 - (0.0177 + (\frac{15}{6+15+24})^2 + (\text{the prob. of "Sick"})^2)$$

Gini impurity for a node i

$$G_i = 1 - \sum_{c=1}^{n_classes} p_{i,c}^2$$

$p_{i,c}$ is the **probability** of **class c** of i^{th} node.

Ratio of **class c instances** among **all the instances** in the i^{th} node.



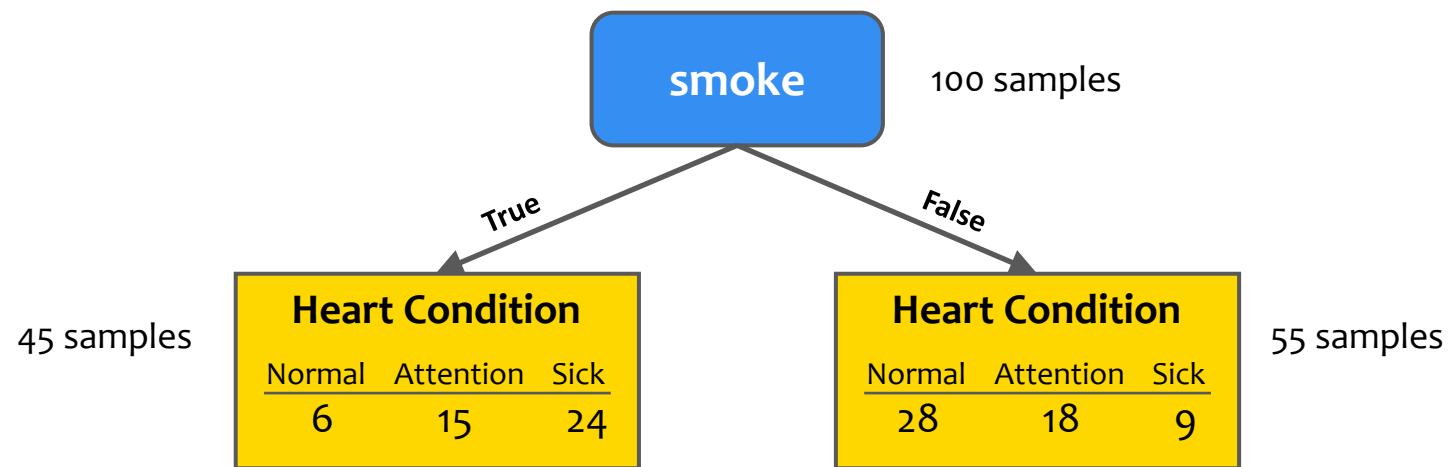
$$G_{smoke_{true}} = 1 - (0.0177 + 0.1111 + (\text{the prob. of "Sick"})^2)$$

Gini impurity for a node i

$$G_i = 1 - \sum_{c=1}^{n_classes} p_{i,c}^2$$

$p_{i,c}$ is the **probability** of **class c** of i^{th} node.

Ratio of **class c instances** among **all the instances** in the i^{th} node.



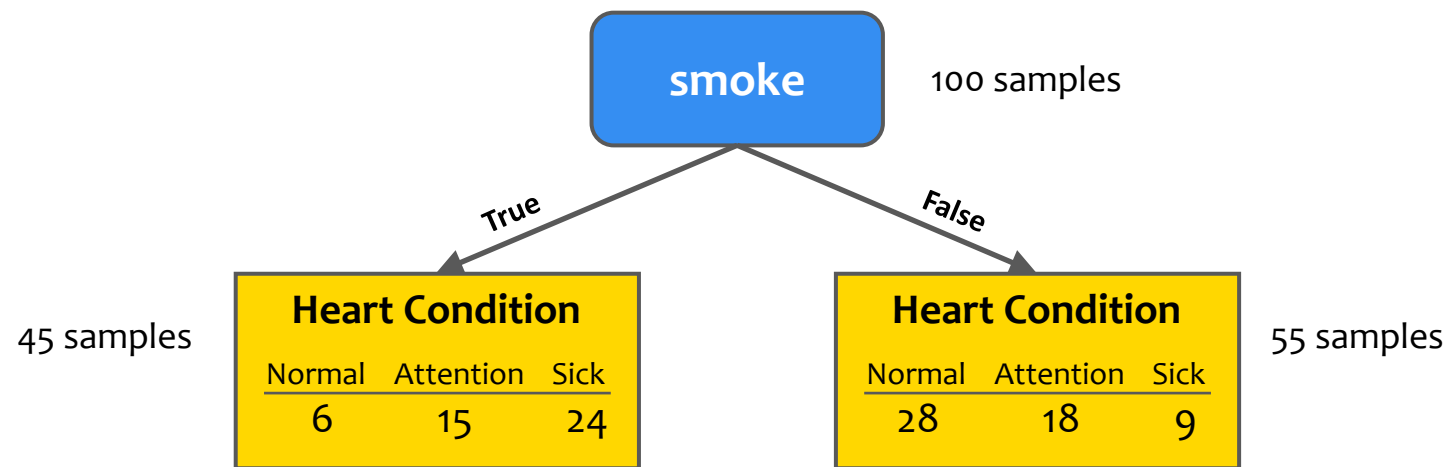
$$G_{smoke_{true}} = 1 - (0.0177 + 0.1111 + \boxed{\text{(the prob. of "Sick")}^2})$$

Gini impurity for a node i

$$G_i = 1 - \sum_{c=1}^{n_classes} p_{i,c}^2$$

$p_{i,c}$ is the **probability** of **class c** of i^{th} node.

Ratio of **class c instances** among **all the instances** in the i^{th} node.



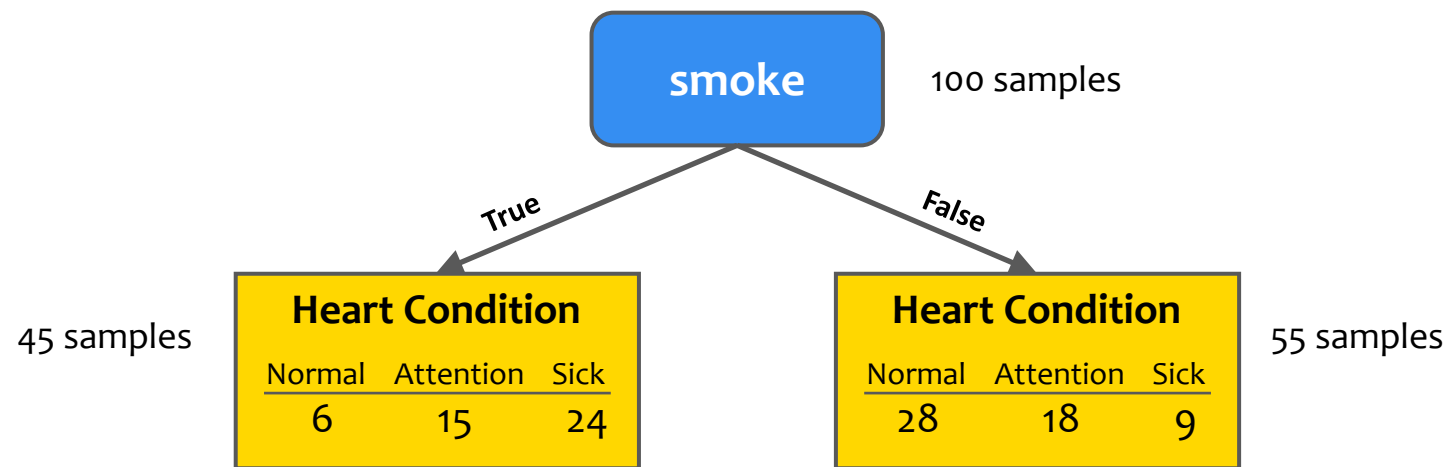
$$G_{smoke_{true}} = 1 - \left(0.0177 + 0.1111 + \left(\frac{24}{6+15+24} \right)^2 \right)$$

Gini impurity for a node i

$$G_i = 1 - \sum_{c=1}^{n_classes} p_{i,c}^2$$

$p_{i,c}$ is the **probability** of **class c** of i^{th} node.

Ratio of **class c instances** among **all the instances** in the i^{th} node.



$$G_{smoke_{true}} = 1 - (0.0177 + 0.1111 + 0.2844)$$

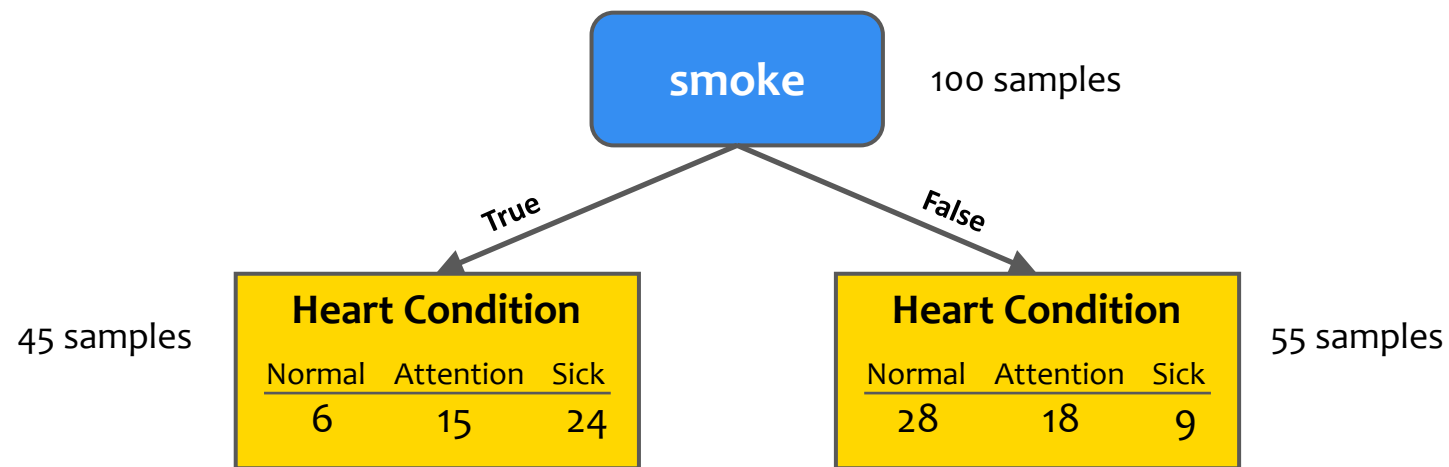
Gini impurity for a node i

$$G_i = 1 - \sum_{c=1}^{n_classes} p_{i,c}^2$$

$p_{i,c}$ is the **probability** of **class c** of i^{th} node.

Ratio of **class c instances** among **all the instances** in the i^{th} node.

$$G_{smoke_{true}} = 1 - 0.4132$$

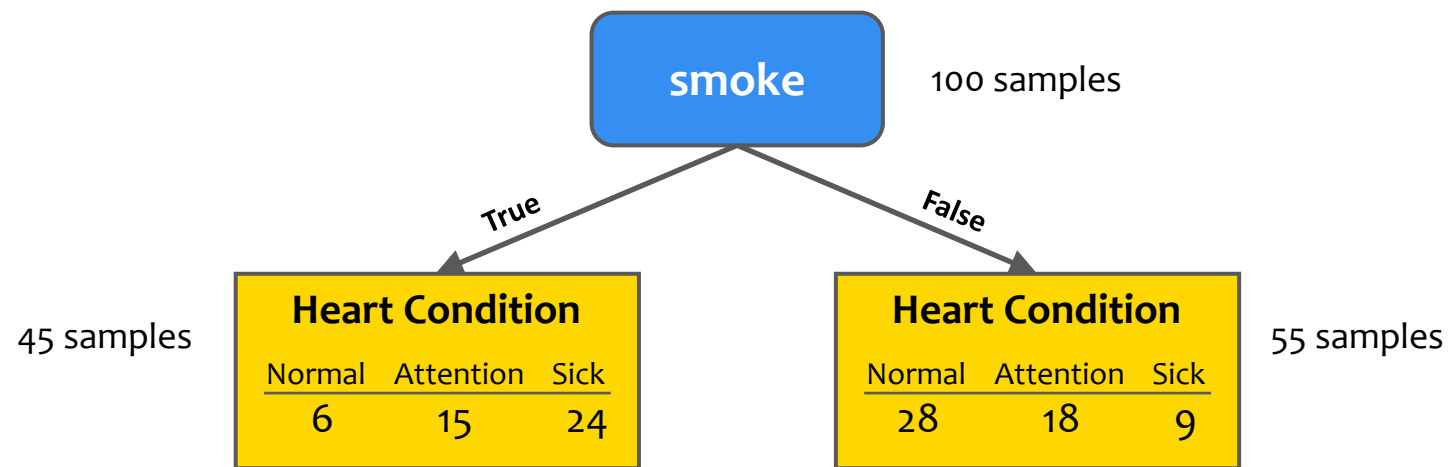


Gini impurity for a node i

$$G_i = 1 - \sum_{c=1}^{n_classes} p_{i,c}^2$$

$p_{i,c}$ is the **probability** of **class c** of i^{th} node.

Ratio of **class c instances** among **all the instances** in the i^{th} node.



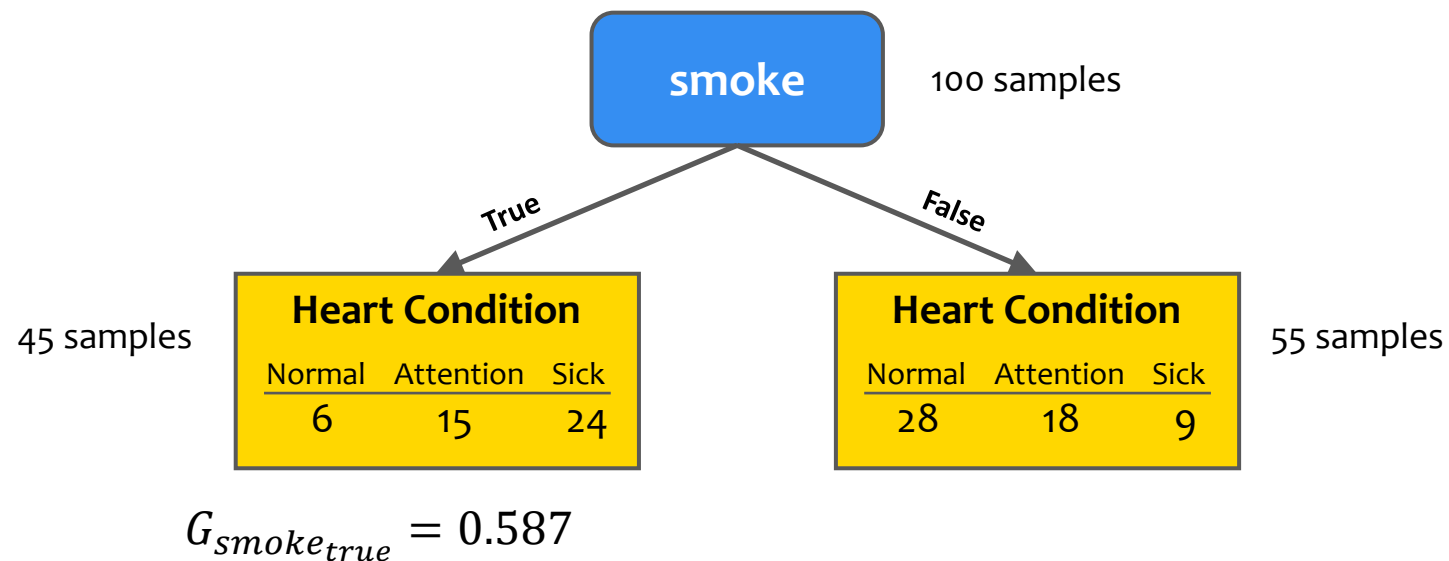
$$G_{smoke_{true}} = 1 - 0.4132 = 0.587$$

Gini impurity for a node i

$$G_i = 1 - \sum_{c=1}^{n_classes} p_{i,c}^2$$

$p_{i,c}$ is the **probability** of **class c** of i^{th} node.

Ratio of **class c instances** among **all the instances** in the i^{th} node.



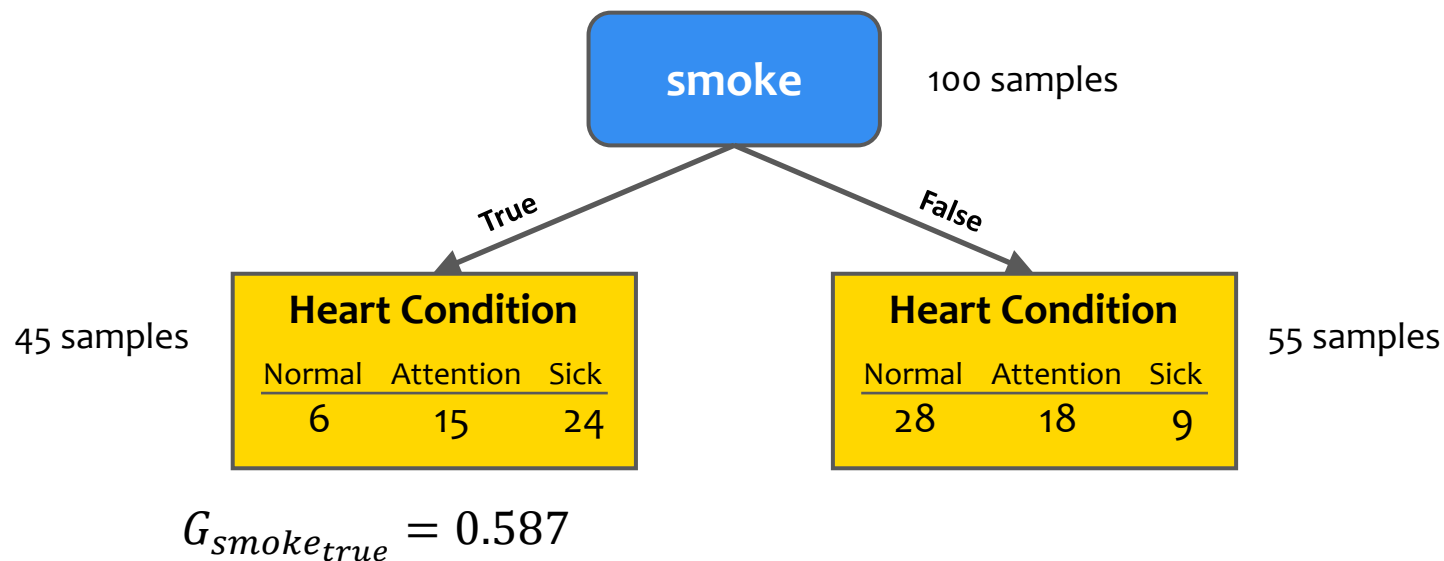
$$G_{smoke_{false}} = 1 - ((\text{the prob. of "Normal"})^2 + (\text{the prob. of "Attention"})^2 + (\text{the prob. of "Sick"})^2)$$

Gini impurity for a node i

$$G_i = 1 - \sum_{c=1}^{n_classes} p_{i,c}^2$$

$p_{i,c}$ is the **probability** of **class c** of i^{th} node.

Ratio of **class c instances** among **all the instances** in the i^{th} node.



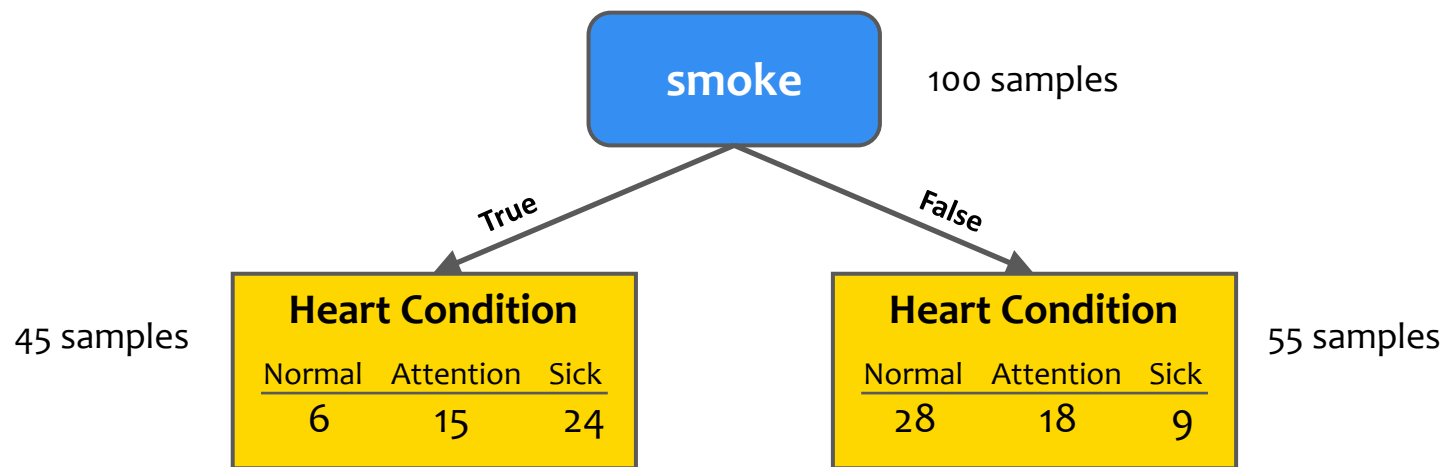
$$G_{smoke_{false}} = 1 - \left(\left(\frac{28}{28+18+9} \right)^2 + \left(\frac{18}{28+18+9} \right)^2 + \left(\frac{9}{28+18+9} \right)^2 \right)$$

Gini impurity for a node i

$$G_i = 1 - \sum_{c=1}^{n_classes} p_{i,c}^2$$

$p_{i,c}$ is the **probability** of **class c** of i^{th} node.

Ratio of **class c instances** among **all the instances** in the i^{th} node.



$$G_{smoke_{true}} = 0.587$$

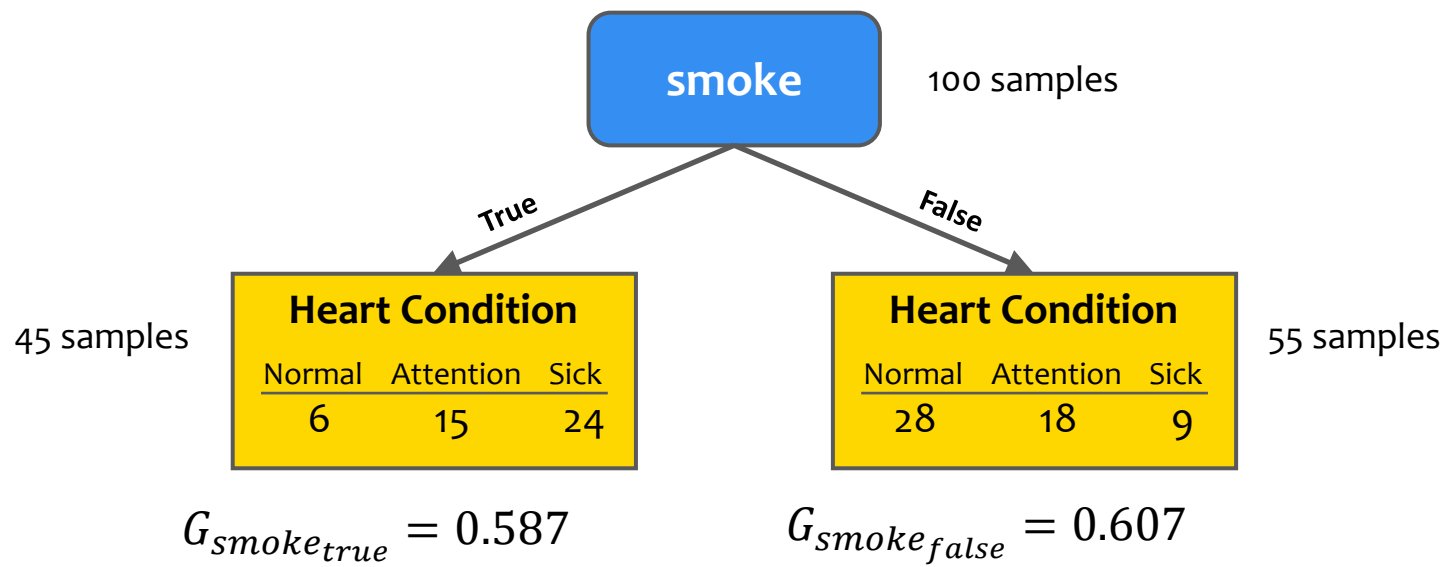
$$G_{smoke_{false}} = 1 - 0.393 = 0.607$$

Gini impurity for a node i

$$G_i = 1 - \sum_{c=1}^{n_classes} p_{i,c}^2$$

$p_{i,c}$ is the **probability** of **class c** of i^{th} node.

Ratio of **class c instances** among **all the instances** in the i^{th} node.

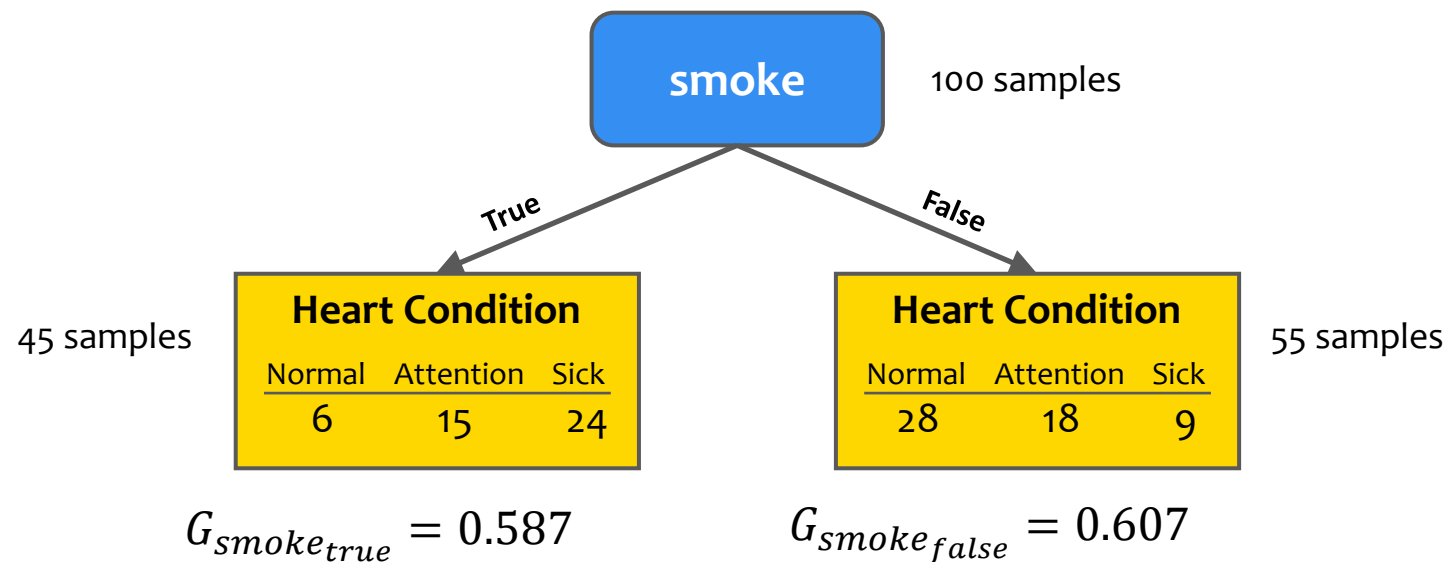


Gini impurity for a node i

$$G_i = 1 - \sum_{c=1}^{n_classes} p_{i,c}^2$$

$p_{i,c}$ is the **probability** of **class c** of i^{th} node.

Ratio of **class c instances** among **all the instances** in the i^{th} node.



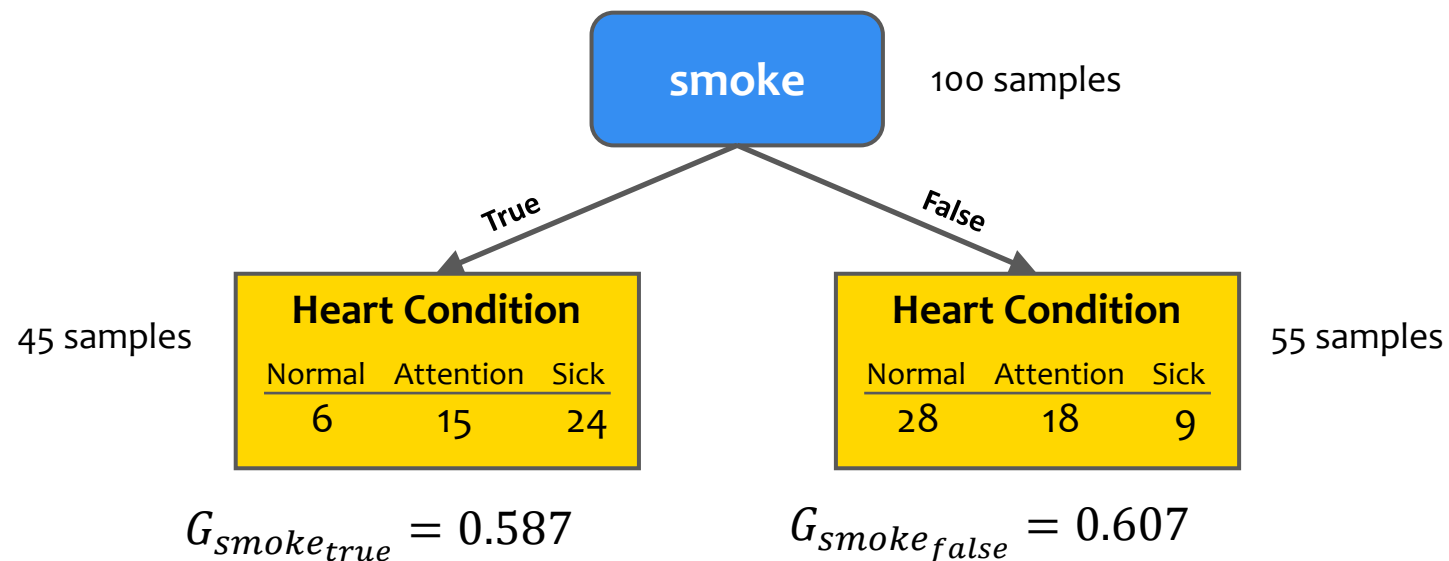
 A **pure node** has **Gini = 0**.

Gini impurity for a node i

$$G_i = 1 - \sum_{c=1}^{n_classes} p_{i,c}^2$$

$p_{i,c}$ is the **probability** of **class c** of i^{th} node.

Ratio of **class c instances** among **all the instances** in the i^{th} node.



Because the **leaf on the left** has a **different** number of samples from the **leaf on the right**, so the **Total Gini Impurity** for the feature **Smoke** is the **weighted average Gini Impurity** of its **sub-nodes**.



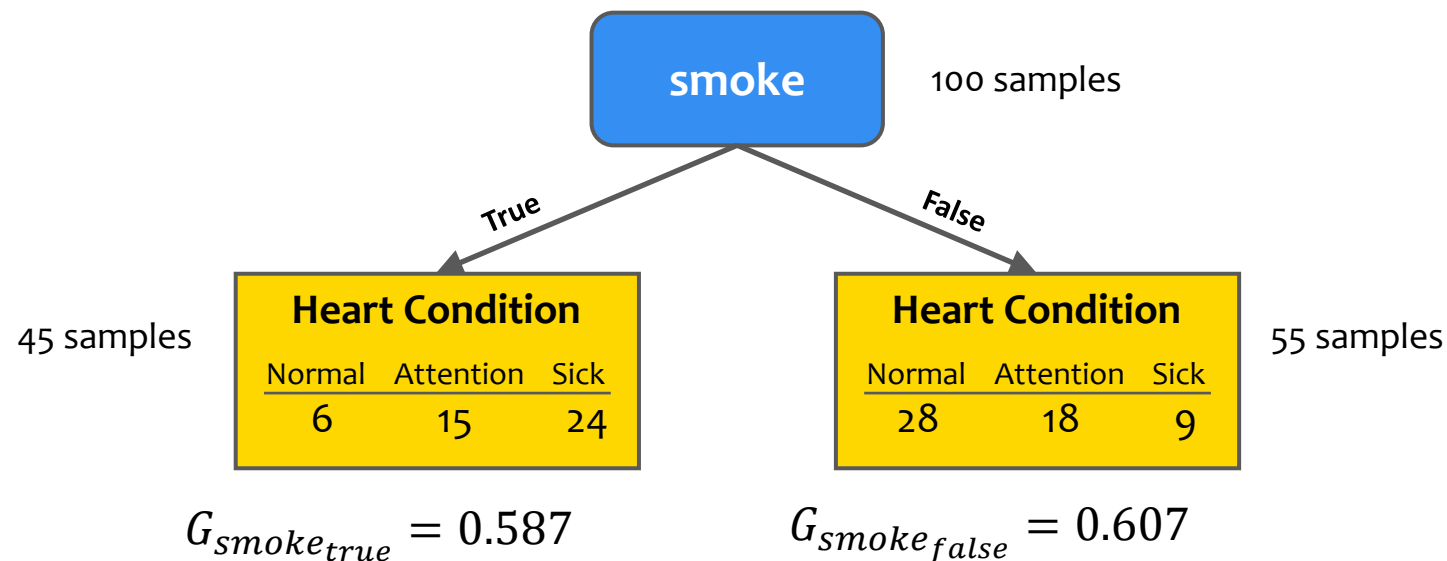
This is exactly the **cost function** used for **CART** during training.

Gini impurity for a node i

$$G_i = 1 - \sum_{c=1}^{n_classes} p_{i,c}^2$$

$p_{i,c}$ is the **probability** of **class c** of i^{th} node.

Ratio of **class c** instances among **all the instances** in the i^{th} node.



CART cost function for feature k for classification

$$J(k) = \frac{m_{left}}{m} G_{left} + \frac{m_{right}}{m} G_{right}$$

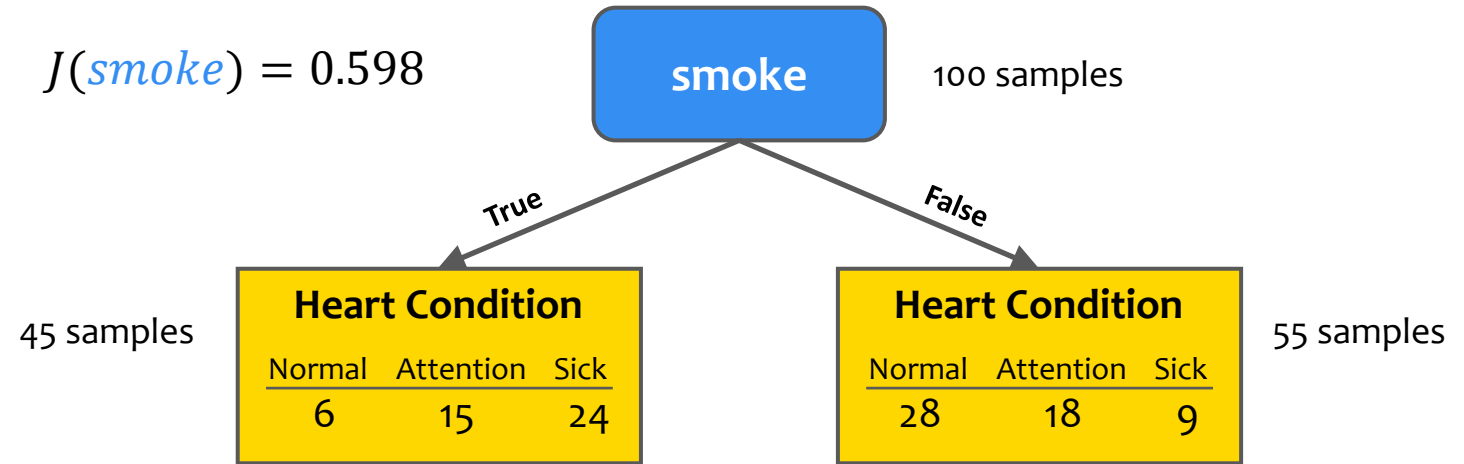
where $\begin{cases} G_{left/right} & \text{measures the **impurity** of the left/right subset,} \\ m_{left/right} & \text{is the **number of instances** in the left/right subset.} \end{cases}$

Because the **leaf on the left** has a **different** number of samples from the **leaf on the right**, so the **Total Gini Impurity** for the feature **Smoke** is the **weighted average Gini Impurity** of its **sub-nodes**.

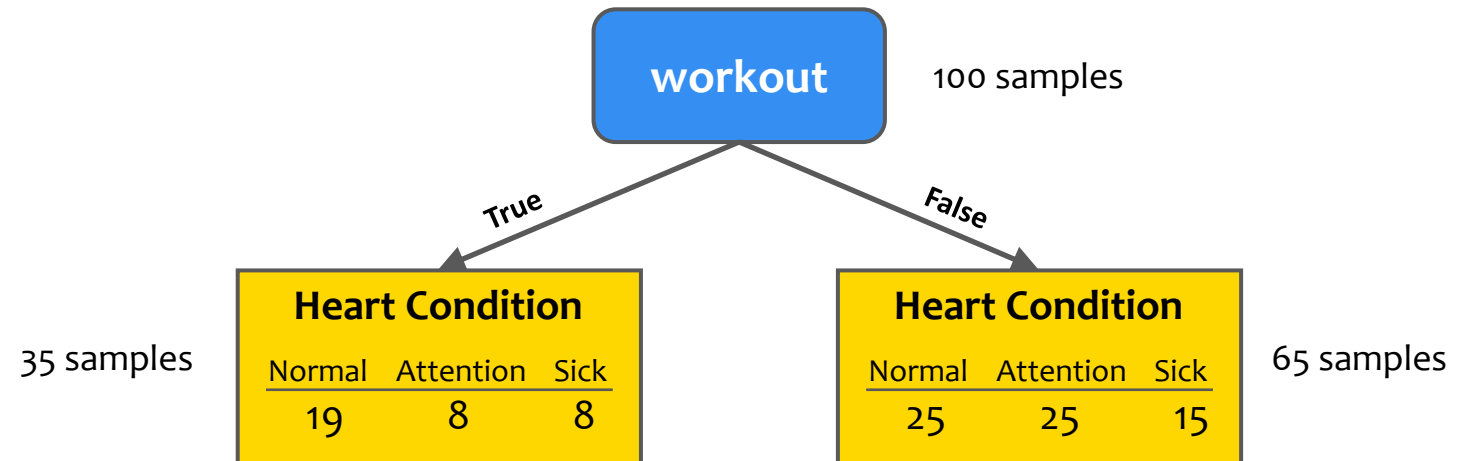
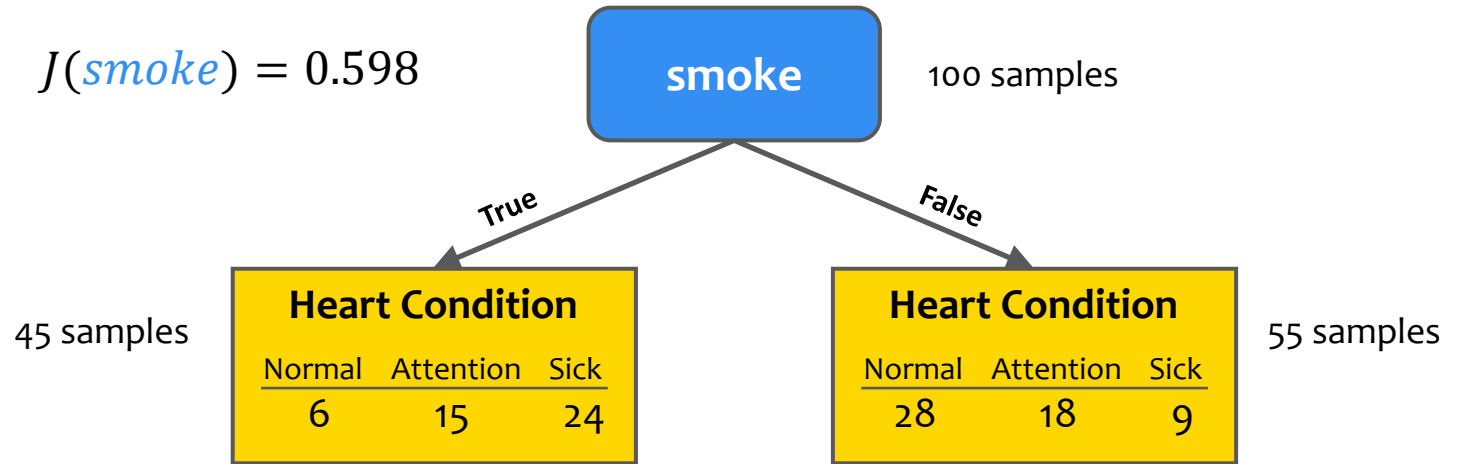


This is exactly the **cost function** used for **CART** during training.

$$J(\textit{smoke}) = 0.598$$

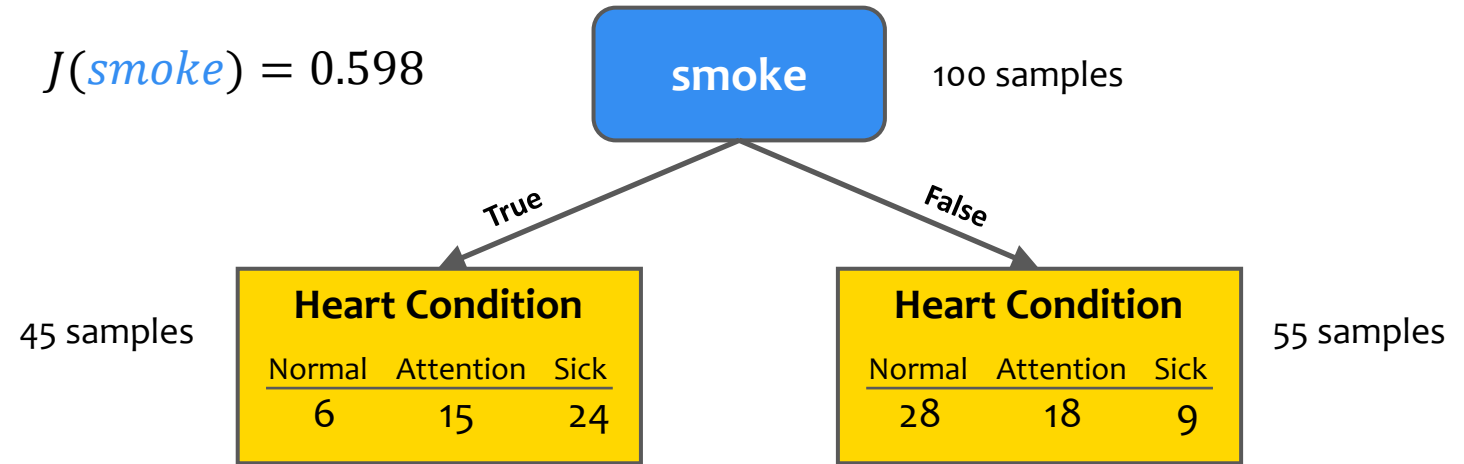


Now, let's compute the
Total Gini Impurity for the
feature **Workout**.

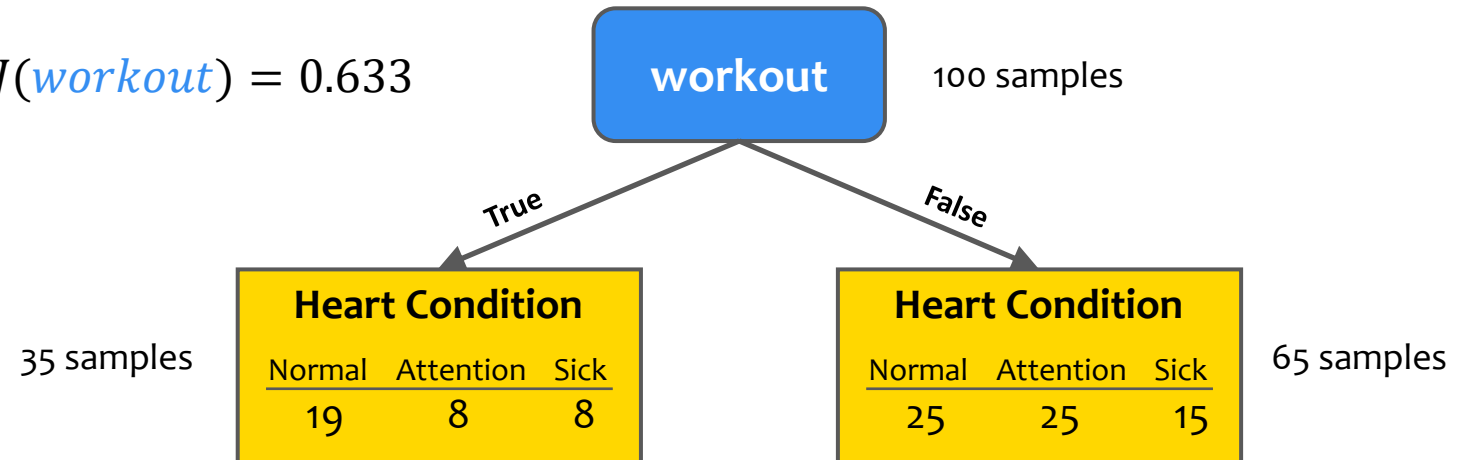


Now, let's compute the
Total Gini Impurity for the
feature **Workout**.

$$J(\text{smoke}) = 0.598$$



$$J(\text{workout}) = 0.633$$



smoke	workout	weight	heart condition
No	Yes	85.0	Sick
No	Yes	78.0	Normal
Yes	No	75.0	Sick
No	Yes	84.0	Normal
No	Yes	72.0	Normal
No	No	78.0	Sick
Yes	No	120.0	Attention
...	...	...	...
Yes	Yes	84.0	Sick
No	No	98.0	Attention

Now, let's compute the **Total Gini Impurity** for the feature **weight**.

$m = 100$ samples

smoke	workout	weight	heart condition
No	Yes	85.0	Sick
No	Yes	78.0	Normal
Yes	No	75.0	Sick
No	Yes	84.0	Normal
No	Yes	72.0	Normal
No	No	78.0	Sick
Yes	No	120.0	Attention
...	...	...	...
Yes	Yes	84.0	Sick
No	No	98.0	Attention

$m = 100$ samples

Now, let's compute the **Total Gini Impurity** for the feature **weight**.

Because **weight** is a **numerical feature**, we will need to define a **threshold** to make the **rule/statement**.

smoke	workout	weight	heart condition
No	Yes	85.0	Sick
No	Yes	78.0	Normal
Yes	No	75.0	Sick
No	Yes	84.0	Normal
No	Yes	72.0	Normal
No	No	78.0	Sick
Yes	No	120.0	Attention
...	...	...	...
Yes	Yes	84.0	Sick
No	No	98.0	Attention

$m = 100$ samples

Now, let's compute the **Total Gini Impurity** for the feature **weight**.

Because **weight** is a **numerical feature**, we will need to define a **threshold** to make the **rule/statement**.

Let's see a strategy for that!

A possible strategy...

weight	heart condition
85.0	Sick
78.0	Normal
75.0	Sick
84.0	Normal
72.0	Normal
78.0	Sick
120.0	Attention
...	...
84.0	Sick
98.0	Attention

$m = 100$ samples

A possible strategy...

weight	heart condition
56	Normal
56	Attention
57	Attention
57	Normal
57	Attention
60	Attention
63	Normal
...	...
140	Sick
140	Attention

$m = 100$ samples

First of all, we **sort** the rows by **weight** in **ascending order**.

A possible strategy...

weight	heart condition
56	Normal
56	Attention
57	Attention
57	Normal
57	Attention
60	Attention
63	Normal
...	...
140	Sick
140	Attention

$m = 100$ samples

First of all, we **sort** the rows by **weight** in **ascending order**.

Then, we calculate the **average weight** for all **adjacent instances** to become possible **thresholds**.

A possible strategy...

weight	heart condition
56	Normal
56	Attention
56.5	Attention
57	Normal
57	Attention
60	Attention
63	Normal
...	...
140	Sick
140	Attention

$m = 100$ samples

First of all, we **sort** the rows by **weight** in **ascending order**.

Then, we calculate the **average weight** for all **adjacent instances** to become possible **thresholds**.

A possible strategy...

weight	heart condition
56	Normal
56	Attention
56.5	Attention
57	Normal
57	Attention
57	Attention
58.5	Attention
60	Attention
61.5	Normal
...	...
140	Sick
140	Attention

$m = 100$ samples

First of all, we **sort** the rows by **weight** in **ascending order**.

Then, we calculate the **average weight** for all **adjacent instances** to become possible **thresholds**.

A possible strategy...

	weight	heart condition
	56	Normal
56	56	Attention
56.5	57	Attention
57	57	Normal
57	57	Attention
58.5	60	Attention
61.5	63	Normal
	...	...
	140	Sick
140	140	Attention

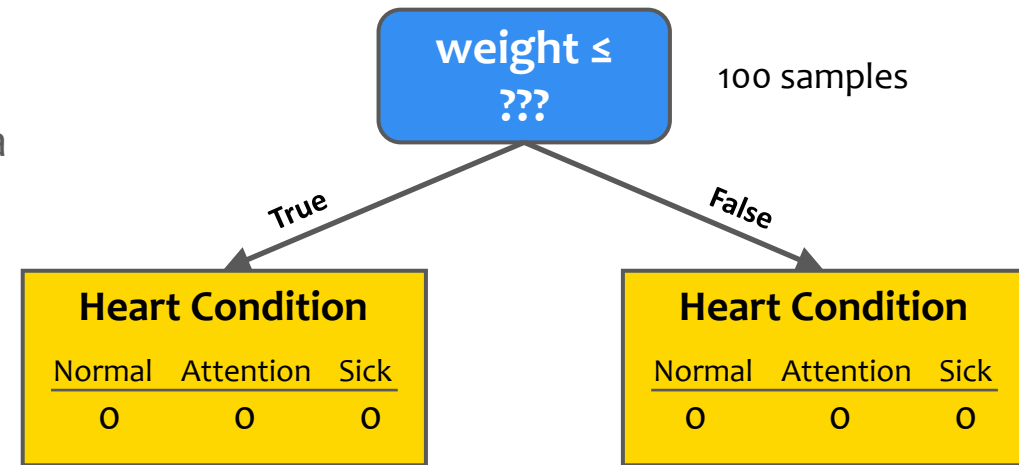
$m = 100$ samples

A possible strategy...

weight	heart condition
56	Normal
56	Attention
56.5	Attention
57	Normal
57	Attention
58.5	Attention
60	Attention
61.5	Normal
...	...
140	Sick
140	Attention

$m = 100$ samples

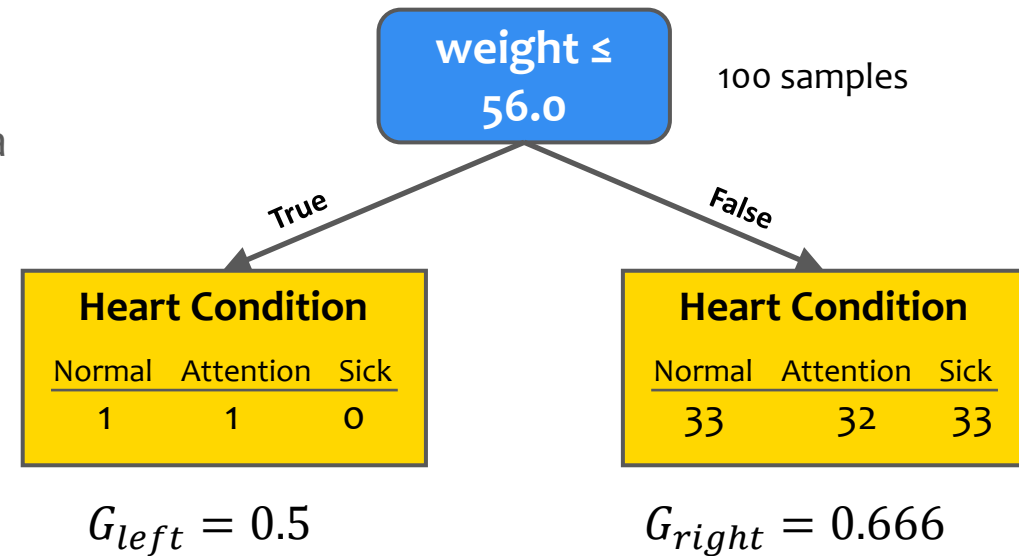
For each **threshold**, we build a **statement/rule...**



A possible strategy...

weight	heart condition
56	Normal
56	Attention
56.5	Attention
57	Normal
57	Attention
57	Attention
58.5	Attention
60	Attention
61.5	Normal

For each **threshold**, we build a **statement/rule...**



**CART cost function for
feature k and threshold t_k for classification**

$$J(k, t_k) = \frac{m_{\text{left}}}{m} G_{\text{left}} + \frac{m_{\text{right}}}{m} G_{\text{right}}$$

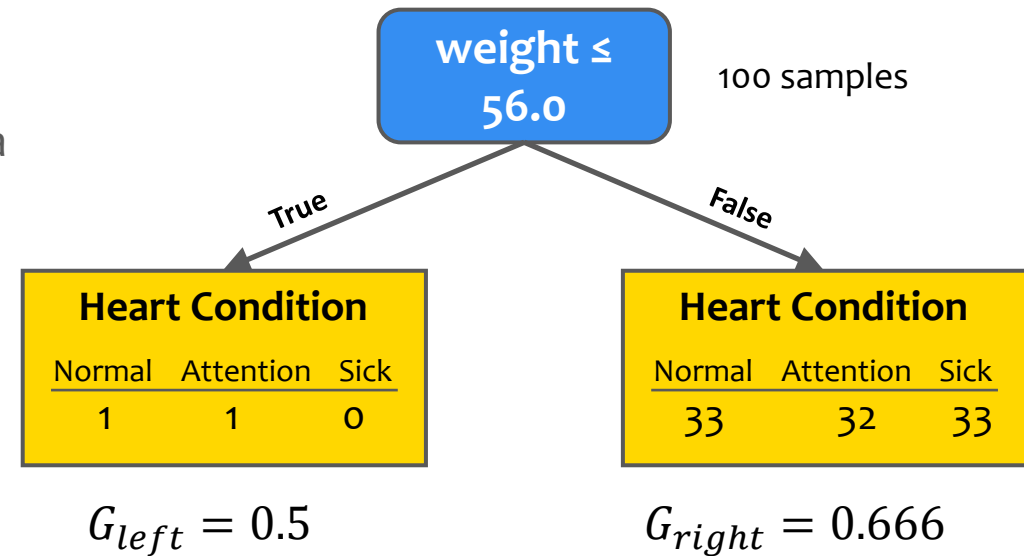
where $\begin{cases} G_{\text{left/right}} \\ m_{\text{left/right}} \end{cases}$ measures the **impurity** of the left/right subset.
is the **number of instances** in the left/right subset.

A possible strategy...

weight	heart condition
56	Normal
56	Attention
56.5	Attention
57	Normal
57	Attention
57	Attention
58.5	Attention
60	Attention
61.5	Normal
...	...
140	Sick
140	Attention

$m = 100$ samples

For each **threshold**, we build a **statement/rule...**



$$J(\text{weight}, 56) = \frac{2}{100} 0.5 + \frac{98}{100} 0.666 = 0.663$$

A possible strategy...

weight	heart condition	
56	Normal	$J(\text{weight}, 56) = 0.663$
56	Attention	
56.5	Attention	
57	Normal	
57	Attention	
58.5	Attention	
61.5	Normal	
...	...	
140	Sick	
140	Attention	

$m = 100$ samples

A possible strategy...

weight	heart condition	
56	Normal	$J(\text{weight}, 56) = 0.663$
56.5	Attention	$J(\text{weight}, 56.5) = 0.663$
57	Attention	$J(\text{weight}, 57) = 0.656$
57	Normal	$J(\text{weight}, 57) = 0.656$
57	Attention	$J(\text{weight}, 57) = 0.656$
58.5	Attention	$J(\text{weight}, 58.5) = 0.656$
61.5	Normal	$J(\text{weight}, 61.5) = 0.652$
...	...	...
140	Sick	$J(\text{weight}, 140) = 0.666$
140	Attention	

$m = 100$ samples

A possible strategy...

	weight	heart condition	
	56	Normal	
56	56	Attention	$J(\text{weight}, 56) = 0.663$
56.5	57	Attention	$J(\text{weight}, 56.5) = 0.663$
57	57	Normal	$J(\text{weight}, 57) = 0.656$
57	57	Attention	$J(\text{weight}, 57) = 0.656$
58.5	60	Attention	$J(\text{weight}, 58.5) = 0.656$
61.5	63	Normal	$J(\text{weight}, 61.5) = 0.652$
	...	...	...
	140	Sick	
140	140	Attention	$J(\text{weight}, 140) = 0.666$

$m = 100$ samples

For each **threshold**, we build a **statement/rule...**

We select the **threshold** with the **lowest gini impurity...**

$J(\text{weight}, \mathbf{94.5}) = 0.636$

A possible strategy...

weight	heart condition	
	56	Normal
56	56	Attention
56.5	57	Attention
57	57	Normal
57	57	Attention
58.5	60	Attention
61.5	63	Normal
...	...	...
140	140	Sick
140	140	Attention

$m = 100$ samples

For each **threshold**, we build a **statement/rule**...

$$J(\text{weight}, 56) = 0.663$$

$$J(\text{weight}, 56.5) = 0.663$$

$$J(\text{weight}, 57) = 0.656$$

$$J(\text{weight}, 57) = 0.656$$

$$J(\text{weight}, 58.5) = 0.656$$

$$J(\text{weight}, 61.5) = 0.652$$

...

$$J(\text{weight}, 140) = 0.666$$

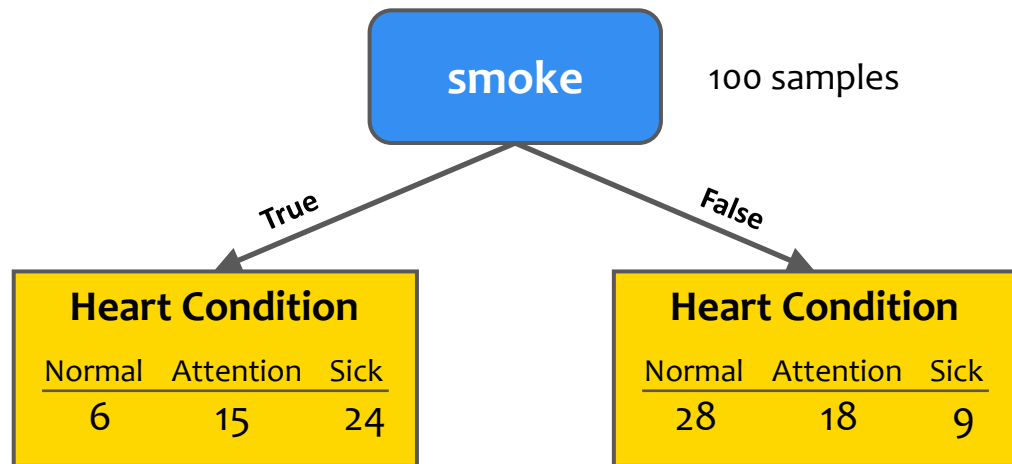


Some algorithms use different strategies to define the **candidate thresholds**.

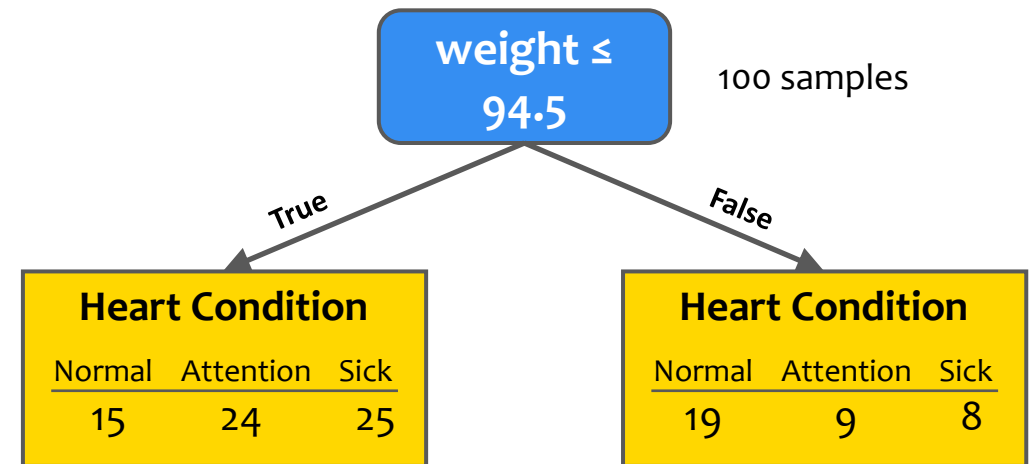
We select the **threshold** with the **lowest gini impurity**...



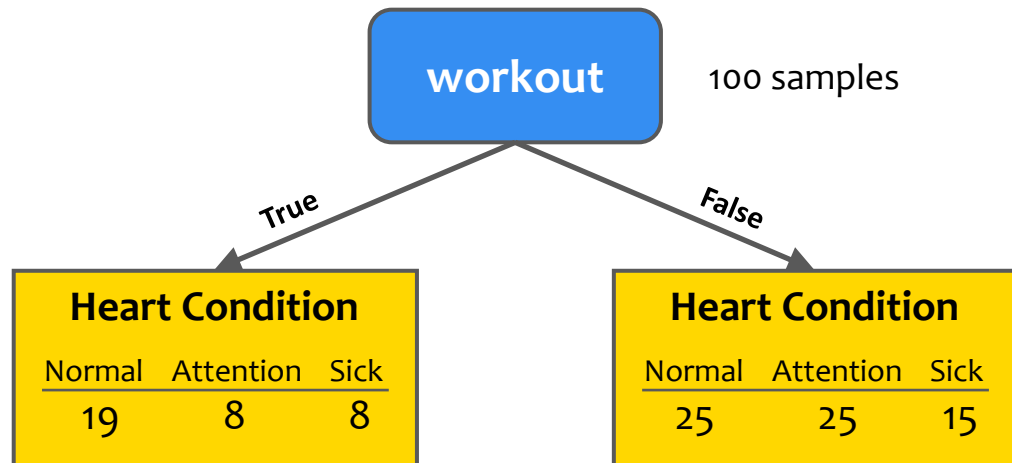
$$J(\text{weight}, \mathbf{94.5}) = 0.636$$



$$J(\text{smoke}) = 0.598$$

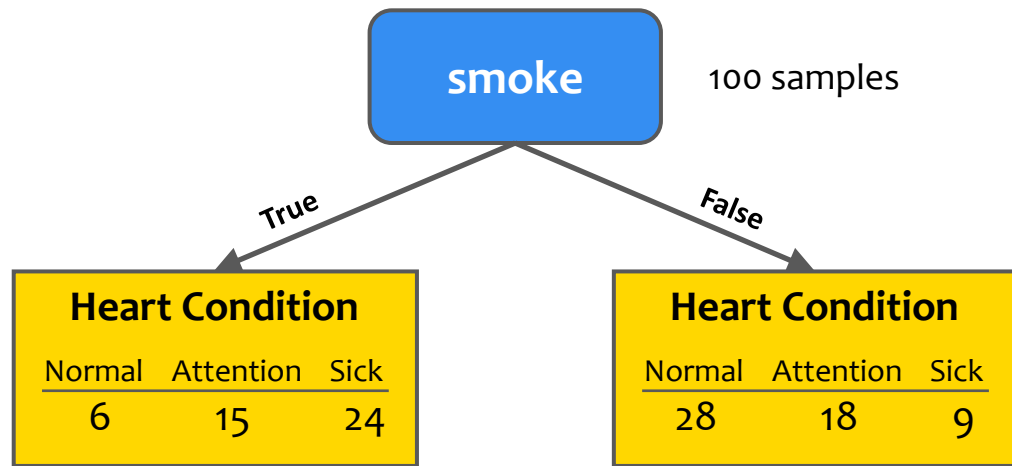


$$J(\text{weight}, 94.5) = 0.636$$

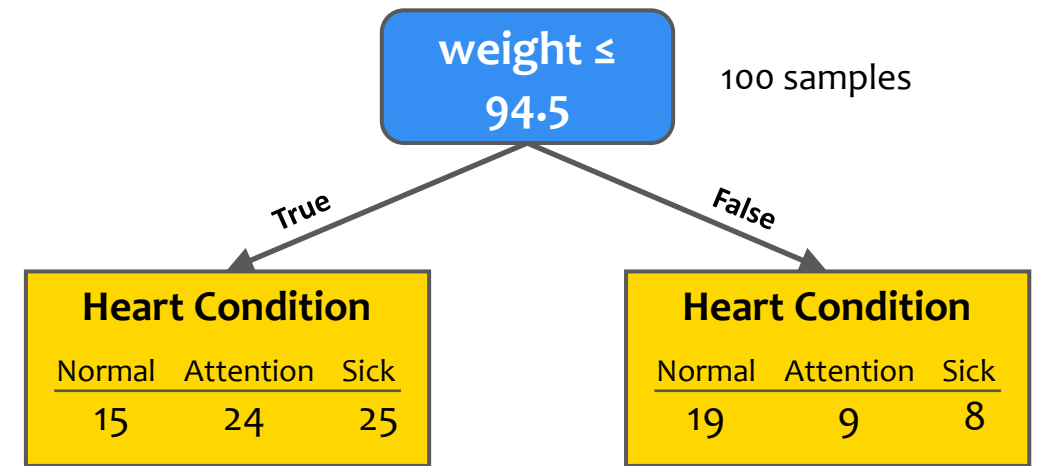


$$J(\text{workout}) = 0.633$$

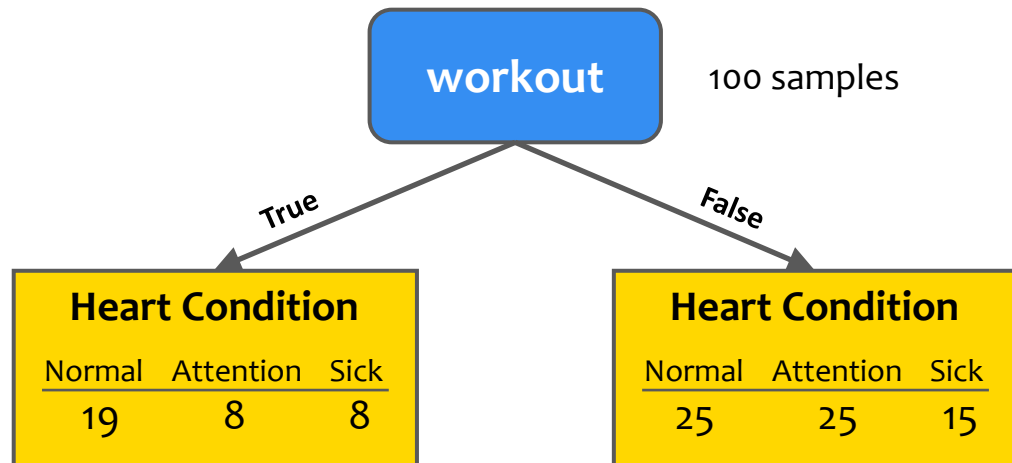
Finally, we compare all **Gini Impurities** and choose rule (**feature + threshold**) the lowest one.



$$J(\text{smoke}) = 0.598$$



$$J(\text{weight}, 94.5) = 0.636$$



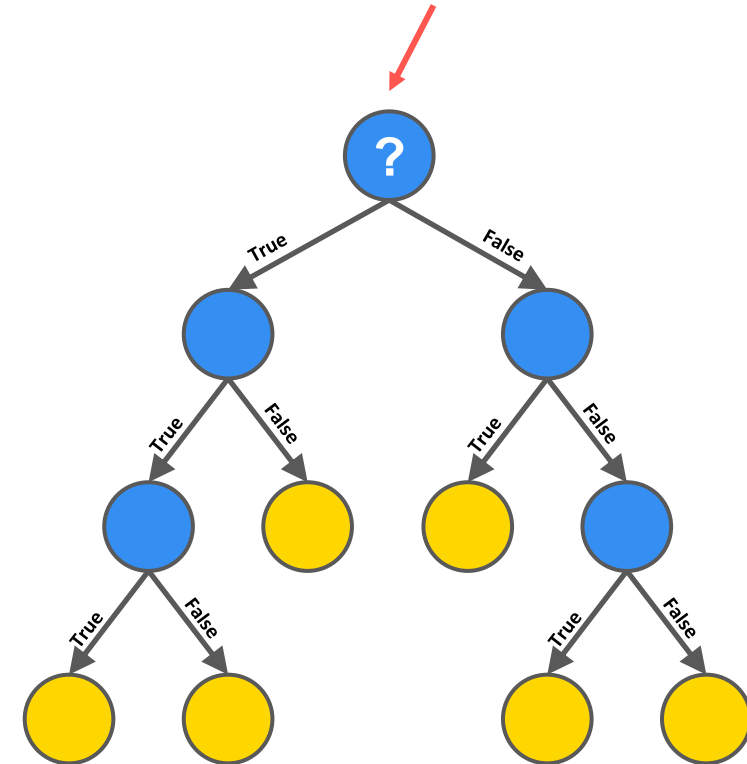
$$J(\text{workout}) = 0.633$$

Finally, we compare all **Gini Impurities** and choose rule (**feature + threshold**) the lowest one.

smoke	workout	weight	heart condition
No	Yes	85.0	Sick
No	Yes	78.0	Normal
Yes	No	75.0	Sick
No	Yes	84.0	Normal
No	Yes	72.0	Normal
No	No	78.0	Sick
Yes	No	120.0	Attention
...	...	...	...
Yes	Yes	84.0	Sick
No	No	98.0	Attention

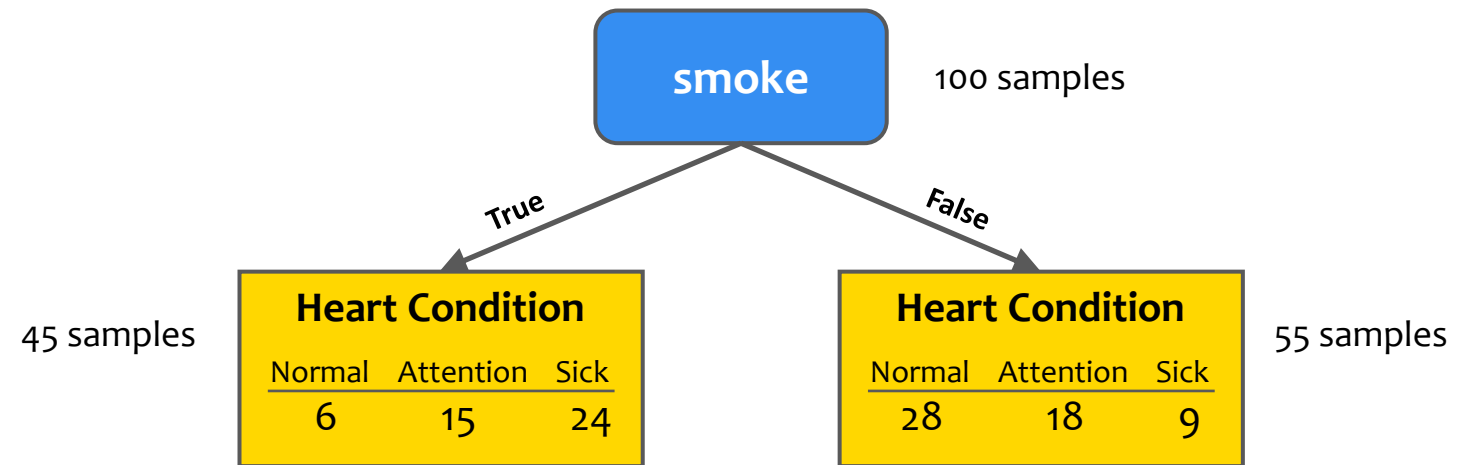
$m = 100$ samples

Which **feature** should be at the **root**?



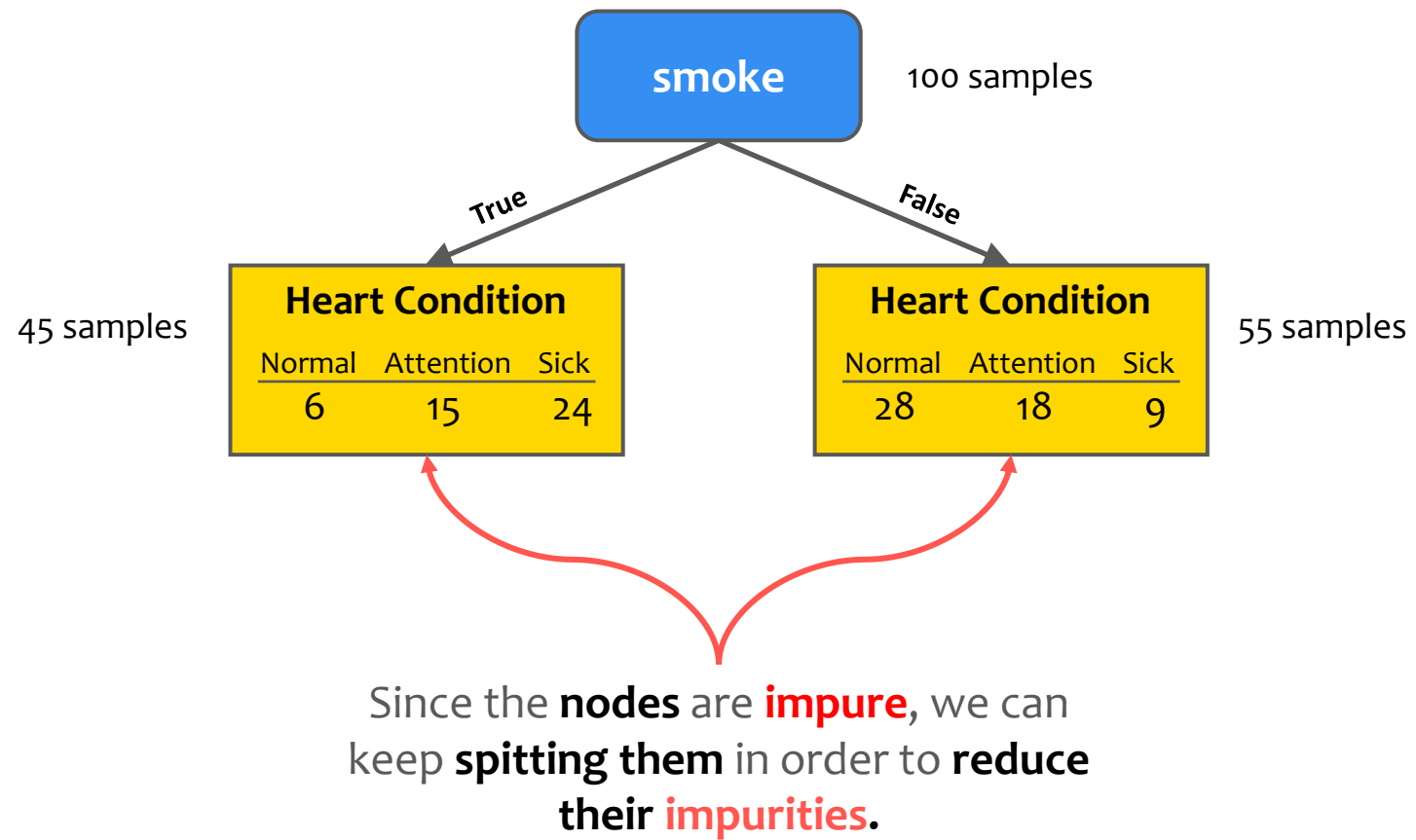
smoke	workout	weight	heart condition
No	Yes	85.0	Sick
No	Yes	78.0	Normal
Yes	No	75.0	Sick
No	Yes	84.0	Normal
No	Yes	72.0	Normal
No	No	78.0	Sick
Yes	No	120.0	Attention
...	...	...	...
Yes	Yes	84.0	Sick
No	No	98.0	Attention

$m = 100$ samples



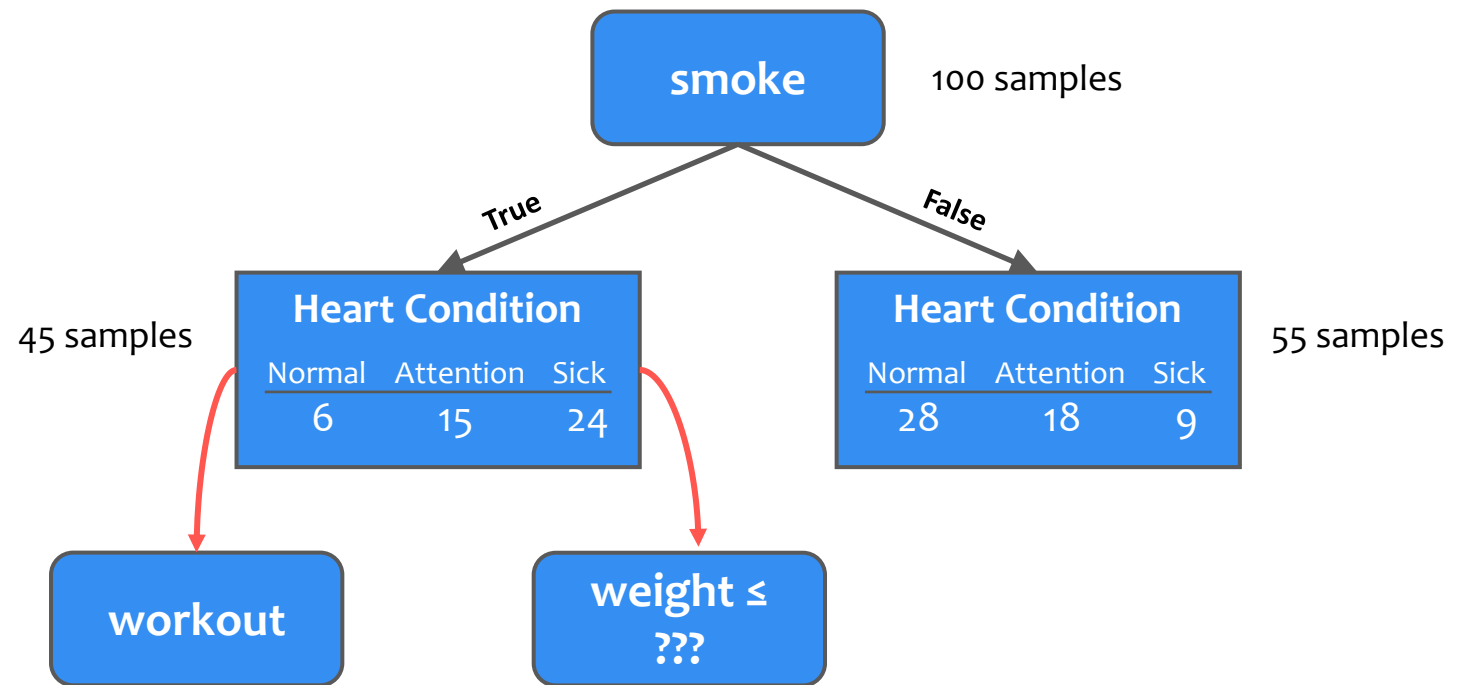
smoke	workout	weight	heart condition
No	Yes	85.0	Sick
No	Yes	78.0	Normal
Yes	No	75.0	Sick
No	Yes	84.0	Normal
No	Yes	72.0	Normal
No	No	78.0	Sick
Yes	No	120.0	Attention
...	...	...	...
Yes	Yes	84.0	Sick
No	No	98.0	Attention

$m = 100$ samples



smoke	workout	weight	heart condition
No	Yes	85.0	Sick
No	Yes	78.0	Normal
Yes	No	75.0	Sick
No	Yes	84.0	Normal
No	Yes	72.0	Normal
No	No	78.0	Sick
Yes	No	120.0	Attention
...	...	...	...
Yes	Yes	84.0	Sick
No	No	98.0	Attention

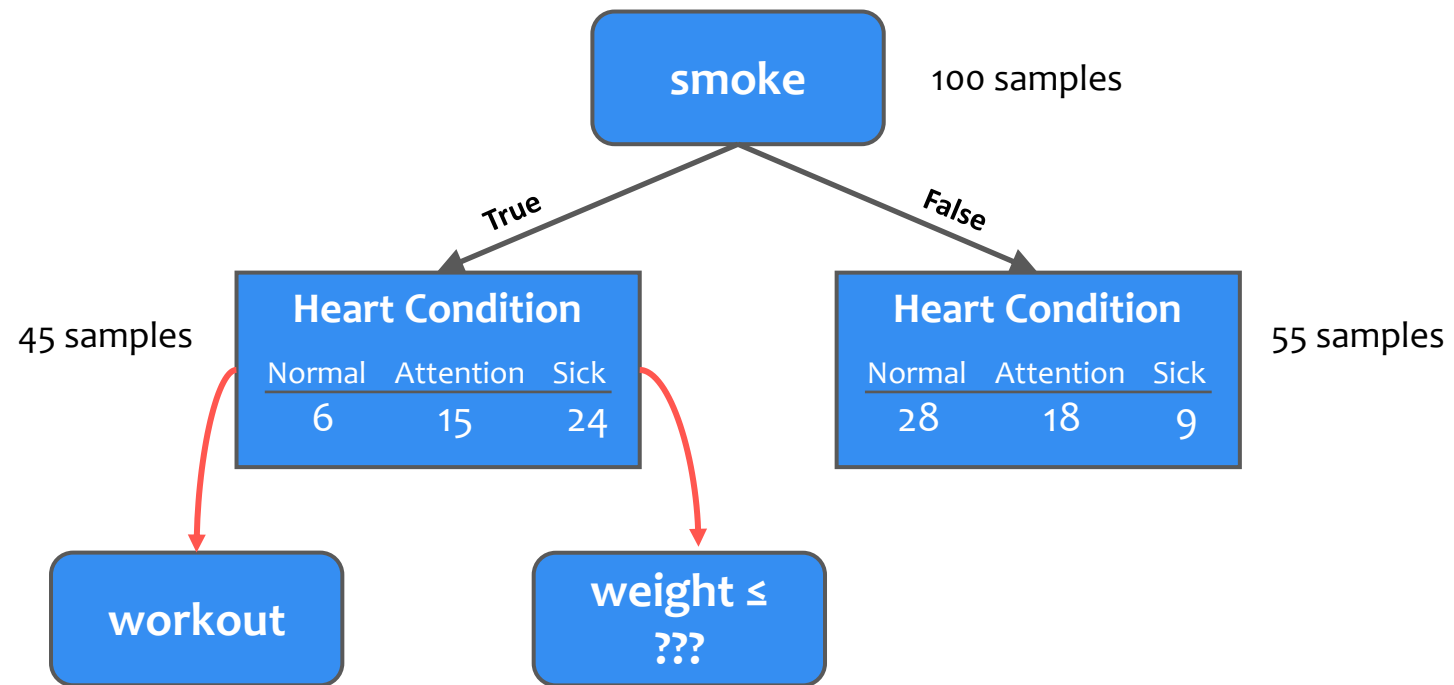
$m = 100$ samples



Should we **split** these smokers (45 people) by the **workout** or **weight** feature?

smoke	workout	weight	heart condition
No	Yes	85.0	Sick
No	Yes	78.0	Normal
Yes	No	75.0	Sick
No	Yes	84.0	Normal
No	Yes	72.0	Normal
No	No	78.0	Sick
Yes	No	120.0	Attention
...	...	...	...
Yes	Yes	84.0	Sick
No	No	98.0	Attention

$m = 100$ samples



Should we **split** these smokers (45 people) by the **workout** or **weight** feature?

Let's repeat **the same steps** we did before.

smoke	workout	weight	heart condition
Yes	No	74	Sick
Yes	No	64	Sick
Yes	No	140	Attention
Yes	No	80	Attention
Yes	No	75	Sick
Yes	No	98	Sick
Yes	No	73	Normal
...	...	...	...
Yes	Yes	105	Sick
Yes	Yes	105	Attention

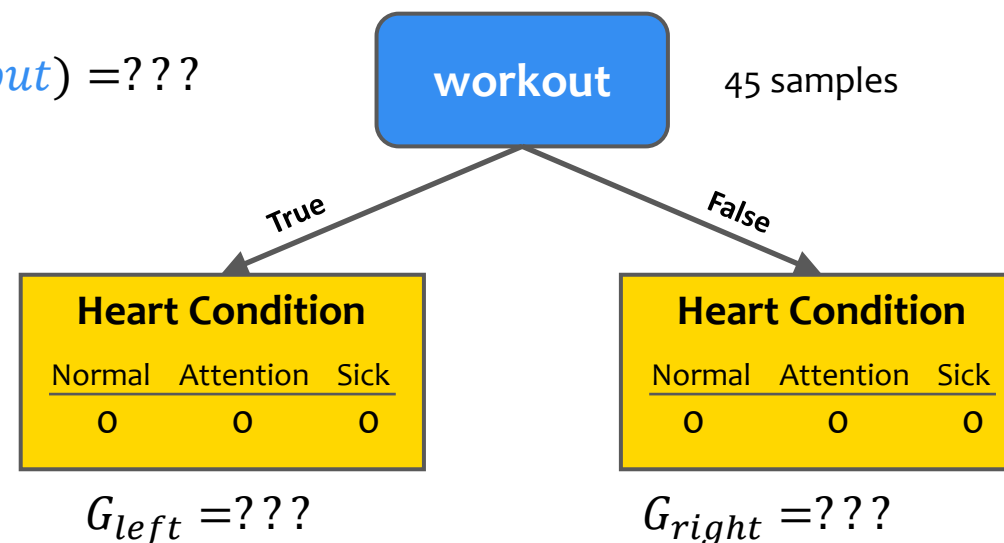
$m = 45$ samples

Only smokers

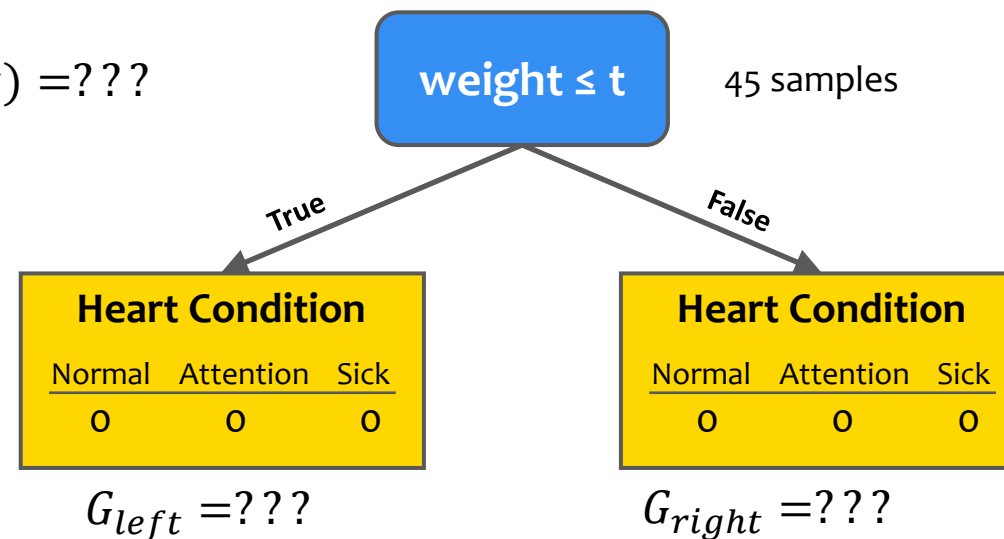
smoke	workout	weight	heart condition
Yes	No	74	Sick
Yes	No	64	Sick
Yes	No	140	Attention
Yes	No	80	Attention
Yes	No	75	Sick
Yes	No	98	Sick
Yes	No	73	Normal
...	...	...	...
Yes	Yes	105	Sick
Yes	Yes	105	Attention

$m = 45$ samples

$$J(\text{workout}) = ???$$



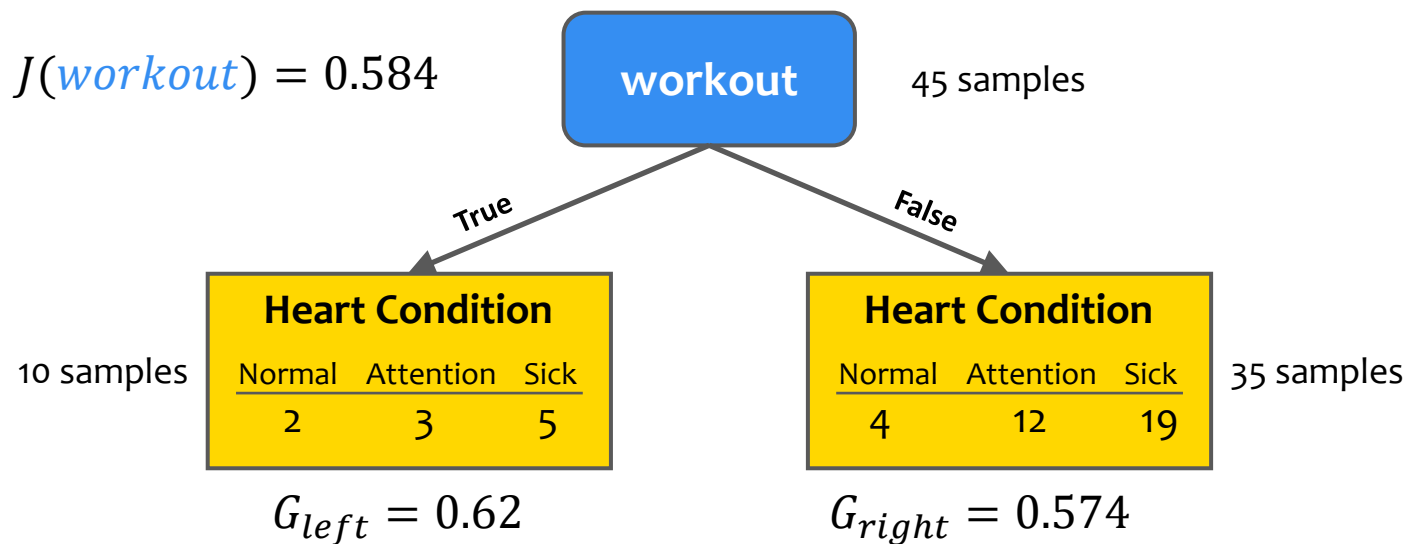
$$J(\text{weight}, t) = ???$$



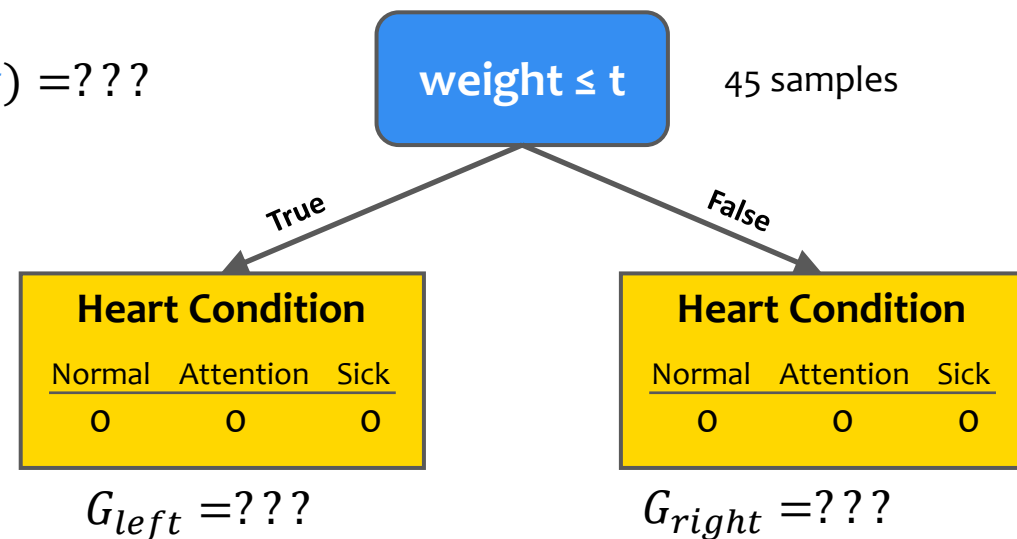
smoke	workout	weight	heart condition
Yes	No	74	Sick
Yes	No	64	Sick
Yes	No	140	Attention
Yes	No	80	Attention
Yes	No	75	Sick
Yes	No	98	Sick
Yes	No	73	Normal
...	...	...	...
Yes	Yes	105	Sick
Yes	Yes	105	Attention

$m = 45$ samples

$$J(\text{workout}) = 0.584$$



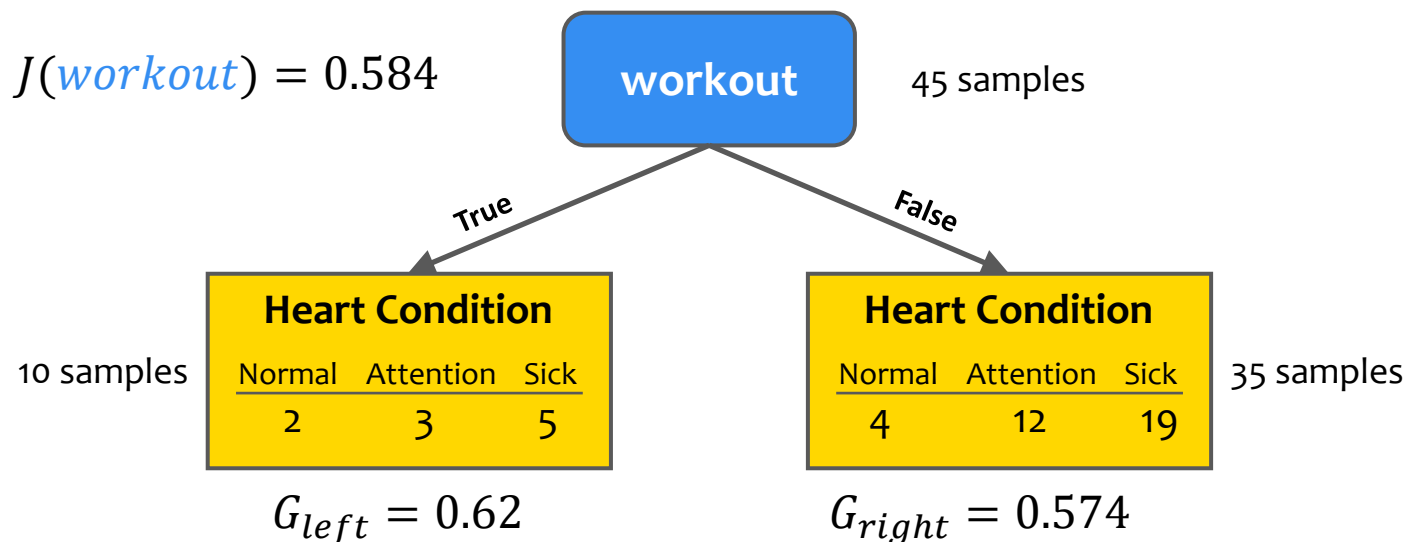
$$J(\text{weight}, t) = ???$$



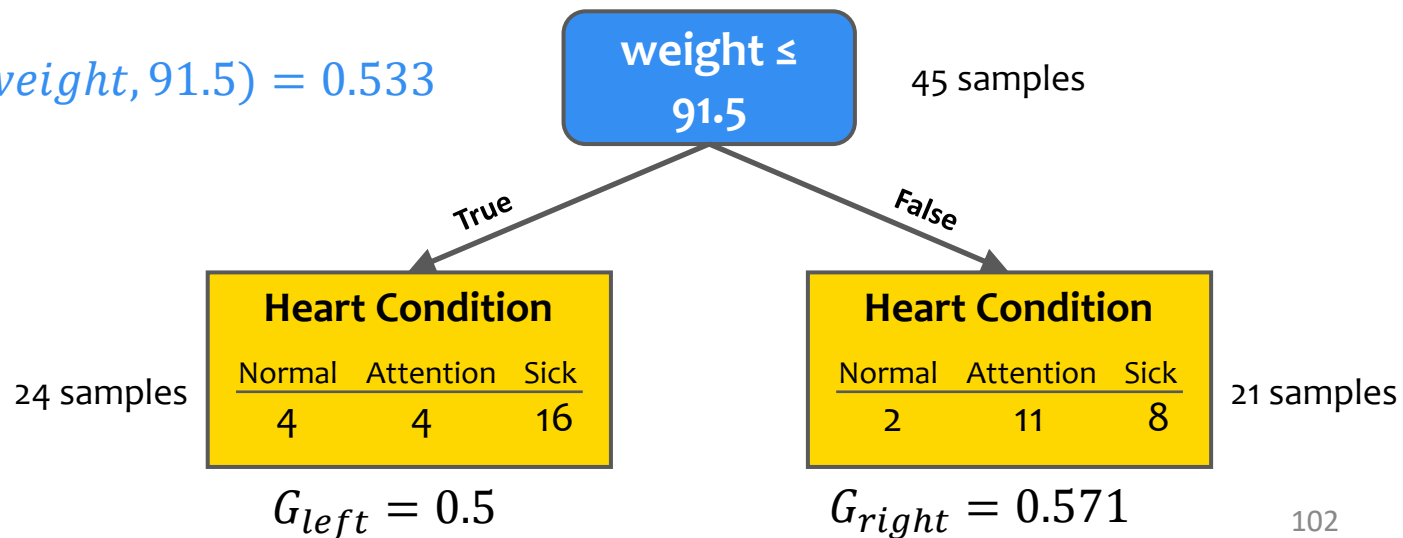
smoke	workout	weight	heart condition
Yes	No	74	Sick
Yes	No	64	Sick
Yes	No	140	Attention
Yes	No	80	Attention
Yes	No	75	Sick
Yes	No	98	Sick
Yes	No	73	Normal
...	...	...	...
Yes	Yes	105	Sick
Yes	Yes	105	Attention

$m = 45$ samples

$$J(\text{workout}) = 0.584$$



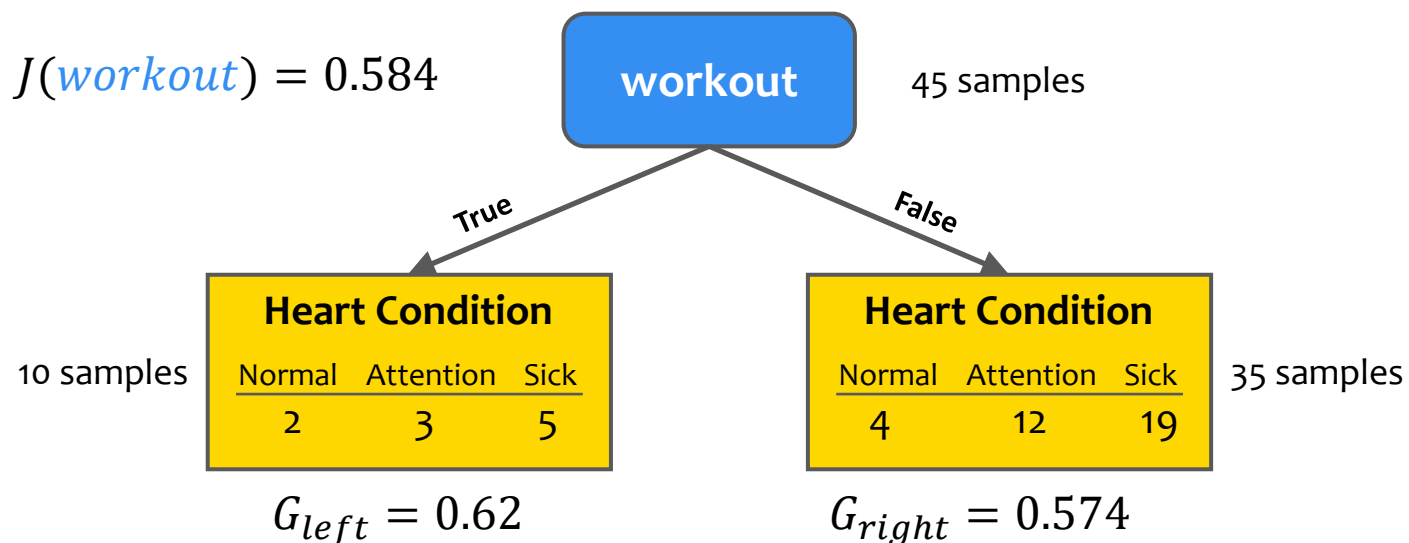
$$J(\text{weight}, 91.5) = 0.533$$



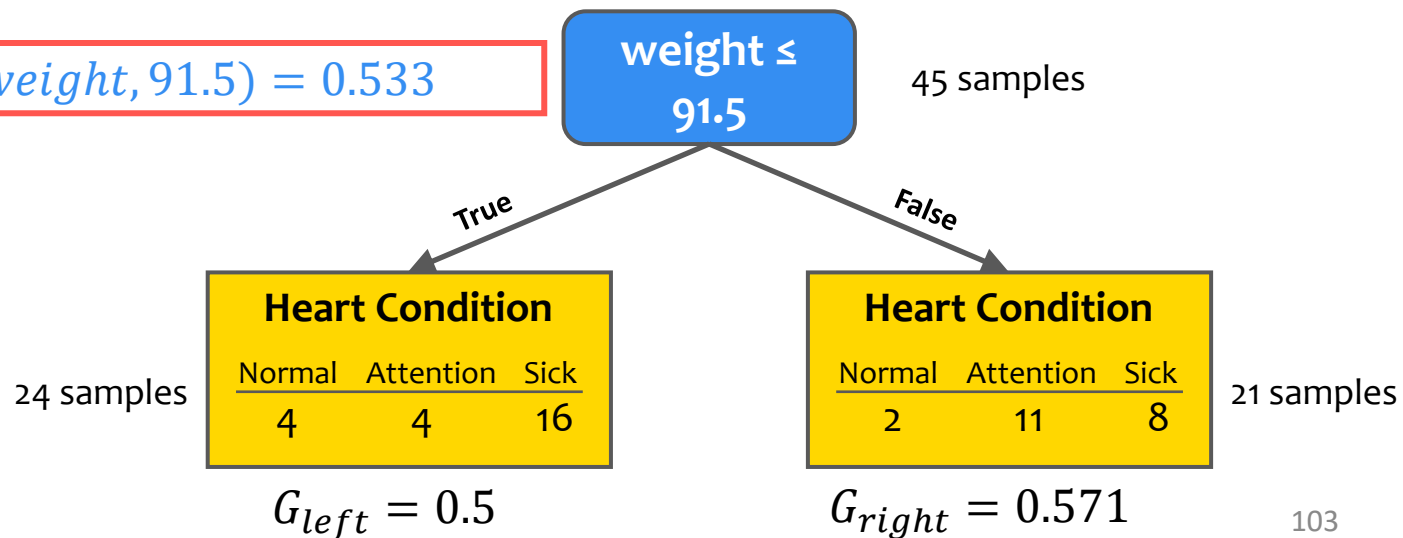
smoke	workout	weight	heart condition
Yes	No	74	Sick
Yes	No	64	Sick
Yes	No	140	Attention
Yes	No	80	Attention
Yes	No	75	Sick
Yes	No	98	Sick
Yes	No	73	Normal
...	...	...	...
Yes	Yes	105	Sick
Yes	Yes	105	Attention

$m = 45$ samples

$$J(\text{workout}) = 0.584$$

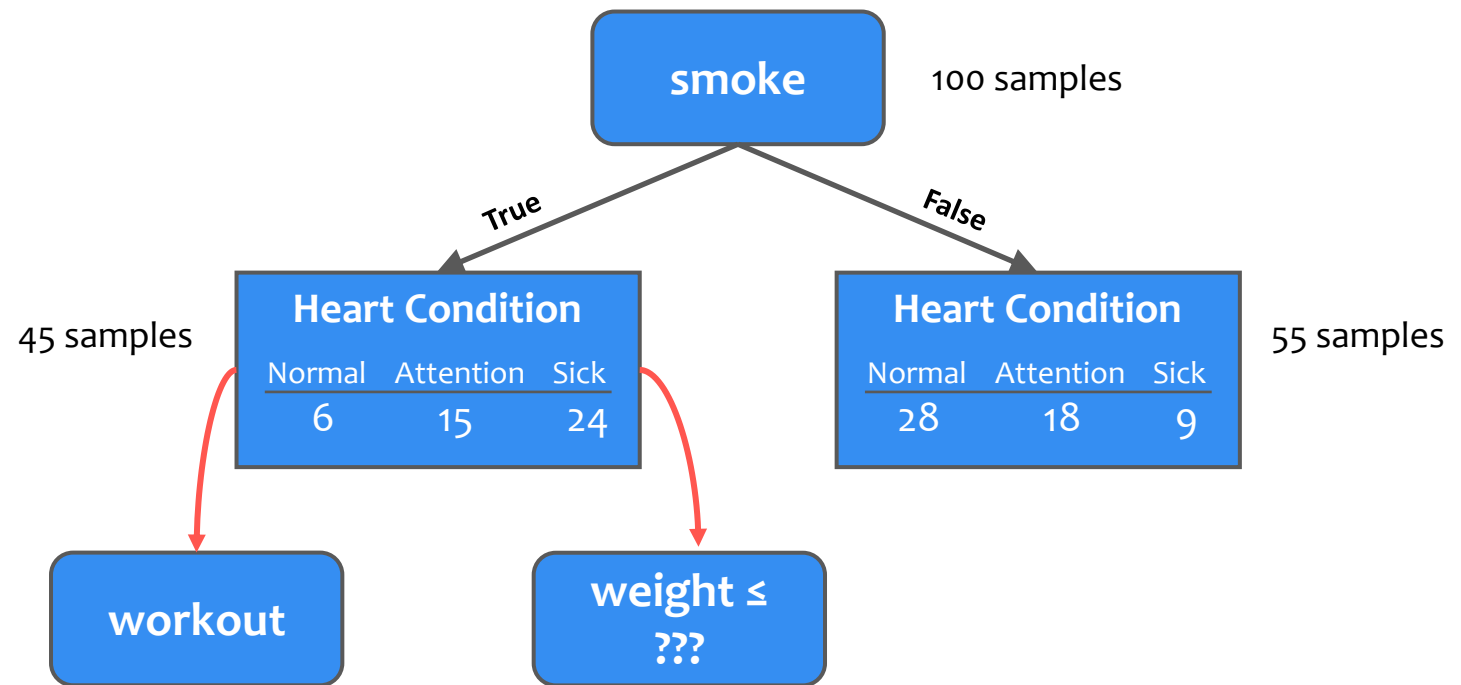


$$J(\text{weight}, 91.5) = 0.533$$



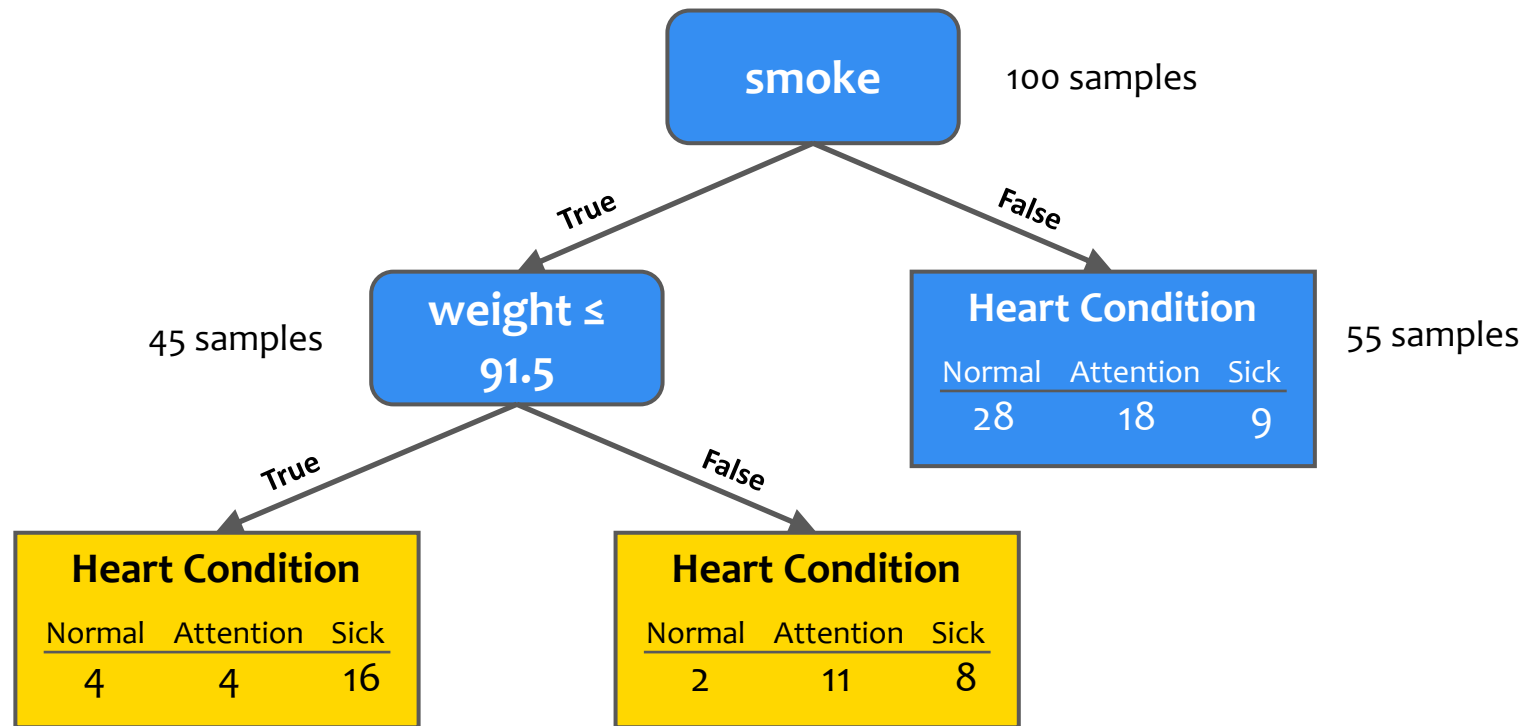
smoke	workout	weight	heart condition
No	Yes	85.0	Sick
No	Yes	78.0	Normal
Yes	No	75.0	Sick
No	Yes	84.0	Normal
No	Yes	72.0	Normal
No	No	78.0	Sick
Yes	No	120.0	Attention
...	...	...	...
Yes	Yes	84.0	Sick
No	No	98.0	Attention

$m = 100$ samples



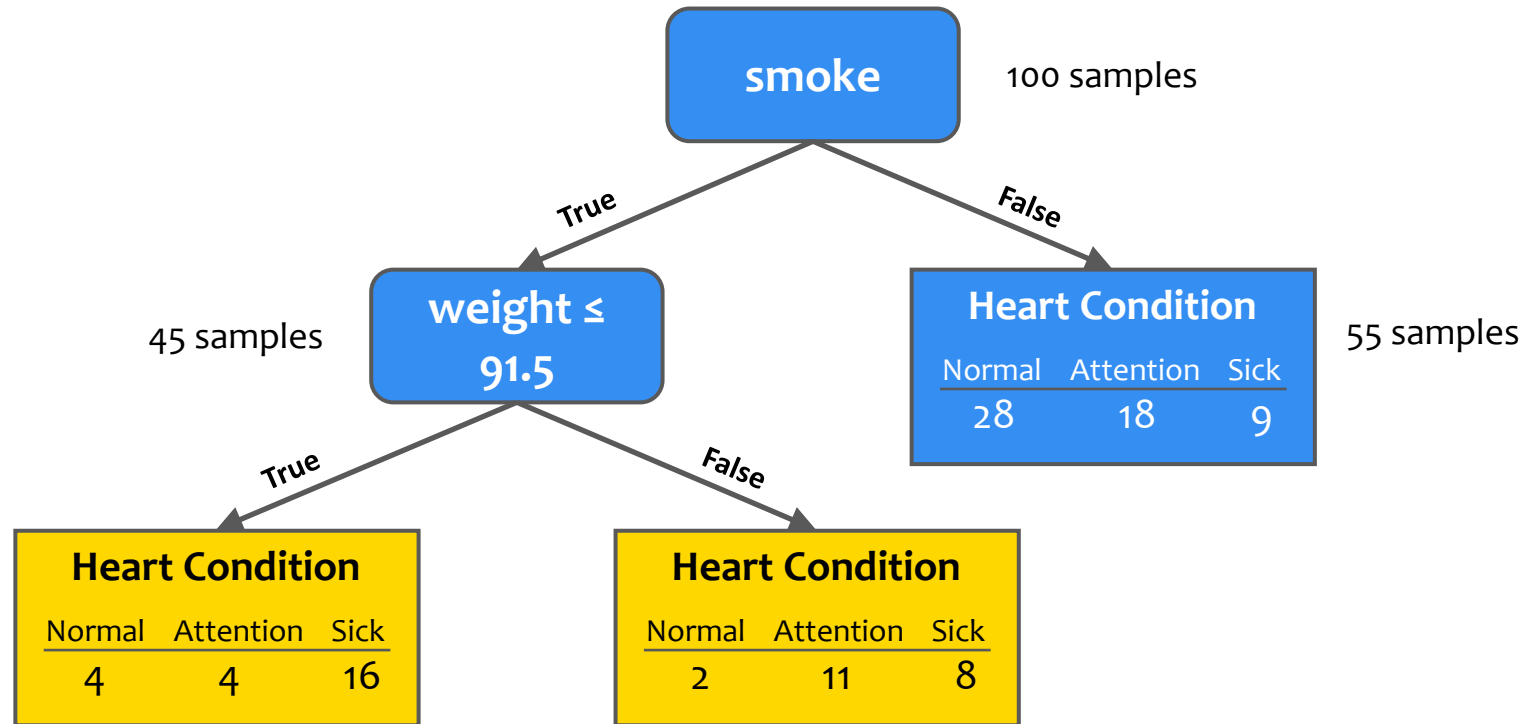
smoke	workout	weight	heart condition
No	Yes	85.0	Sick
No	Yes	78.0	Normal
Yes	No	75.0	Sick
No	Yes	84.0	Normal
No	Yes	72.0	Normal
No	No	78.0	Sick
Yes	No	120.0	Attention
...	...	...	...
Yes	Yes	84.0	Sick
No	No	98.0	Attention

$m = 100$ samples



smoke	workout	weight	heart condition
No	Yes	85.0	Sick
No	Yes	78.0	Normal
Yes	No	75.0	Sick
No	Yes	84.0	Normal
No	Yes	72.0	Normal
No	No	78.0	Sick
Yes	No	120.0	Attention
...	...	...	...
Yes	Yes	84.0	Sick
No	No	98.0	Attention

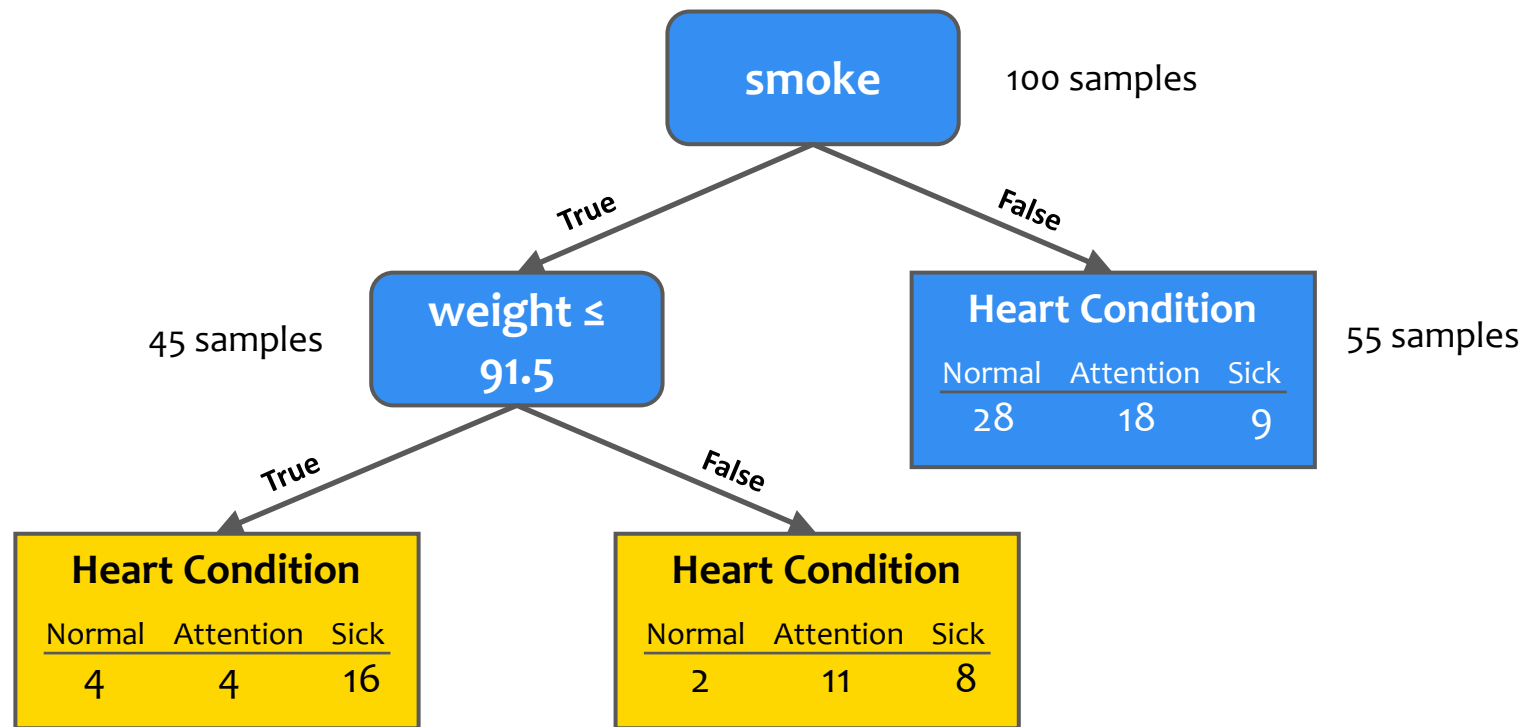
$m = 100$ samples



The **splitting process** continues until the algorithm reaches a **stop condition**.

smoke	workout	weight	heart condition
No	Yes	85.0	Sick
No	Yes	78.0	Normal
Yes	No	75.0	Sick
No	Yes	84.0	Normal
No	Yes	72.0	Normal
No	No	78.0	Sick
Yes	No	120.0	Attention
...	...	...	...
Yes	Yes	84.0	Sick
No	No	98.0	Attention

$m = 100$ samples



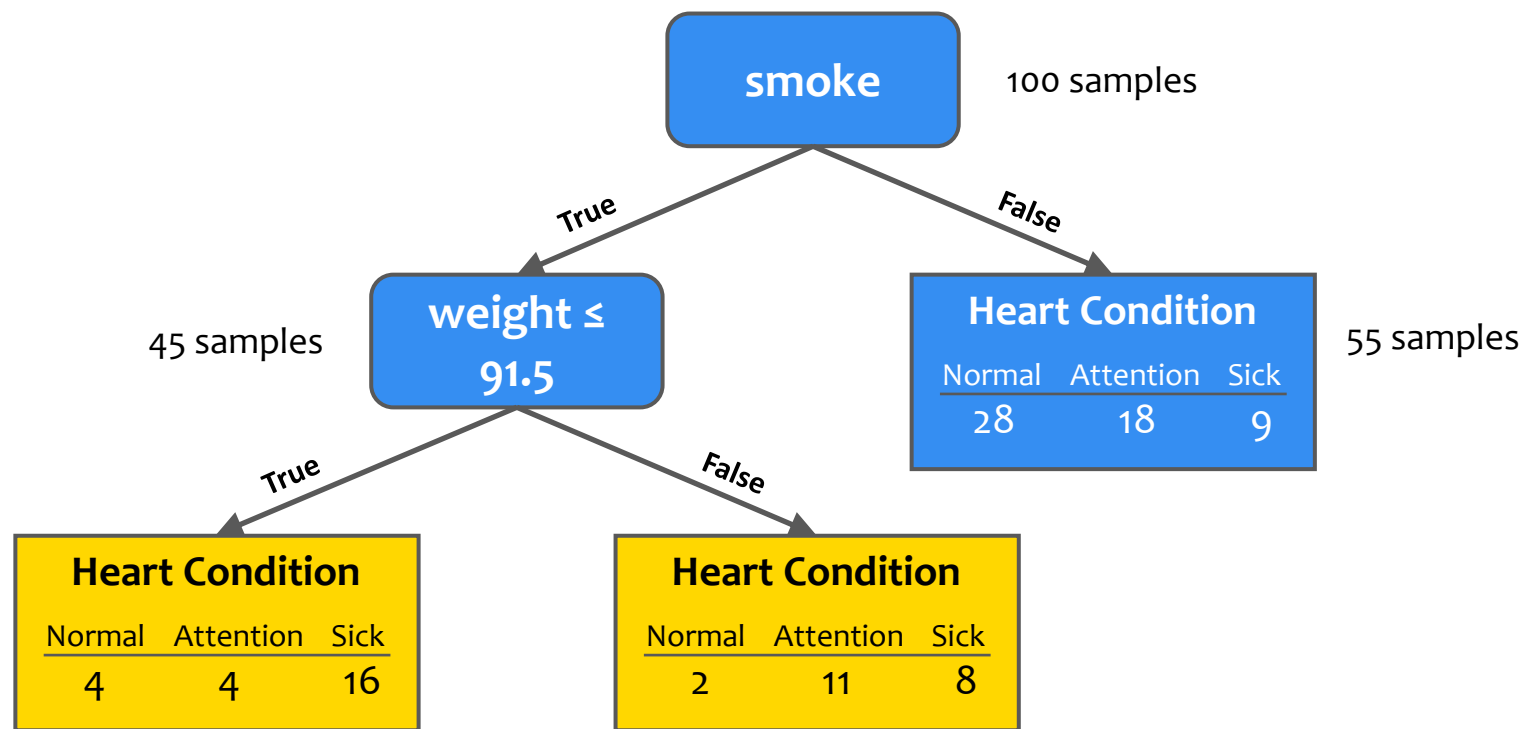
The **splitting process** continues until the algorithm reaches a **stop condition**.

Examples:

- All **leaves** are **pure**;
 - high chances of **overfitting**;
- **Maximum tree depth** (hyperparameter);
 - Regularization

smoke	workout	weight	heart condition
No	Yes	85.0	Sick
No	Yes	78.0	Normal
Yes	No	75.0	Sick
No	Yes	84.0	Normal
No	Yes	72.0	Normal
No	No	78.0	Sick
Yes	No	120.0	Attention
...	...	...	...
Yes	Yes	84.0	Sick
No	No	98.0	Attention

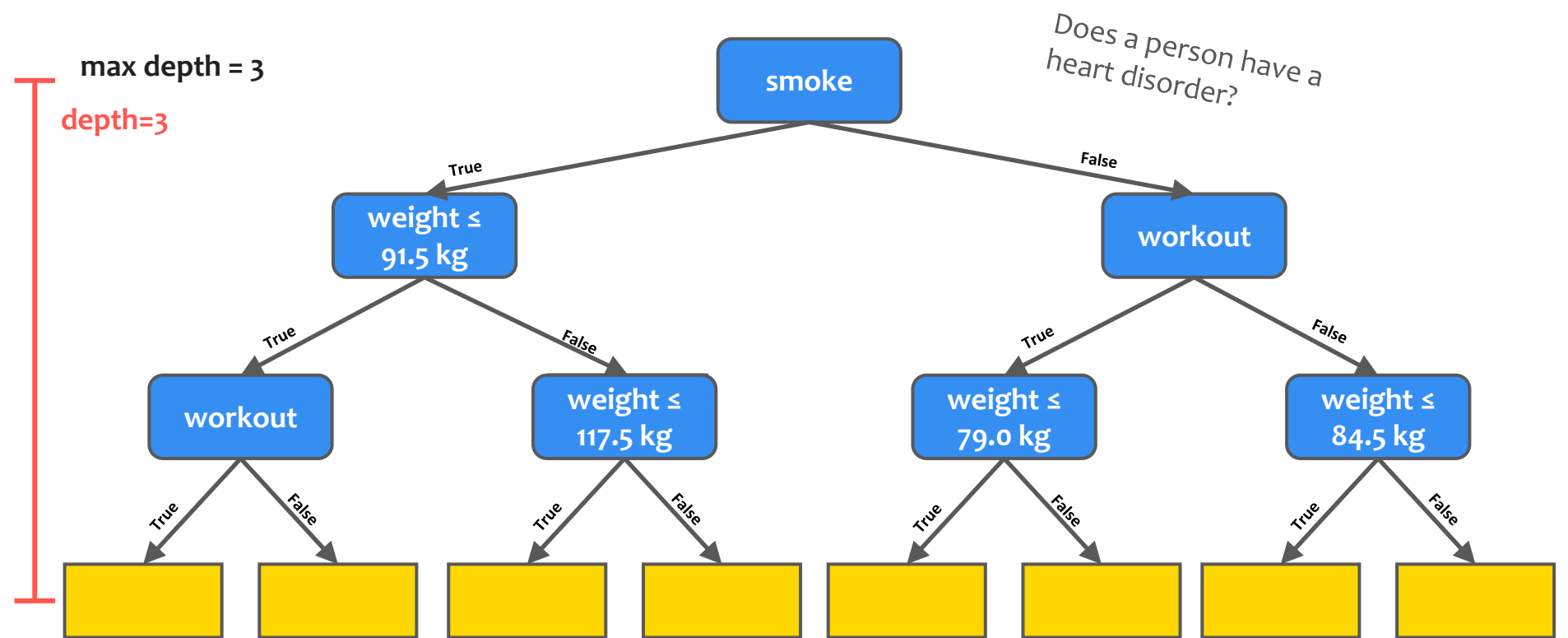
$m = 100$ samples



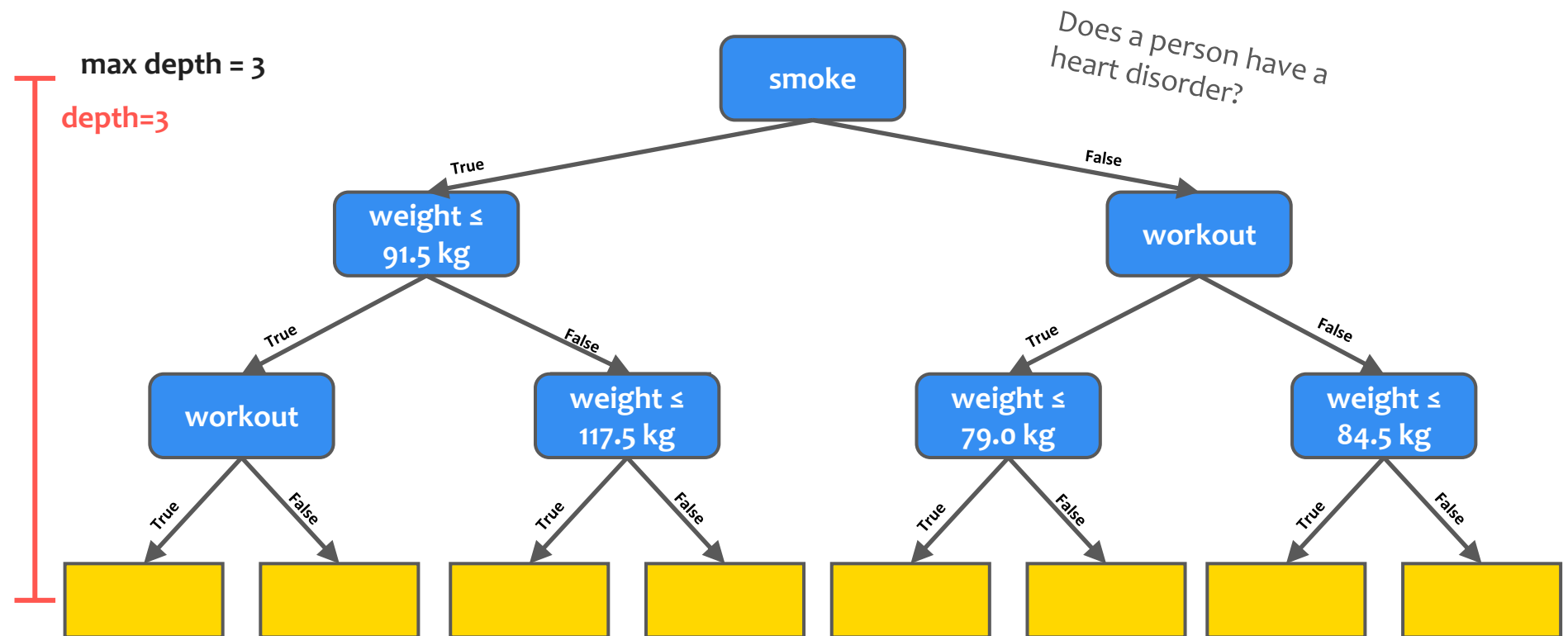
The **splitting process** continues until the algorithm reaches a **stop condition**.

Examples:

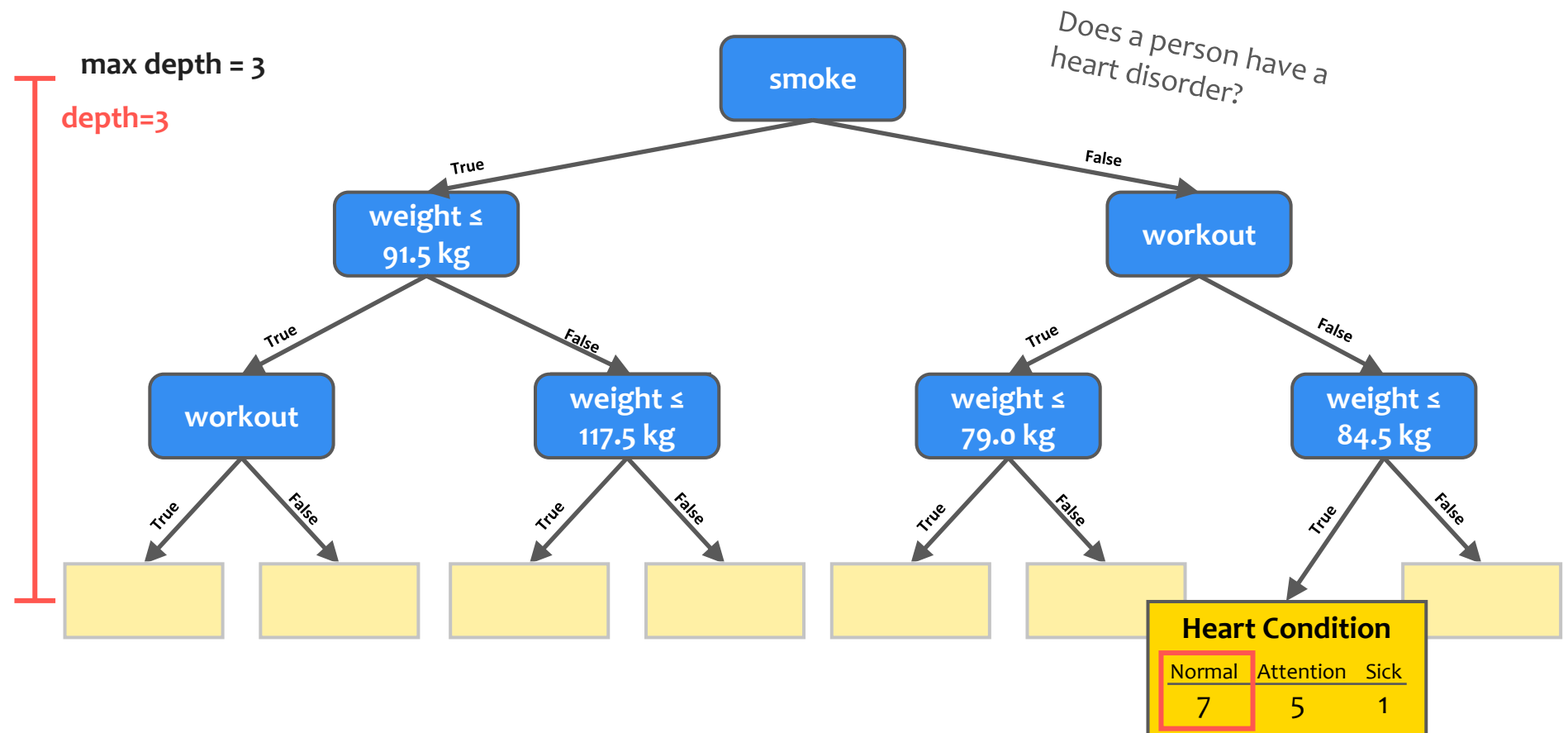
- All **leaves** are **pure**;
- high chances of **overfitting**;
- **Maximum tree depth** (hyperparameter);
- Regularization



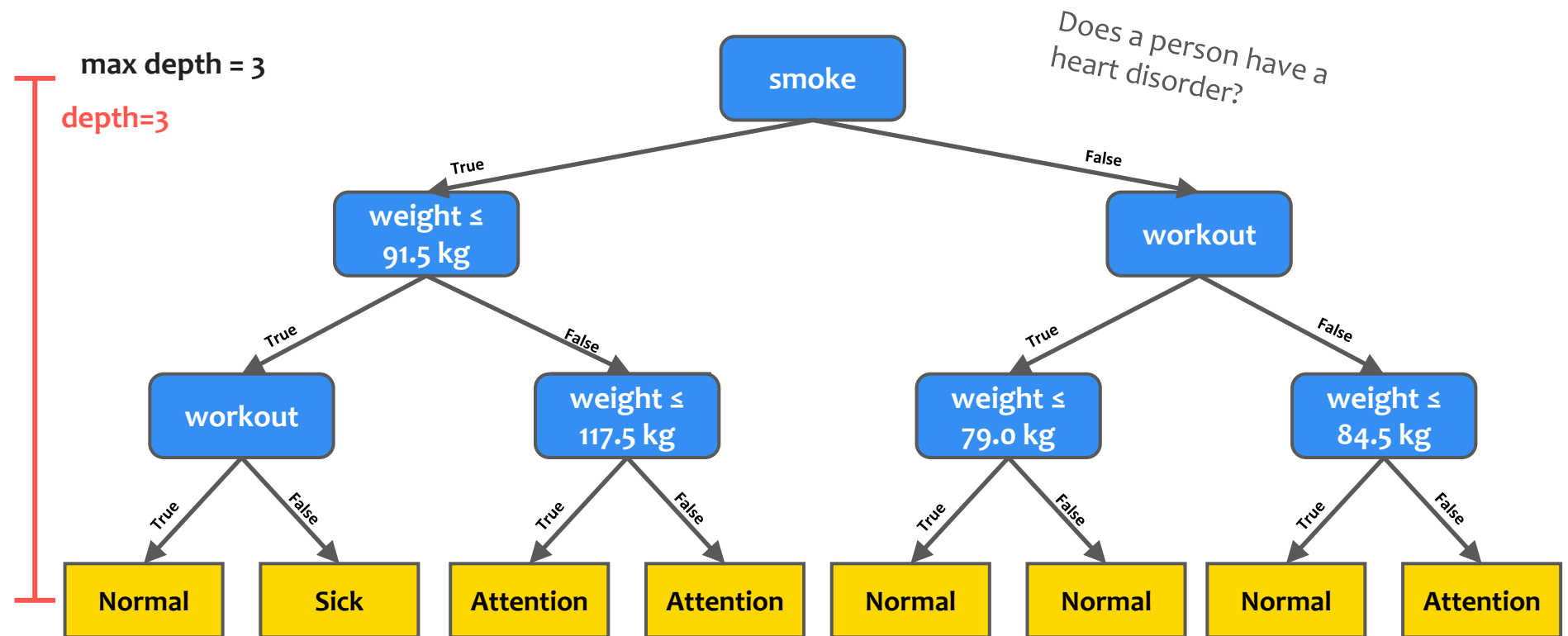
In **Classification Trees**, the **label/class** of each **leaf** is its **most frequent class**.



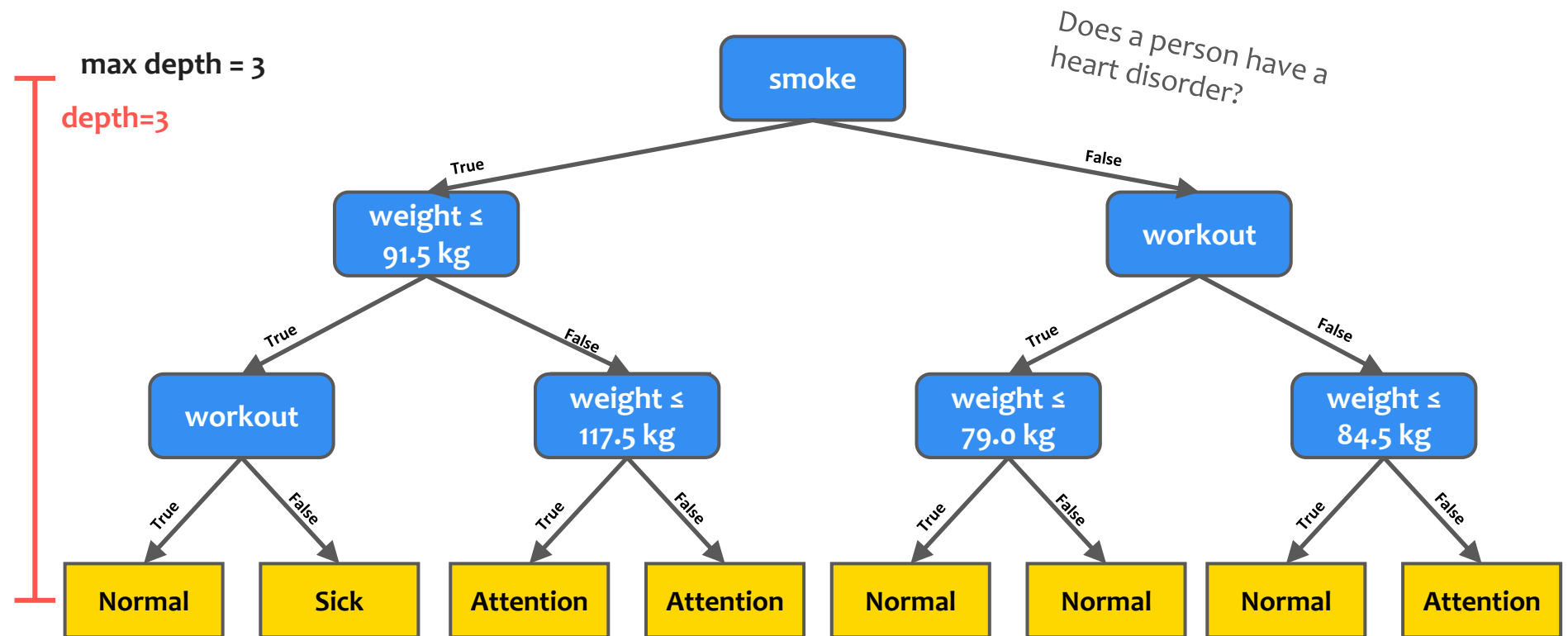
In **Classification Trees**, the **label/class** of each **leaf** is its **most frequent class**.



In **Classification Trees**, the **label/class** of each **leaf** is its **most frequent class**.

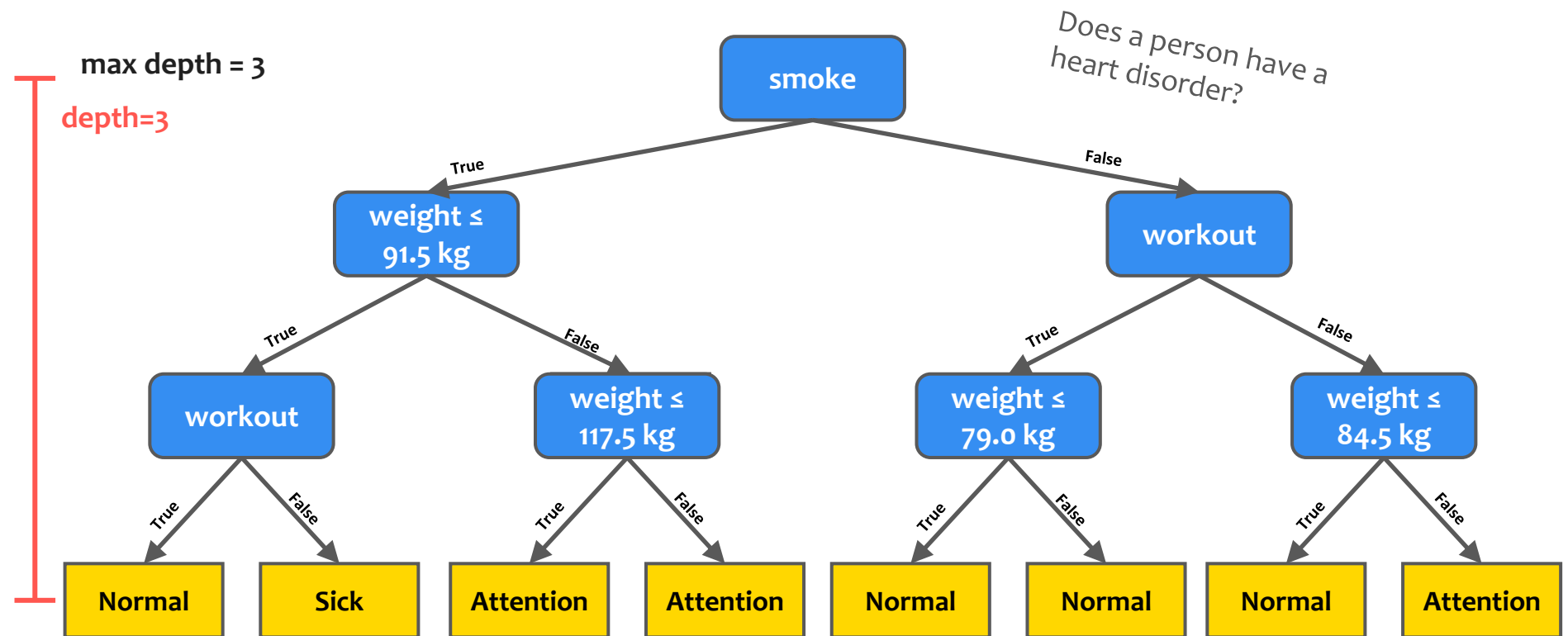


To **classify** a given instance, we just **traverse** the **Decision Tree** from the **root** to a **leaf** according to the **instance's features**.



To **classify** a given instance, we just **traverse** the **Decision Tree** from the **root** to a **leaf** according to the **instance's features**.

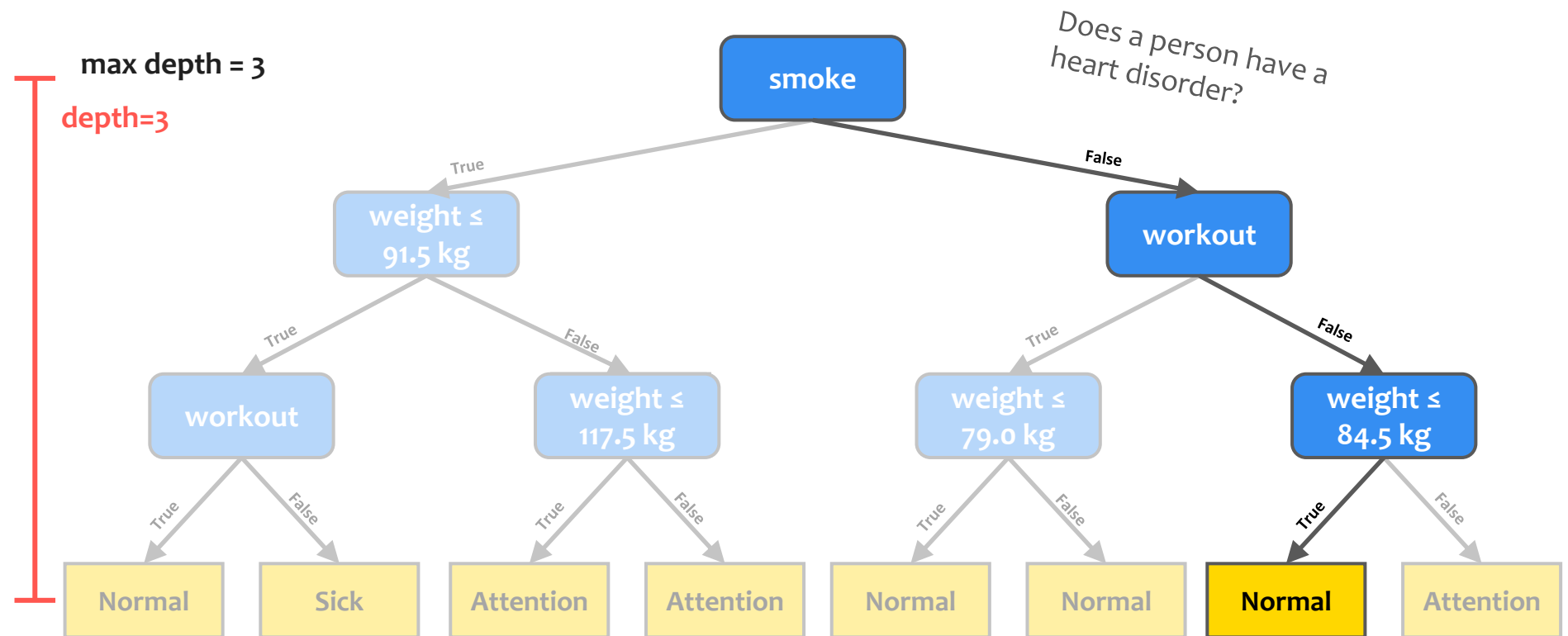
Weight: 80 kg
Smoke: No
Work out: No



To **classify** a given instance, we just **traverse** the **Decision Tree** from the **root** to a **leaf** according to the **instance's features**.

Weight: 80 kg
Smoke: No
Work out: No

→ **Normal**



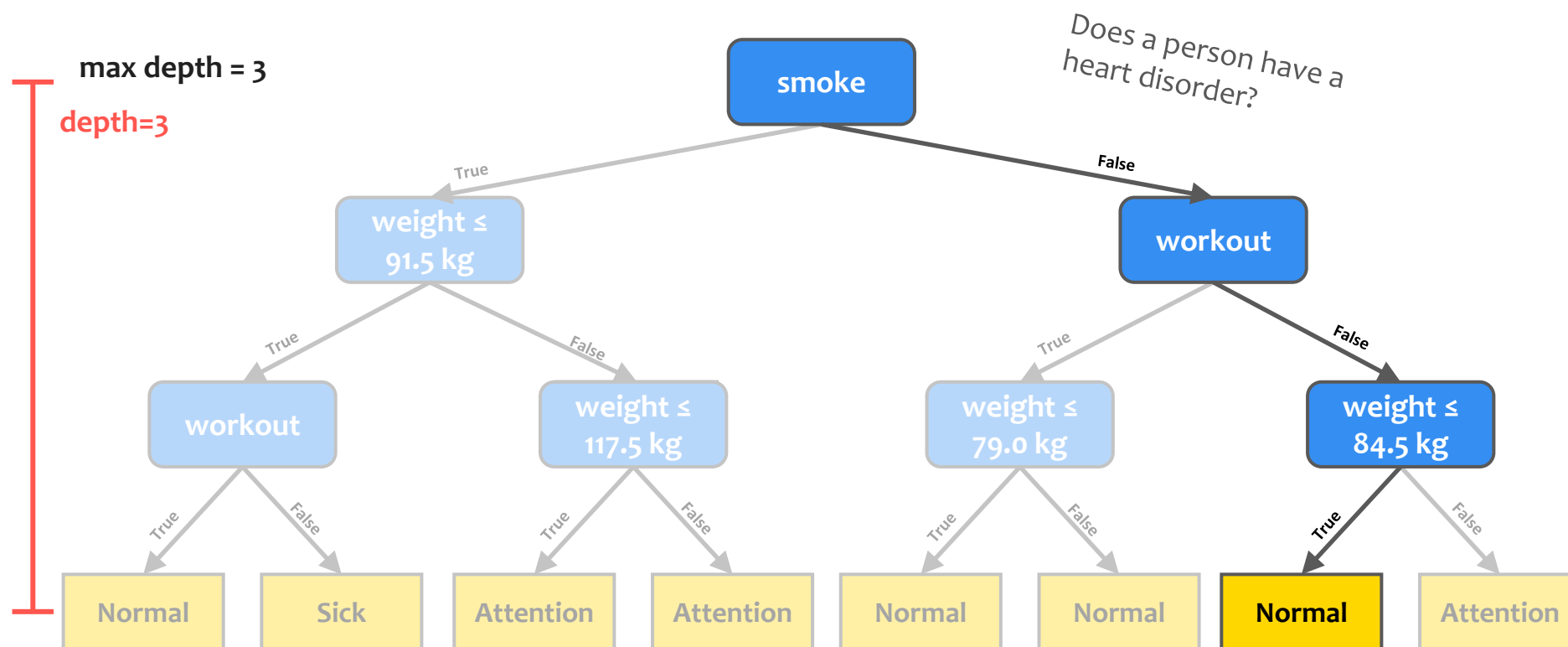
To **classify** a given instance, we just **traverse** the **Decision Tree** from the **root** to a **leaf** according to the **instance's features**.

Weight: 80 kg
Smoke: No
Work out: No

→ **Normal**

Decision Tree can also estimate the **probability** of an **instance** belongs to a particular **class c**.

This **probability** is the ratio of training samples of **class c** in the **leaf node** found for the **instance**.



To **classify** a given instance, we just **traverse** the **Decision Tree** from the **root** to a **leaf** according to the **instance's features**.

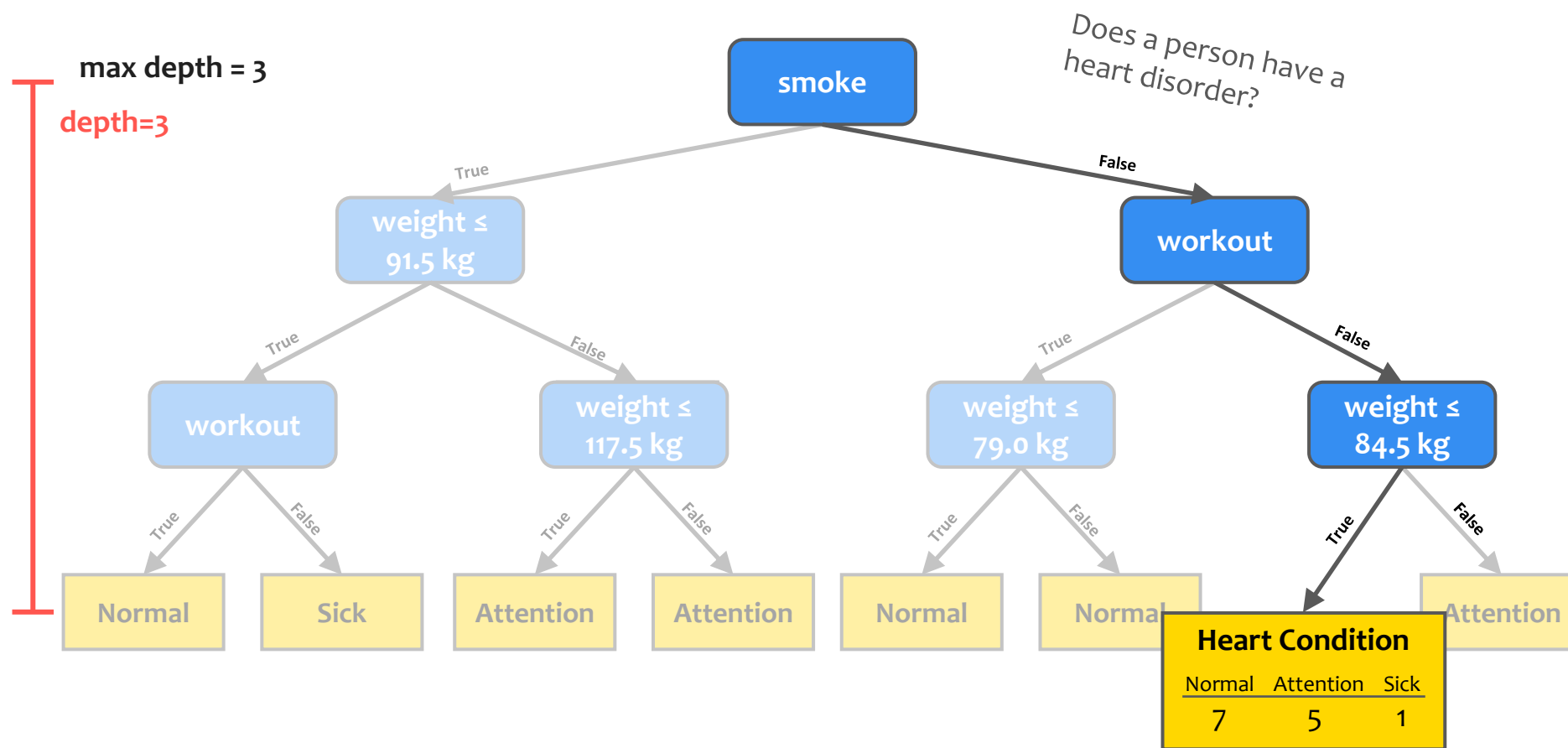
Weight: 80 kg
Smoke: No
Work out: No

Normal

$$P_{Normal} = \frac{7}{13} = 0.538$$

$$P_{Attention} = \frac{5}{13} = 0.384$$

$$P_{Sick} = \frac{1}{13} = 0.076$$



Decision Tree can also estimate the **probability** of an **instance** belongs to a particular **class c**.

This **probability** is the ratio of training samples of **class c** in the **leaf node** found for the **instance**.

Codes

sklearn.tree.DecisionTreeClassifier

```
from sklearn.tree import DecisionTreeClassifier  
  
tree_clf = DecisionTreeClassifier(max_depth=3)  
tree_clf.fit(X, y)
```



Scikit-learn implements the **CART** algorithm for **Decision Tree**.

Codes

sklearn.tree.DecisionTreeClassifier

```
from sklearn.tree import DecisionTreeClassifier  
  
tree_clf = DecisionTreeClassifier(max_depth=3)  
tree_clf.fit(X, y)
```



Scikit-learn implements the **CART** algorithm for **Decision Tree**.

default=None

it expands the **nodes** until **all leaves** are **pure** or until **all leaves** contain less than a *min_samples_split* samples.

Codes

sklearn.tree.DecisionTreeClassifier

```
from sklearn.tree import DecisionTreeClassifier

tree_clf = DecisionTreeClassifier(max_depth=3)
tree_clf.fit(X, y)
```



The default **cost function** (criterion) is based on **Gini Impurity**, but **Entropy** is also supported.



Scikit-learn implements the **CART** algorithm for **Decision Tree**.

Codes

sklearn.tree.DecisionTreeClassifier



Scikit-learn implements the **CART** algorithm for **Decision Tree**.

```
from sklearn.tree import DecisionTreeClassifier  
  
tree_clf = DecisionTreeClassifier(max_depth=3)  
tree_clf.fit(X, y)
```

```
tree_clf.predict(X_test)
```

Classification/prediction

```
tree_clf.predict_proba(X_test)
```

Predicting class probabilities

Codes

sklearn.tree.DecisionTreeClassifier



Scikit-learn implements the **CART** algorithm for **Decision Tree**.

```
from sklearn.tree import DecisionTreeClassifier

tree_clf = DecisionTreeClassifier(max_depth=3)
tree_clf.fit(X, y)
```

To visualize a trained **Decision Tree**:

```
from sklearn import tree

tree.plot_tree(tree_clf, feature_names=['smoke', 'workout', 'weight'],
               class_names=['normal', 'attention', 'sick'], filled=True,
               rounded=True)
```



We need to install **graphviz** first.

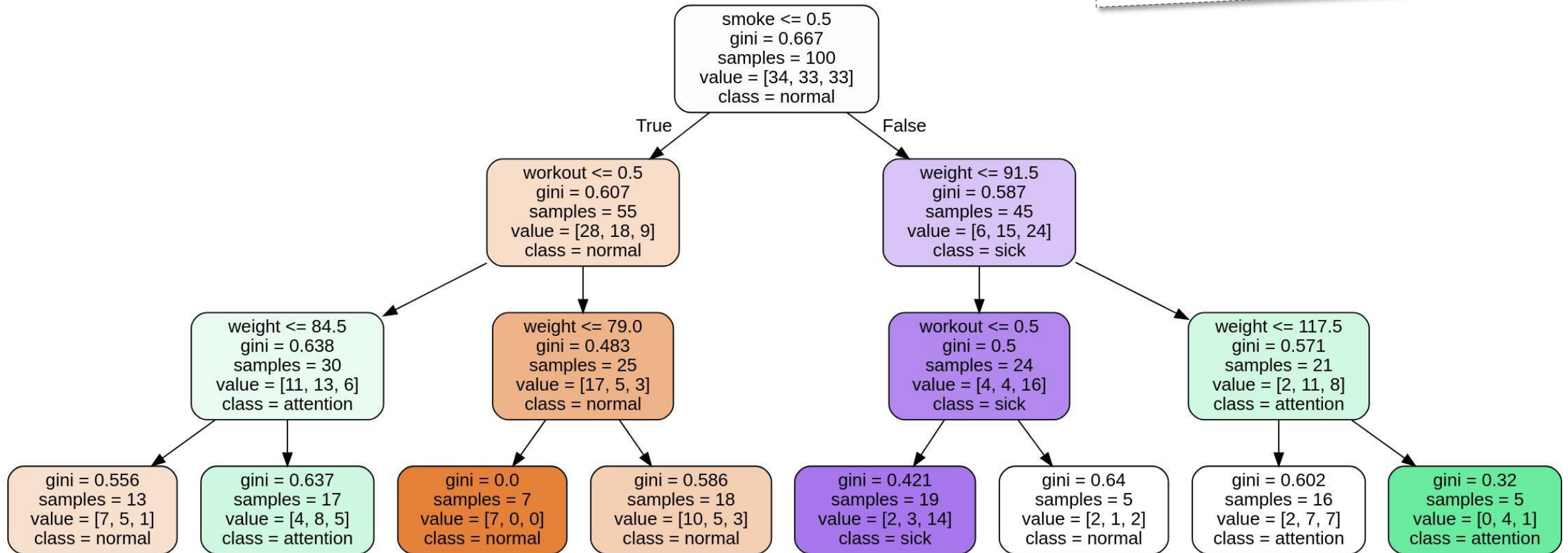
```
import graphviz
from sklearn import tree

dot_data = tree.export_graphviz(tree_clf, out_file=None,
                                feature_names=['smoke', 'workout', 'weight'],
                                class_names=['normal', 'attention', 'sick'],
                                filled=True, rounded=True)

graph = graphviz.Source(dot_data, format="png")
graph
```

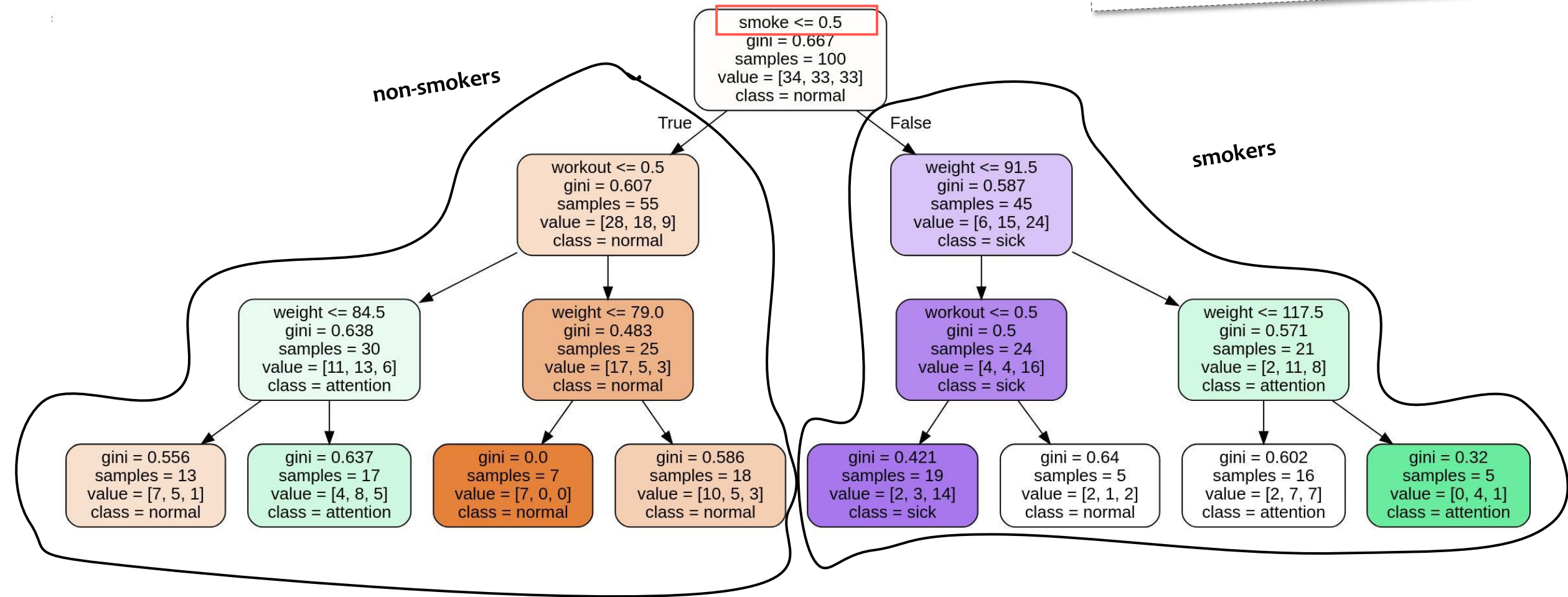


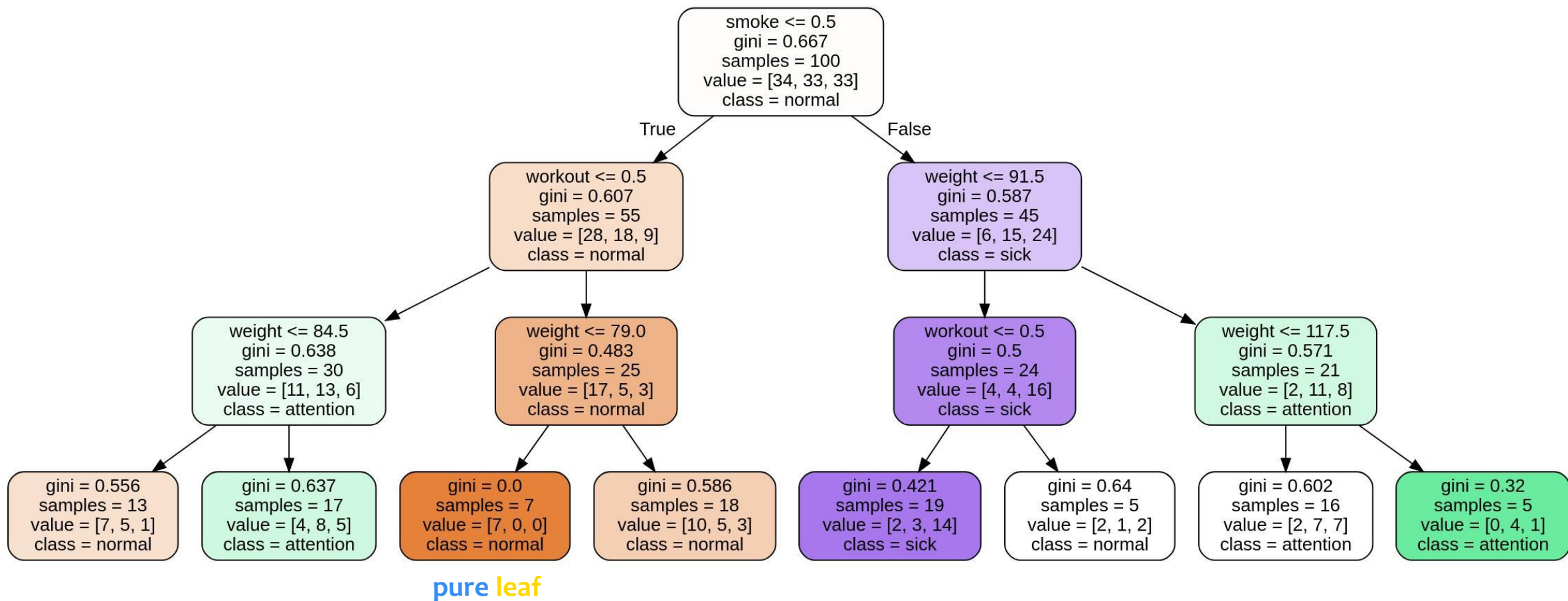
Scikit-learn Decision Tree
treat **all** variables as **numeric!**





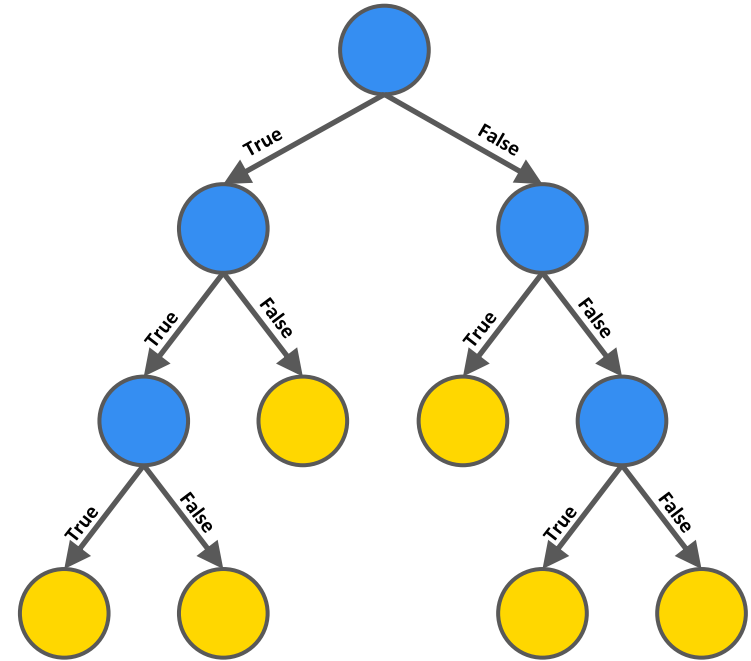
Scikit-learn Decision Tree
treat **all** variables as **numeric**!

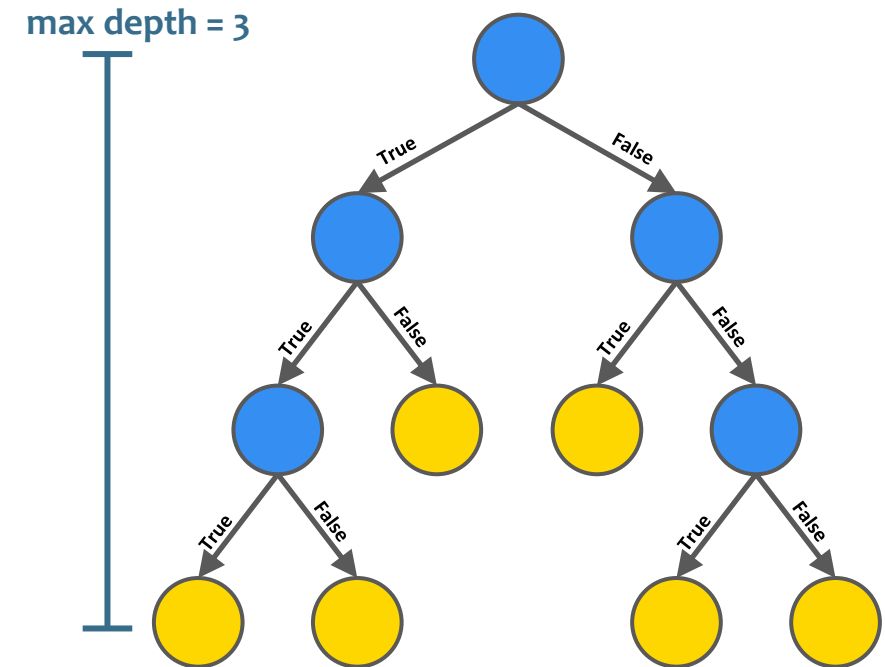




Regularization Parameters

- To avoid **overfitting** the training data, you need to **restrict** the **Decision Tree's freedom** during training (**Regularization**).



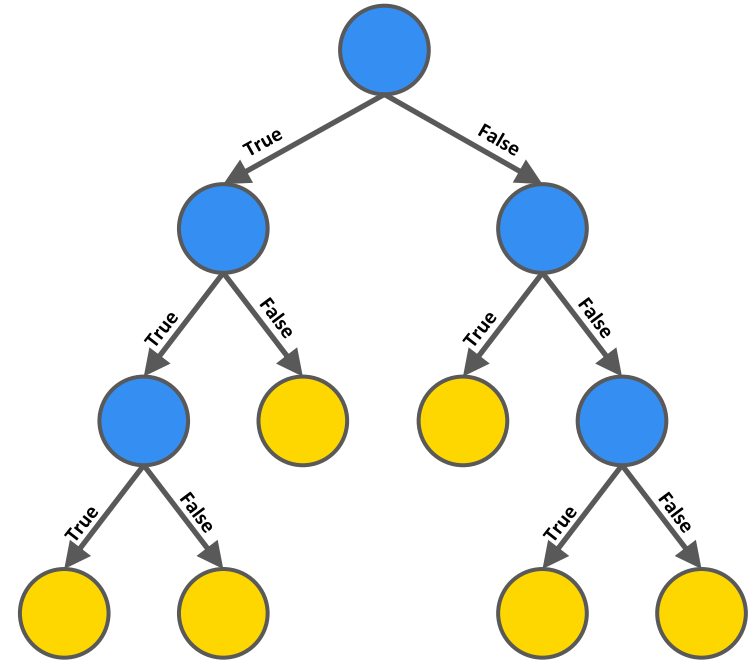


-  `max_depth` is a **hyperparameter** that can be **optimized** by a tuning technique.



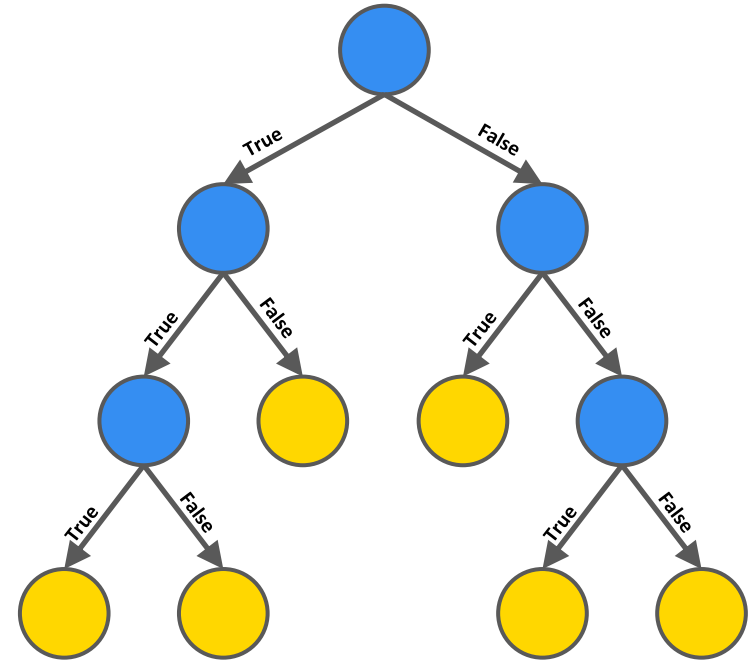
Regularization Parameters

- To avoid **overfitting** the training data, you need to **restrict** the **Decision Tree's freedom** during training (**Regularization**).
- The common strategy is to **restrict** the **maximum depth** of the **Decision Tree**.
 - `max_depth` in sklearn
- Reducing `max_depth` will **regularize** the model and thus **reduce** the risk of **overfitting**.
- The **Scikit-learn DT** has a few other hyperparameters that similarly **restrict** the **DT's shape**:
 - `min_samples_split`, `min_samples_leaf`, `min_weight_fraction_leaf`;
 - `max_leaf_nodes`, `max_features`;



Regularization Parameters

- To avoid **overfitting** the training data, you need to **restrict** the **Decision Tree's freedom** during training (**Regularization**).
- The common strategy is to **restrict** the **maximum depth** of the **Decision Tree**.
 - `max_depth` in sklearn
- Reducing `max_depth` will **regularize** the model and thus **reduce** the risk of **overfitting**.
- The **Scikit-learn DT** has a few other hyperparameters that similarly **restrict** the **DT's shape**:
 - `min_samples_split`, `min_samples_leaf`, `min_weight_fraction_leaf`;
 - `max_leaf_nodes`, `max_features`;
- **↑Increasing** `min_*` hyperparameters or **↓reducing** `max_*` hyperparameters will **regularize** the model.

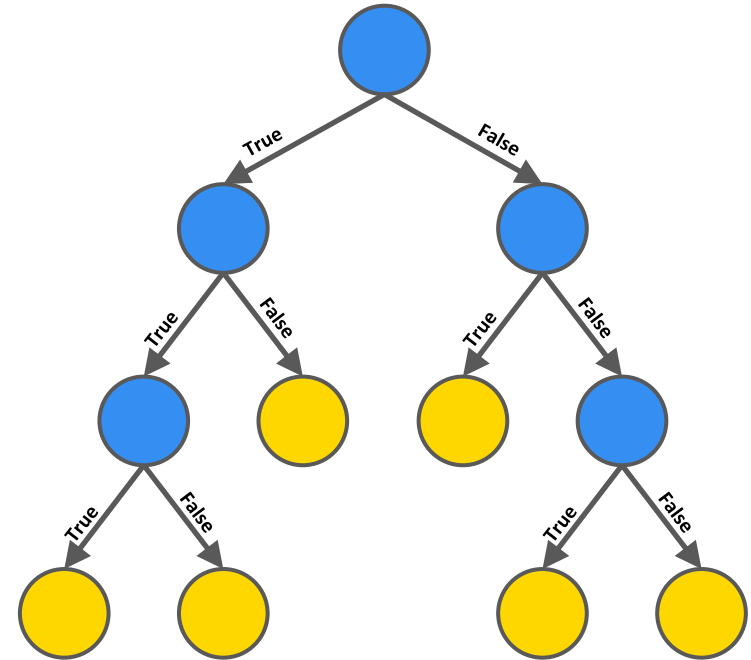


Regularization Parameters

- To avoid **overfitting** the training data, you need to **restrict** the **Decision Tree's freedom** during training (**Regularization**).
- The common strategy is to **restrict the maximum depth** of the **Decision Tree**.
 - `max_depth` in sklearn
- Reducing `max_depth` will **regularize** the model and thus **reduce** the risk of **overfitting**.
- The **Scikit-learn DT** has a few other hyperparameters that similarly **restrict** the **DT's shape**:
 - `min_samples_split`, `min_samples_leaf`, `min_weight_fraction_leaf`;
 - `max_leaf_nodes`, `max_features`;
- ↑Increasing `min_*` hyperparameters or ↓reducing `max_*` hyperparameters will **regularize** the model.



Other algorithms work by first training the **Decision Tree** without restrictions, then **pruning (deleting)** unnecessary nodes.



Decision Trees

Pros

- Easy to **understand** and **interpret**, perfect for visual representation (white box model).
- Requires **little data preprocessing**:
 - no need for one-hot encoding, dummy variables, and so on.
- **Non-parametric model**: no assumptions about the shape of data.
- **Feature selection** happens automatically:
 - unimportant features will not influence the result;
 - the presence of features that depend on each other (multicollinearity) also does not affect the quality.

Decision Trees

Cons

- A **small change** in the data can cause a **large change** in the **structure** of the decision tree causing **instability**;
- Sometimes calculation can go far more complex compared to other algorithms;
- It often involves **higher time to train** the model;
- Decision Tree is **often inadequate** for applying **regression** and predicting continuous values.

Inteligência Artificial



Decision Trees

Prof. Dr. Samuel Martins (Samuka)
samuel.martins@ifsp.edu.br

